

Generative AI and Large Language Models for Financial Markets: From Behavioral Prediction to Autonomous Trading

Zangir Iklassov Hachem Madmoun Jean-Jacques Duhot Salem Lahlou

MBZUAI, Abu Dhabi, UAE & Brevan Howard Asset Management, London, UK

KDD 2026 Tutorial

Three Audiences — All Welcome

You don't need a finance degree or an AI PhD. If you are curious and willing to think carefully, this tutorial is for you.

ML Researchers

- You know machine learning but not finance
- You want **real-world** problems that push the frontier
- Finance = adversarial, non-stationary, high-stakes

Quant Practitioners

- You know markets but want to understand Large Language Models (LLMs) and agents deeply
- You want to know what hype is real vs. useful
- Practical pipelines and pitfalls highlighted

Policy People

- You govern or regulate financial AI
- You want to understand what these systems actually do
- Ethics, risk, and systemic concerns covered in depth

Three Learning Outcomes

By the end of this tutorial, you will be able to:

1. **Understand** how financial markets work from the ground up — including the human behaviors and information flows that create tradable signals.
2. **Build** and critically evaluate modern AI systems for finance — from FinBERT sentiment pipelines to RAG systems to multi-agent trading frameworks.
3. **Assess** the risks — statistical, financial, ethical, and systemic — that arise when powerful AI is deployed in markets that affect everyone.

Our Promise

Every concept is explained from scratch. We never say “as you know...” about something we haven’t taught you yet.

Part 1 (60 min) Foundations (<i>Iklassov / Madmoun / Lahlou</i>)	Part 2 (60 min) Advanced Techniques (<i>Lahlou / Iklassov</i>)	Part 3 (60 min) Robustness & Ethics (<i>Madmoun / All</i>)
• Markets & microstructure (15 + 20 min) • Demo 1: LOBSTER live (10 min) • Behavioral finance (5 min) • Financial NLP + LLM bridge (10 min) <i>Slides 9–98</i>	• LLM primer + RAG + Demo 2: live pipeline (25 min) • Agentic AI (20 min) • Multi-agent systems (15 min) <i>Slides 99–188</i>	• Rigorous backtesting (25 min) • Risk & execution (10 min) • Ethics & governance (25 min) • Open questions & Q&A <i>Slides 189–262</i>

Two Live Demos + Three Optional Colab Notebooks

Demo 1: LOBSTER order-book visualizer. Demo 2: End-to-end RAG pipeline on an earnings call. Notebooks: FinBERT fine-tuning, agent conflict resolution, backtest repair.

The Big Question

Why can't you just throw a neural network at stock prices?

Because markets are not random physics problems — they are **ecosystems of human behavior**.

- **Section A:** Markets explained from scratch — what they are, why they exist, and what makes them hard
- **Section B:** Market microstructure — the plumbing inside a stock exchange, order books, and the Flash Crash
- **Section C:** Behavioral finance — how human biases create predictable patterns
- **Section D:** Financial NLP — turning text (news, earnings calls, filings) into signals

Key Insight — The Tutorial's Central Thesis

The best AI for finance models *people*, not just prices. Prices are just the footprints people leave.

Why this matters: behavioral patterns are *durable* (rooted in human psychology), *explainable* (we know why they exist), and *text-measurable* (LLMs read the behavior directly). Price-only models have none of these properties.

The Big Question

How do you build an AI system that reasons about markets?

- **Section E:** LLMs from first principles — transformers, attention, scaling, prompting, hallucination
- **Section F:** Retrieval-Augmented Generation (RAG) — grounding LLMs in real facts to prevent hallucination
- **Section G:** Agentic AI — AI that plans, uses tools, and takes sequences of actions
- **Section H:** Multi-agent systems — teams of AI agents with specialization, debate, and conflict resolution

Reality Check

Most exciting demos break in production. We will show you both the possibilities *and* the failure modes.

Story Link

Part 1 showed *why* behavior matters and *what* signals encode it. Part 2 asks: **how do we build a system that processes these signals at scale, stays grounded in facts, and takes actions?**

The Big Question

How do you know if it works — and whether it should?

- **Section I:** Rigorous backtesting — the many ways historical tests lie, and how to build honest ones
- **Section J:** Risk management and execution — the gap between paper returns and real-world trading
- **Section K:** Ethics and governance — three deep tensions in deploying AI in markets, regulation landscape

For Builders

These sections will save you from costly mistakes in research and production.

For Policy People

This section is especially relevant — we discuss FSB, EU AI Act, MiFID II, and SEC guidance.

Story Link

Part 2 built the system. Part 3 asks: **does it actually work in the real world, and should we deploy it?** Two questions that are almost never asked together — and must be.

Color Coding

- **Blue boxes** = definitions and key concepts
- **Gold boxes** = concrete examples and demos
- **Red boxes** = common mistakes and warnings

Notation

- **Bold text** = important term being defined
- *Italic* = analogy or informal explanation
- ★ *starred examples* = results from published studies

Everything Is Self-Contained

Every term is defined when first used. If you see a word you don't know, it was either just explained or is about to be.

Don't Skip Sections A–D

The advanced material in Parts 2 and 3 builds directly on Part 1. The foundations are not optional.

Part 1

Foundations

Section A: What Are Financial Markets?

Slides 9–98

Presenters: Iklassov (framing) • Madmoun (microstructure & behavior) • Lahlou (NLP bridge)

Financial markets move because of human behavior.

LLMs read behavior in text.

RAG keeps them honest.

Agents act on grounded analysis.

Rigorous testing proves it works.

Ethics governs what we should deploy.

The Red Thread Through Every Slide

Every concept in this tutorial connects to this chain. When you feel lost, come back here.

Part 1: Why behavior drives prices, and how text encodes behavior (Tetlock → FinBERT).

Part 2: How LLMs + RAG read behavior at scale, and agents act on it (grounded, not hallucinated).

Part 3: How to prove it works honestly, deploy it safely, and govern it responsibly.

Definition: Market

A **market** is any place — physical or virtual — where buyers and sellers come together to exchange things they have for things they want.

The Farmer's Market Analogy

Imagine a Saturday farmer's market. A baker has bread she made. You are hungry and have money. She wants \$3 for a loaf. You are willing to pay up to \$4. You agree on \$3.50. **That is a trade.** The market is just the space where this happens efficiently.

- Markets solve a **coordination problem**: how do millions of strangers agree on prices?
- The answer: *competition*. Many buyers and sellers competing drives prices toward fair value
- Markets are ancient — bazaars, trading posts, grain exchanges all predate computers by millennia

Definition: Financial Market

A **financial market** is a market where people buy and sell *financial instruments* — not bread or tomatoes, but **claims on future money**.

Stocks	Bonds	Currencies (FX)
A stock is a small ownership slice of a company. You buy Apple stock, you own a tiny piece of Apple.	A bond is a loan. You lend money to a company or government; they pay you back with interest.	The foreign exchange (FX) market is where you swap one country's money for another's. Tourists do this; banks do it at massive scale.

- Also: commodities (oil, gold), derivatives (options, futures), crypto, real estate investment trusts...
- Financial markets are **huge**: global stock markets alone exceed \$100 trillion in value

Definition: Stock (Equity)

A **stock** (also called a **share** or **equity**) represents partial ownership of a company. If a company has 1,000 shares and you own 100, you own 10% of that company.

Pizza Shop Analogy

You and 3 friends start a pizza shop. You each put in \$1,000 and agree you each own 25%. You've just created 4 "shares" worth \$1,000 each. If the shop later makes profit, each share is worth more. If it fails, shares are worth less. That's exactly how publicly listed companies work — just with millions of shareholders.

- **Dividend:** some companies pay shareholders a portion of profits regularly
- **Capital gain:** if you sell your shares for more than you paid, you made a profit
- **Stock exchange:** the marketplace where shares are bought and sold (NYSE, NASDAQ, LSE...)

What Is a Price?

Definition: Price

The **price** of a stock at any moment is simply the value at which the most recent transaction between a willing buyer and a willing seller occurred.

- Price is *not* intrinsic value — it is what someone just paid
- Price is *not* what you think it is worth — it is what another person paid
- Price is the result of **agreement** between two people at a specific moment

Common Mistake

“The price went up because the company is doing well.”
Sometimes — but prices also move on rumors, sentiment, and trader behavior that has nothing to do with fundamentals.

eBay Analogy

Think of buying something on eBay. If two people really want the same rare item, they bid it up far above what it cost to make. If nobody wants it, it sells for almost nothing. The price is where a buyer and seller finally agree — and it changes every time someone new makes a deal.

The Fundamental Law of Markets

When more people **want to buy** a stock than sell it, the price goes **up**. When more people want to **sell** than buy, the price goes **down**.

The Seesaw Analogy

Imagine a seesaw. Buyers sit on the left side, sellers on the right. When more buyers pile on, the left side goes down... wait, that means *prices go up*? Yes — because buyers are bidding the price higher trying to attract sellers. Add more sellers, price drops as they compete for buyers.

Demand increases when:

- Company reports great earnings
- Analyst upgrades the stock
- Positive news coverage
- Investor sentiment improves

Supply increases when:

- Large shareholders want to exit
- Insiders sell after lockup expires
- Negative news arrives
- Fear spreads through the market

The Efficient Market Hypothesis (simplified)

Prices change because **new information arrives** that changes what people think a stock is worth. In a perfectly efficient market, prices instantly reflect all available information.

Step-by-Step: An Earnings Surprise

1. Apple announces earnings after market close
 2. Analysts had expected \$2.00 profit per share
 3. Apple reports \$2.45 — a 22.5% beat
 4. Millions of investors simultaneously update beliefs
 5. More people want to buy than sell
 6. Opening price gaps up 8% the next morning
- Not just earnings: *any* new information can move prices — Fed announcements, geopolitical events, CEO tweets, weather forecasts affecting crops...
 - **Key insight for AI:** if we can process information *faster* or *better* than others, we can predict price moves

Two Types of Market Participants

Retail Investors

- Individual people investing their own money (you, your parents)
- Smaller amounts: \$100 to \$100,000 typically
- Trade infrequently, often via apps like Robinhood or TD Ameritrade
- Less information, less sophisticated tools
- Collectively: a significant force (especially since 2020)

Institutional Investors

- Organizations: hedge funds, pension funds, mutual funds, banks
- Huge amounts: \$1 billion to \$1 trillion+
- Trade constantly, with teams of analysts and algorithms
- Access to expensive data, sophisticated models
- **Hedge funds** like Brevan Howard are at the cutting edge

The Power Asymmetry

Institutional investors have *vast* advantages in speed, data, and computing power. AI is increasingly relevant because it helps both sides — and changes the competitive dynamics.

What Is “Trading”?

Definition: Trading

Trading is the act of buying and selling financial instruments with the goal of making a profit from price changes, rather than long-term ownership.

Investing vs. Trading

Investing: Buy Apple stock and hold it for 10 years, believing the company will grow.

Trading: Buy Apple stock Monday morning, sell it Thursday afternoon, because you predicted a short-term price move.

- **Day trading:** open and close all positions in a single day
- **Swing trading:** hold positions for days to weeks
- **Position trading:** hold for weeks to months
- **High-frequency trading (HFT):** hold for microseconds to milliseconds

- Trading provides **liquidity** (we'll define this soon) to markets — making it easier for everyone to buy and sell
- The AI systems in this tutorial are primarily about *trading* rather than long-term investing

Going Long

You **buy** a stock hoping it will go **up** in price, then sell it later for a profit.

Normal intuition: buy low, sell high.

Maximum loss: you lose what you paid (stock goes to zero).

Going Short

You **borrow** a stock from someone, **sell** it immediately at today's price, hoping it goes **down**, then buy it back cheaper and return it.

Profit = price you sold at minus price you bought back at.

The Shoe Analogy

Your friend has expensive sneakers worth \$200. You think the price will drop.

Step 1: Borrow the sneakers, sell them to a stranger for \$200.

Step 2: Wait. Price drops to \$120.

Step 3: Buy new identical sneakers for \$120, return them to your friend.

Profit: $\$200 - \$120 = \$80$ (minus borrowing fee).

Short Selling Risk

If the price *rises*, you lose money — potentially unlimited losses!

What Is Liquidity?

Definition: Liquidity

Liquidity is how easily and quickly you can buy or sell an asset *without significantly changing its price*. A highly liquid asset can be bought or sold instantly at a fair price.

High Liquidity

Apple stock (AAPL) trades **millions of shares per minute**. If you want to sell 100 shares, someone is always ready to buy them at the market price. Fast, cheap, no price impact.

Low Liquidity

A small company listed on a minor exchange might trade only **1,000 shares per day**. If you try to sell 5,000 shares, you'd have to lower your price significantly to find enough buyers.

- **Liquid markets:** stocks of large companies (Apple, Amazon, Microsoft), major currency pairs (EUR/USD)
- **Illiquid markets:** penny stocks, rare bonds, some crypto tokens, real estate
- Liquidity is *not constant* — it can dry up suddenly in a crisis (we'll see this in the Flash Crash)

The Thin Ice Analogy

Imagine a frozen lake. In winter, the ice is thick — you can drive a truck across it safely. This is a **liquid market**: you can trade large amounts without breaking through.

In spring, the ice thins. One person can walk across, but if everyone tries at once, everyone falls through. This is an **illiquid market in a crisis**: when everyone tries to sell at the same time, prices collapse.

Why liquidity matters for AI trading:

- AI strategies that work in backtests assume you can trade at the observed price
- In reality, large AI-driven trades move the market against you
- Illiquid markets make strategies **impossible to execute**

Why liquidity matters for risk:

- If you can't exit a position, you can't cut your losses
- Correlated AI strategies may all try to exit simultaneously
- This creates systemic risk (covered in Section K)

Markets Are Not Like Other ML Problems

In most ML problems, the data distribution is stable. In markets, *the act of finding a pattern destroys it*.

Challenge 1: Non-Stationarity

Patterns that worked last year may not work next year. The market “regime” changes: bull markets, bear markets, crises, booms. Your model trained on 2010–2019 may fail completely in 2020.

Challenge 2: Adversarial Environment

Unlike MNIST (handwritten digits don't adapt), **other traders will adapt** to your strategy. If you find an edge, competitors discover it too and arbitrage it away.

Challenge 3: Low Signal-to-Noise

Stock returns are mostly *noise*. The predictable component (the “alpha”) is tiny compared to randomness.

Challenge 4: Feedback Loops

Large AI-driven trades *change* the market, which changes future data, which changes the AI's inputs. The system affects its own training distribution.

Two Approaches to Market Prediction

Approach 1: Model Prices Directly

Feed historical prices into a neural network. Predict the next price.

Problems:

- Prices are nearly a random walk — almost all variation is noise
- No explanatory power — black box patterns
- Patterns disappear as soon as they are found

Approach 2: Model Participant Behavior

Model how investors read earnings calls, how sentiment spreads on social media, how fund managers respond to news.

Advantages:

- Grounded in human psychology (durable patterns)
- Explainable: “Price rose because investors overreacted to negative language”
- LLMs are naturally suited for this

This Tutorial's Philosophy

We focus on **behavior-first** modeling: understand people, then predict what prices they'll drive.

Why This Tutorial Is Different

Standard “ML in Finance” Coverage

- Feed price data to LSTM
- Show it backtests well
- Call it a day
- Ignore microstructure, behavior, ethics, and why it will fail in production

This Tutorial's Coverage

- Start with *why* markets exist and how they work
- Connect human behavior to tradable signals
- Show how LLMs and agents change the picture
- Rigorously test and critique strategies
- Confront the ethics head-on

The Connection to KDD

KDD is about discovering *useful* knowledge in data. In finance, “useful” means *deployable, robust, and ethical* — not just “has a good Sharpe ratio on historical data.”

Caveat

Nothing in this tutorial is investment advice. All strategies are presented for educational and research purposes only.

What We Covered in Section A

- A **market** is a coordination mechanism for buyers and sellers
- **Financial markets** trade claims on future money (stocks, bonds, FX)
- **Prices** reflect the latest agreed transaction — not inherent value
- **Supply and demand** drive price changes
- **New information** causes prices to update
- **Long** = bet on price rising; **Short** = bet on price falling
- **Liquidity** = ease of trading without moving the price
- Markets are hard for ML: *non-stationary, adversarial, low signal*
- Best approach: model **human behavior**, not just prices

Coming Up: Section B

Now we go *inside* the exchange. What actually happens when you click “Buy”? Who is on the other side? And why does this matter for AI?

Section B

Market Microstructure

“Inside the Market Machine”

Slides 25–54

Step-by-Step: Buying 10 Shares of Apple

1. **You click Buy** on your broker's app for 10 shares of Apple at the market price
2. **Order reaches broker:** Your broker (TD Ameritrade, Robinhood, etc.) receives your order
3. **Order routing:** The broker routes your order to an exchange (NYSE, NASDAQ) or an alternative trading venue
4. **Matching engine:** The exchange's computer system finds someone willing to sell 10 Apple shares
5. **Match!:** Your order is matched with a seller. The trade is executed at the agreed price.
6. **Settlement:** 2 business days later (“T+2”), shares are officially transferred to your account

How Fast Is This?

Steps 1–5 happen in **less than 1 millisecond** (one thousandth of a second) for large-cap stocks. High-frequency traders can complete an entire trade in under 10 *microseconds* — faster than a blink.

Two Order Types: Market Order vs. Limit Order

Market Order

“Buy this stock **right now** at whatever price is available.”

- **Guaranteed execution:** you will definitely buy
- **Uncertain price:** you get whatever the current ask price is
- Used when speed matters more than price

Like buying at a vending machine: you pay whatever it says.

Limit Order

“Buy this stock, but **only if the price is at most \$X.**”

- **Guaranteed price:** you never pay more than your limit
- **Uncertain execution:** it might never fill if price doesn't reach your limit
- Used when price matters more than speed

Like bidding on eBay: you set your max price and wait.

Why This Matters for AI

AI trading systems must decide whether to use market orders (fast but expensive) or limit orders (patient but risky of not filling). This is the **execution problem** covered in Section J.

Definition: Limit Order Book (LOB)

The **limit order book** is a continuously updated list of all outstanding buy and sell orders that have not yet been matched, organized by price.

Restaurant Waitlist Analogy

Imagine a popular restaurant. On the left, a list of people waiting for a table (buyers: "I'll wait if I can sit for under \$50"). On the right, tables available at certain prices (sellers). When a buyer's price matches a seller's price, they get seated (the trade executes).

Key Properties

- Buy orders are ranked: highest price first (most eager buyer at top)
- Sell orders are ranked: lowest price first (most eager seller at top)
- When a buy price $\geq$ sell price: **trade executes**
- The book updates thousands of times per second

The Two Sides of the Book

- **Bid price**: the highest price any buyer in the book is currently willing to pay
- **Ask price** (also called **offer**): the lowest price any seller is currently willing to accept

Example Order Book Snapshot

BUY SIDE (Bids)			SELL SIDE (Asks)		
Price	Qty	Orders	Price	Qty	Orders
\$149.95	500	3	\$150.00	300	2
\$149.90	1200	8	\$150.05	800	5
\$149.85	700	4	\$150.10	1500	9

- **Best bid** = \$149.95 (highest buyer); **Best ask** = \$150.00 (lowest seller)
- **Mid-price** = $(\$149.95 + \$150.00) / 2 = \$149.975$ — often used as the “true” price estimate

Definition: Bid-Ask Spread

The **bid-ask spread** is the difference between the ask price and the bid price. It is the immediate cost of trading — you buy at the ask and sell at the bid.

Simple Calculation

Best bid = \$149.95, Best ask = \$150.00

Spread = \$150.00 - \$149.95 = **\$0.05 per share**

If you buy 1,000 shares at \$150.00 and immediately sell them back at \$149.95, you lose \$50. That is the spread cost — and you haven't made a single investment decision yet!

Typical spreads:

- Large-cap stocks: \$0.01 (very liquid)
- Small-cap stocks: \$0.10–\$1.00
- FX major pairs: 0.0001 (1 pip)
- Crypto: varies wildly

Spread matters because:

- For a high-frequency strategy: spread is the main cost
- For long-term investors: spread is negligible
- For AI trading systems: must be modeled carefully in backtests

Why Does the Spread Exist?

The Market Maker's Problem

Someone must be willing to both buy *and* sell at all times. This person is called a **market maker**. The spread is their compensation for taking on risk.

The Currency Exchange Booth Analogy

At an airport currency exchange booth, they buy euros at \$1.08 and sell euros at \$1.12. The \$0.04 spread pays for their costs and risk. They don't know if the exchange rate will move against them while holding euros. The spread is their protection.

Spread compensates for:

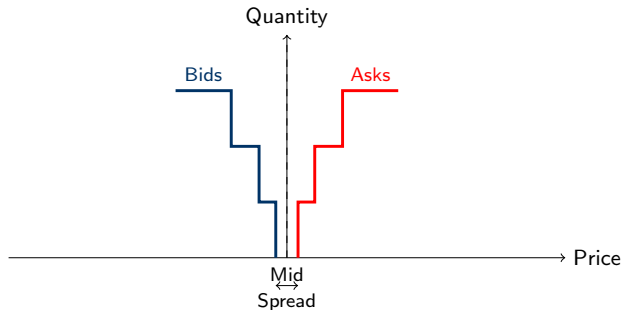
- **Inventory risk**: the stock might move against you while holding it
- **Adverse selection**: the person trading against you might know something you don't
- **Processing costs**: computers, staff, infrastructure

Key Insight

The **adverse selection** component is the most important for AI. If informed traders (those with private information) are active, market makers widen the spread to protect themselves.

The LOB as a Depth Chart

If we plot the cumulative quantity available at each price level, we get a characteristic “V” shape:



- **Left side:** buyers piling up below mid-price (want to buy cheap)
- **Right side:** sellers piling up above mid-price (want to sell high)
- The **gap in the middle** is the spread
- **Depth:** thick books = many orders = liquid; thin books = few orders = illiquid

What Is LOBSTER Data?

Definition: LOBSTER

LOBSTER (Limit Order Book System: The Efficient Reconstructor) is a dataset and platform from Humboldt University Berlin that provides high-resolution, message-by-message records of NASDAQ limit order books.

- **Coverage:** all NASDAQ-listed stocks (thousands of symbols)
- **Resolution:** every single order event, down to nanosecond timestamps
- **Depth:** captures the top N price levels on each side (typically $N = 5$ or $N = 10$)
- **Used in:** the majority of academic microstructure research since 2011

Why LOBSTER Matters for This Tutorial

LOBSTER data is the input for many of the AI systems we will discuss. It contains **behavioral signals** — every limit order placed, canceled, or modified tells us something about what market participants think and plan to do.

Access Note

LOBSTER data requires a subscription but provides sample data free for research. DeepLOB (a neural network for LOB data) is a key paper we'll reference.

LOBSTER Produces Two Files Per Stock Per Day

Message File (Event Log)

Each row = one event:

- **Time:** nanosecond timestamp
- **Type:** 1=new order, 2=cancel partial, 3=cancel full, 4=execute visible, 5=execute hidden
- **Order ID:** unique identifier
- **Size:** number of shares
- **Price:** in cents (integer)
- **Direction:** 1=buy, -1=sell

Orderbook File (Snapshots)

Each row = full LOB state after each event:

- **Ask price 1:** best ask (cents)
- **Ask size 1:** quantity at best ask
- **Bid price 1:** best bid
- **Bid size 1:** quantity at best bid
- **Ask price 2, 3...:** deeper levels
- **Bid price 2, 3...:** deeper levels

- Features derived from this: mid-price, spread, order imbalance, queue lengths, cancellation rates...

Definition: Order Flow

Order flow is the time-ordered sequence of all buy and sell orders arriving at an exchange. It is the raw pulse of market activity.

Analogy: Traffic on a Highway

Imagine monitoring a highway. You count every car (order) that passes, note its direction, speed, and size. Over time, you notice patterns: rush hour (open/close), accidents (news events), detours (market structure changes). Order flow is like reading this traffic pattern to predict congestion (price moves).

Key order flow signals:

- **Order imbalance:** more buys than sells? Price likely goes up
- **Cancellation rate:** high cancellations = market makers nervous
- **Trade size distribution:** large trades = institutional; small = retail

Order flow prediction = deep problem:

- Predicting *which* orders will arrive next
- Models: Hawkes processes, neural point processes
- Deep LOB models learn directly from raw order flow

Demo 1 Preview: LOBSTER Visualizer (10 Minutes)

What We Will Show You Live

We will open a Jupyter notebook using **real LOBSTER sample data** (AAPL, one trading day) and visualize order book dynamics in real time.

What You Will See

- **Order book heatmap**: bid and ask levels updating tick-by-tick — the “depth of market” as a live visualization
- **Order imbalance chart**: how the imbalance between buyers and sellers changes second-by-second
- **Price impact trace**: how a single large market order moves the mid-price and depletes depth

Why This Matters

- Makes **abstract concepts concrete**: you can see adverse selection happen — price moves away from you the instant a large order hits
- Reveals the **imbalance signal**: buy-side pressure exceeds sell-side $\Rightarrow$ price rises within seconds (a predictable, exploitable pattern)
- Shows **what AI models learn**: DeepLOB and similar models use exactly these patterns as features

★ DeepLOB Benchmark (Zhang et al., 2019 — LOBSTER AAPL data)

DeepLOB (CNN on 10-level LOB snapshots) achieves **~88% directional mid-price accuracy** (up/down/stationary, 10-event horizon) vs. ~52% for a naïve baseline. TransLOB (Transformer, Wallbridge 2020) extends this with attention across price levels.

Follow Along

The notebook is available in our GitHub repository (link on the final slide). You can run it on Google Colab — no local installation required.

What Is Adverse Selection?

Definition: Adverse Selection

Adverse selection occurs when the person you are trading with knows more than you do, and uses that advantage to trade against your interests.

The Used Car Analogy (Akerlof 1970)

Imagine buying a used car. The seller knows if the car is reliable or a “lemon.” You don’t. To protect yourself, you only offer a low price (average quality). But this means good-quality car owners won’t sell at that low price — leaving only lemon sellers in the market. This is adverse selection: you end up trading with the worst-informed party.

In financial markets:

- An **informed trader** knows a company is about to report bad earnings
- They sell to a market maker who doesn’t know
- The market maker loses money on this trade

Market maker’s defense:

- Widen the spread (charge more)
- Quote less depth (take less risk)
- Detect informed flow and adjust prices quickly

Informed Traders

- Have **private information**: inside knowledge, superior analysis, or faster data processing
- Trade because they believe the current price is **wrong**
- Profit from the price eventually correcting to reflect their information
- Examples: hedge funds with proprietary models, insiders (illegally), arbitrageurs

Uninformed Traders (“Noise Traders”)

- Trade for **other reasons**: rebalancing, liquidity needs, emotion, index tracking
- Don't believe the current price is especially wrong
- May lose money on average to informed traders
- Examples: retail investors, index fund rebalancers, pension funds adjusting allocations

Why This Matters for AI

An AI system is essentially trying to join the **informed** category. The better its models, the more it resembles an informed trader — and the more market makers widen their spreads to protect against it.

Definition: Order Flow Toxicity

Order flow toxicity measures the fraction of order flow that comes from *informed* traders. High toxicity = market makers are trading mostly against people who know more than them.

- High toxicity leads market makers to **widen spreads** or withdraw from the market entirely
- Toxicity spikes before major news events (earnings, Fed decisions) as informed traders position themselves
- The VPIN (Volume-synchronized Probability of Informed Trading) metric was proposed by Easley et al. (2012) to measure toxicity in real time

VPIN in Practice

VPIN was cited as a predictor of the 2010 Flash Crash (we'll cover this shortly). High VPIN $\approx$ informed traders flooding in $\approx$ market makers about to retreat $\approx$ liquidity about to dry up $\approx$ dangerous conditions for automated systems.

AI Implication

An AI trading system generating large, directional order flow will be detected as “toxic” by market makers, who will widen spreads in response — raising the AI's costs.

Easley, Kiefer, O'Hara & Paperman (1996) — The PIN Model

The **PIN (Probability of Informed Trading)** model estimates what fraction of all trades are from informed traders, using only the observable sequence of buys and sells.

The model's world:

1. Each day, either a **news event** occurs (probability α) or it doesn't
2. If news occurs, it's **good** (prob δ) or **bad** (prob $1 - \delta$)
3. Informed traders arrive at rate μ and trade in the direction of the news
4. Uninformed buyers arrive at rate ϵ_b ; sellers at ϵ_s

PIN Formula

$$\text{PIN} = \frac{\alpha\mu}{\alpha\mu + \epsilon_b + \epsilon_s}$$

- Numerator: informed trading rate
- Denominator: total trading rate
- $\text{PIN} = 0$: all trades uninformed
- $\text{PIN} = 1$: all trades informed
- Typical stocks: $\text{PIN} \approx 0.10\text{--}0.25$

What Is Price Impact?

Definition: Price Impact

Price impact is the change in market price caused by your own trade. When you buy, you push the price up; when you sell, you push it down. The bigger your trade, the larger the impact.

The Stone-in-a-Pond Analogy

Drop a pebble in a still pond: tiny ripple. Drop a boulder: large wave. A small retail order barely moves the price. A large institutional order (selling 5 million shares of a stock that only trades 1 million shares daily) creates massive price impact, and the seller ends up getting a much worse average price than hoped.

Why price impact matters:

- It is an *invisible tax* on large traders
- A strategy that makes \$1M on paper may make \$0 after impact costs
- Capacity constraints: many strategies only work for small amounts

For AI systems:

- AI cannot backtest impact (data shows pre-impact prices)
- Live trading always worse than backtest
- Multi-agent systems: all agents trading same signal simultaneously amplifies impact

Temporary vs. Permanent Price Impact

Temporary Impact

Temporary impact is the price distortion caused by *consuming liquidity* — filling orders that were waiting in the book. Once your trade is done, the book refills and the price reverts partially.

Like pressing a spring: it bounces back.

Caused by: trading faster than the market can absorb your order.

Permanent Impact

Permanent impact reflects the *information content* of your trade. If your trade signals informed trading, other participants update their beliefs and the price stays at the new level.

Like moving a table: it stays where you put it.

Caused by: the market inferring you have private information.

Practical Consequence

A smart execution algorithm (Section J) tries to minimize both: trade slowly to reduce temporary impact, but not so slowly that competitors pick up on your signal (which increases permanent impact). This is the fundamental **execution trade-off**.

Kyle's Setup

Albert Kyle's 1985 paper "Continuous Auctions and Insider Trading" is perhaps the most influential model in market microstructure. It asks: *how does an informed trader optimally trade to hide their information?*

Three players:

1. **Insider:** knows the true value of the stock; wants to profit without revealing information
2. **Noise traders:** random traders who provide camouflage for the insider
3. **Market maker:** sets prices based on total order flow (can't see who is who)

Key results:

- The insider **trades slowly** to hide in the noise and avoid moving the price against themselves
- Market impact is *linear*: price move $\propto$ order size
- More noise traders = more profitable for insider (better camouflage)
- "Kyle's lambda" (λ) measures market illiquidity

Modern Relevance

LLM-based AI systems face the same strategic problem as Kyle's insider: how to extract value from private information without revealing it to competitors.

A Different Microstructure Classic

Glosten and Milgrom model how a market maker sets bid and ask prices when some traders are informed. Their key insight: **the spread is determined by the probability of facing an informed trader.**

The Core Logic

Imagine a market maker facing two types of customers:

- 20% are informed (they know the stock is worth \$110, not the current price of \$100)
- 80% are uninformed (random buys and sells)

The market maker sets:

- Ask = \$102 (charged extra because 20% chance this buyer knows something bad)
 - Bid = \$98 (paid less because 20% chance this seller knows something bad)
 - Spread = \$4 = compensation for adverse selection
-
- As more informed traders enter, the spread **widens**
 - Trading halts when the market maker can't set a profitable spread — **market breakdown**

Who Are Market Makers?

Definition: Market Maker

A **market maker** is a firm that continuously quotes both buy (bid) and sell (ask) prices, committing to trade at those prices. They provide liquidity to the market as a business.

Casino Analogy

A casino doesn't gamble randomly — it *runs the game*. It quotes odds that give it a small edge on every bet. Over millions of bets, it makes consistent profits. A market maker is similar: they earn the spread on every trade, profiting from volume regardless of market direction. They don't need to predict whether prices go up or down — they earn from the difference between buy and sell prices.

Major market makers:

- Citadel Securities
- Virtu Financial
- Jane Street
- IMC Financial Markets

Their AI systems:

- Real-time adverse selection detection
- Optimal quote placement algorithms
- Inventory risk management
- Competitor detection

Definition: High-Frequency Trading

High-frequency trading (HFT) is algorithmic trading that executes large numbers of orders at extremely high speeds (microseconds to milliseconds), exploiting tiny, fleeting price discrepancies.

HFT characteristics:

- Hold periods: microseconds to seconds (rarely minutes)
- Profit per trade: fractions of a cent
- Volume: millions of trades per day
- Infrastructure: co-location, fiber optic cables, microwave towers
- Zero overnight positions

HFT strategies:

- **Market making:** quote both sides, earn spread
- **Statistical arbitrage:** exploit correlated asset mispricings
- **Latency arbitrage:** be faster than competitors at processing news
- **Order flow prediction:** detect large orders before they execute

Controversy

HFT is controversial: it narrows spreads (good for everyone) but may extract value from slower participants and create systemic instability. We'll see both sides.

Definition: Latency

In trading, **latency** is the time delay between when a market event occurs (e.g., a price changes) and when a trading system can react to it (e.g., submit a new order).

Speed Reference Points

- Human blink: ~ 300 milliseconds
- Human reaction: ~ 200 milliseconds
- Retail trading system: ~ 5 milliseconds
- Professional algorithmic: ~ 0.5 milliseconds
- HFT systems: ~ 0.001 milliseconds (1 microsecond)
- Speed of light NYC to London: ~ 28 milliseconds

Why Latency Matters

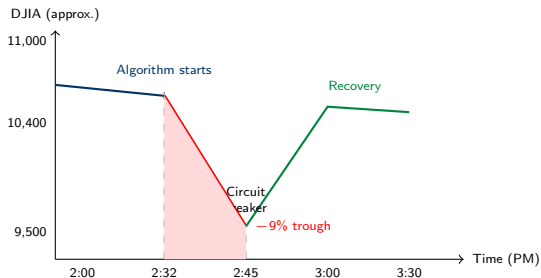
- If two traders see the same price anomaly, **the faster one profits**; the slower one gets nothing (or worse, gets adversely selected)
- HFT firms spend hundreds of millions on infrastructure to shave microseconds
- “Co-location”: pay exchange fees to place servers in the same building as the matching engine

LLM Latency Problem

Current LLMs take *seconds* to respond. They cannot compete in microsecond markets. LLMs are relevant for *longer-horizon* signal generation, not microsecond execution.

What Happened?

On May 6, 2010, the US stock market experienced the **largest intraday point drop in history** at that time. The Dow Jones dropped nearly 1,000 points (about 9%) in minutes, then recovered almost entirely within 20 minutes.



Timeline:

- **2:32 PM:** Automated algorithm sells \$4.1B E-mini futures
- **2:41 PM:** HFT firms detect pressure, pull bids
- **2:45 PM:** Liquidity collapses; stub quotes execute
- **2:45:28 PM:** Circuit breakers pause trading
- **2:45–3:00 PM:** Markets recover nearly fully

Extreme Events

- Accenture: \$40 → \$0.01
- P&G: dropped 37% in seconds
- ~\$1 trillion evaporated briefly

SEC/CFTC Joint Report (2010): The Cascade

1. **Trigger:** Waddell & Reed's algorithm sold 75,000 E-mini futures contracts, calibrating only to trading volume (not price or time) — it accelerated as volatility rose
2. **HFT Amplification:** HFT market makers absorbed initial selling but quickly became net sellers themselves, passing the “hot potato” between each other at declining prices
3. **Liquidity Withdrawal:** HFT firms, detecting abnormal conditions, canceled their limit orders en masse — the order book **went empty** on the buy side
4. **Cross-asset Contagion:** The crash spread from futures to individual stocks, where stub quotes (placeholder orders at absurd prices) became the best available bid
5. **Recovery:** Manual pauses and circuit breakers broke the feedback loop

The Key Mechanism

Automated systems fed into each other. No human was in the loop. The crash was a **reflexive algorithmic cascade** — a new type of risk that didn't exist before automation.

Flash Crash = Preview of Multi-Agent AI Risk

The Flash Crash was essentially what happens when many automated agents interact without coordination. Modern multi-agent AI systems could create similar dynamics.

Lesson 1: Feedback loops are dangerous

- AI agents whose actions affect their own inputs can destabilize
- Must model second-order effects: “What happens if everyone does what I’m doing?”

Lesson 2: Circuit breakers are essential

- AI systems need hard stops that trigger on anomalous conditions
- Cannot rely on the AI to self-regulate in novel regimes

Lesson 3: Liquidity is not always there

- Backtests assume liquid markets; real crises create illiquidity
- AI strategies that work in normal times may fail catastrophically in crises

Lesson 4: Correlated AI = systemic risk

- If many AI systems use similar models and data, they will correlate their actions
- Correlated selling = crash amplification

What Happened?

In January 2021, retail investors on the Reddit forum r/WallStreetBets coordinated to buy shares and options of GameStop (GME), a struggling video game retailer, causing its stock to rise from ~\$20 to nearly \$500 in two weeks.

Background:

- GameStop was **heavily shorted** — hedge funds (including Melvin Capital) had bet the stock would fall
- Short interest was over 140% of the float (more shares shorted than existed!)
- Reddit users recognized this as a potential **short squeeze**

The squeeze mechanics:

1. Reddit users buy GME stock and call options
2. Stock price rises; short sellers start losing money
3. Short sellers must buy shares to close their position (“cover”)
4. Buying by shorts *further* pushes price up
5. Feedback loop: Melvin Capital lost 53% in January alone

What Is Short Selling? (Deeper Dive)

Recap and Expansion

We introduced short selling on slide 18. Let's go deeper with the mechanics relevant to GameStop.

Step-by-Step Short Selling

1. Hedge fund borrows 1,000 GME shares from a broker (pays a borrowing fee, e.g., 1% per year)
2. Immediately sells them at \$20 each: receives \$20,000
3. Hopes price falls to \$5; plans to buy back for \$5,000
4. Profit: $\$20,000 - \$5,000 - \text{fees} = \sim \$15,000$
5. **What actually happened:** price rose to \$400
6. To close position, must buy back 1,000 shares at \$400 = \$400,000
7. **Loss:** $\$400,000 - \$20,000 = \$380,000$ on a \$20,000 bet

Unlimited Downside

This illustrates why short selling has theoretically unlimited loss: a stock can go up *forever*, but can only fall to zero. Short sellers can be forced to cover ("margin calls") at any price.

Collective Retail Action at Scale

GameStop showed that social media could coordinate millions of small investors into a force powerful enough to challenge institutional hedge funds.

The social media effect:

- r/WallStreetBets grew from 2M to 9M members in days
- Memes, humor, and community identity drove coordinated buying
- Information spread faster than hedge fund risk systems could adapt
- “Diamond hands” culture discouraged selling (loss aversion weaponized)

Why This Matters for AI

- NLP systems monitoring Reddit and Twitter could have detected the signal early
- LLMs can now track sentiment at massive scale across platforms
- But: retail coordination is a *new type of market signal* not in historical training data
- Models trained before 2021 had no examples of “social media short squeeze” to learn from

Lesson 1: Social Signal Integration

Traditional quant models used price and fundamental data. GameStop proved that **social media sentiment** is a genuine market-moving force. LLMs that monitor forums, Twitter, and Discord may detect similar events earlier.

Lesson 2: Regime Shifts Happen Suddenly

Any model trained on historical data implicitly assumes the future resembles the past. GameStop-style events have no historical analog. AI systems must **detect regime shifts** and reduce exposure when the distribution has changed.

Lesson 3: Crowded Positioning is Dangerous

Many hedge funds were short GME simultaneously. When forced to cover, they all bought at the same time. AI systems that use similar models will create similar crowded positions — and similar violent unwinds.

Lesson 4: Human Intent Is Hard to Model

The Reddit community was partly motivated by ideology (“fight Wall Street”) and memes, not profit maximization. Traditional rational-agent models completely failed to anticipate this. LLMs are better equipped — but still struggle with emergent collective behavior.

What We Covered in Section B

- **Limit order book:** ranked waiting list of buy/sell orders
- **Bid-ask spread:** cost of immediacy; widens with informed traders
- **Adverse selection:** trading against someone who knows more
- **PIN and VPIN:** measuring informed trading probability
- **Price impact:** your own trades move prices against you
- **Kyle model:** optimal informed trading = hide in noise
- **Market makers:** earn spread, fear informed flow
- **HFT:** microsecond speed, tiny edges at high volume
- **Flash Crash:** algorithmic cascade = systemic AI risk
- **GameStop:** social media = new type of market signal

Key Takeaway

Microstructure tells us that market prices are driven by the *behavior of participants* — their information, their incentives, their fears. This is exactly what LLMs are designed to model. Coming up: the behavioral finance evidence for why human biases are predictable.

Section C

Behavioral Finance

“How Humans (Mis)behave”

Slides 55–76

The “Rational Agent” Myth

Classical Economics Assumption

Traditional economics assumes people are **rational agents**: they always make decisions that maximize their long-term benefit, using all available information correctly and without emotional bias.

The Rational Agent Would...

- Hold a losing stock if the future prospects are good
- Never sell a stock just because it feels uncomfortable
- Always compare opportunity costs correctly
- Never follow the crowd unless the crowd is right
- Make the same decision in equivalent situations regardless of framing

Real Humans Actually...

- Sell winning stocks too early and hold losers too long
- Trade more when emotionally aroused
- Follow the crowd even when it's clearly wrong
- Make different decisions when the same fact is framed differently
- Lose more sleep over a \$100 loss than they gain pleasure from a \$100 win

These systematic deviations from rationality are the subject of **behavioral finance**.

Definition: Behavioral Finance

Behavioral finance combines psychology and economics to explain why investors make systematic mistakes that deviate from rational predictions — and how these mistakes create exploitable patterns in asset prices.

Why “Predictably” Irrational Is the Key Word

If humans were randomly irrational, errors would cancel out and prices would still be efficient. But if humans make the *same type* of mistake in the *same direction*, prices will systematically misprice assets in predictable ways.

Founding researchers:

- Daniel Kahneman (Nobel 2002)
- Amos Tversky
- Richard Thaler (Nobel 2017)
- Robert Shiller (Nobel 2013)

Core insight for AI: If human biases are systematic and predictable, then an AI system that accurately models these biases can anticipate how investors will react to new information — *before* they react.

The Most Important Theory in Behavioral Finance

Prospect theory describes how people actually evaluate gains and losses — not in absolute terms, but *relative to a reference point*, with **losses weighted more heavily than equivalent gains**.

The Classic Question

Which do you prefer?

Option A: Receive \$500 for certain

Option B: 50% chance of \$1,000, 50% chance of \$0

Both options have the same *expected value* (\$500). Rational theory says you should be indifferent. But most people choose A — they are **risk-averse for gains**.

Now flip it:

Option C: Lose \$500 for certain

Option D: 50% chance of losing \$1,000, 50% chance of losing \$0

Most people choose D — they become **risk-seeking for losses**!

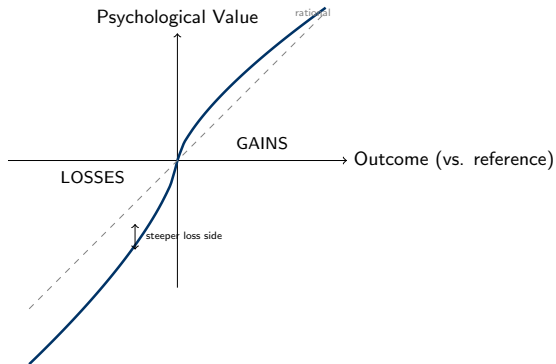
Three Key Properties

1. **Reference dependence:** outcomes judged relative to a starting point, not absolute wealth
2. **Loss aversion:** losses hurt $\sim 2.25\times$ more than equivalent gains feel good
3. **Diminishing sensitivity:** going from 0 to \$100 feels better than going from \$900 to \$1,000

The Value Function: Losses Hurt More Than Gains Feel Good

The S-Shaped Value Function

Prospect theory's value function is S-shaped: steep and concave for gains (diminishing returns to winning), and steep and convex for losses (diminishing sensitivity to losing more), but the loss side is steeper overall.



- The curve is **steeper in the loss domain**: losing \$100 feels worse than gaining \$100 feels good
- This asymmetry drives the **disposition effect** (next slide) and many other trading biases

Definition: Disposition Effect (Shefrin & Statman, 1985)

The **disposition effect** is investors' tendency to sell stocks that have *gone up* (locking in gains) and to hold stocks that have *gone down* (avoiding realizing losses), even when the rational action is the opposite.

The Tax-Efficient Action vs. Actual Behavior

Rational investors should *sell losers* (realize tax losses to offset gains) and *hold winners* (delay tax on gains). But empirically, investors do the opposite: they sell winners 50% faster than losers (Odean, 1998).

Why it happens (prospect theory):

- Reference point = purchase price
- Selling a winner: feels good (definite gain)
- Holding a loser: prefer the gamble (maybe it comes back) over the certain loss
- Loss aversion makes realized losses feel painful

AI implications:

- Winners that are sold create *selling pressure* at resistance levels
- Losers held create *support levels* (owners refuse to sell below buy price)
- Technical analysis levels (support/resistance) have a behavioral foundation
- LLMs can detect when stocks are above/below large investor cost bases

Alice's Portfolio Story

Alice bought two stocks six months ago:

- **TechCorp**: bought at \$100, now worth \$150 (up 50%)
- **OilCo**: bought at \$100, now worth \$70 (down 30%)

She needs \$5,000 cash. Which should she sell?

What a Rational Investor Does

Evaluate *future prospects*: which stock is more likely to rise going forward? The past purchase price is irrelevant (“sunk cost”). Sell whichever has weaker future prospects.

What Alice Actually Does

Alice sells TechCorp — she “feels good” locking in the \$50 gain. She holds OilCo because selling would mean “accepting defeat.” She keeps telling herself: “It’ll come back.”

The Market Consequence

If millions of Alices all sell their winners and hold their losers, winning stocks face artificial selling pressure and losing stocks face artificial buying support. This creates **predictable return patterns** that AI can exploit.

The Overreaction Hypothesis

Overreaction occurs when investors react too strongly to recent news, pushing prices too far in one direction. The correction that follows creates a **contrarian return pattern**.

The Classic Experiment (De Bondt & Thaler, 1985)

They took US stock data from 1926–1982 and:

1. Identified the 35 “worst losers” over the past 3 years (stocks that fell most)
2. Identified the 35 “biggest winners” over the past 3 years (stocks that rose most)
3. Tracked both groups for the *next* 3 years

Result: Past losers *outperformed* past winners by 25% over the next 3 years!

- This contradicts the efficient market hypothesis (past returns shouldn't predict future returns)
- Explanation: investors overreact to bad news on losers (push them too low) and good news on winners (push them too high)
- The market eventually corrects, creating **long-horizon reversal**

A Paradox: Both Overreaction and Underreaction Exist

At short horizons (1–12 months), stocks that went up *keep going up* (momentum). At long horizons (3–5 years), the opposite happens (reversal). How can markets both under- and over-react?

The Gradual Information Diffusion Story

Imagine a company reports great earnings. Not everyone learns instantly:

1. **Day 1:** Only sophisticated investors read the filing immediately — they buy
2. **Week 1:** Analysts publish reports — more investors buy
3. **Month 1–6:** Media coverage reaches retail investors — more buying
4. **Month 6–12:** Retail investors pile in enthusiastically — price overshoots
5. **Year 2–3:** Price corrects as initial enthusiasm fades

Underreaction Creates Momentum

If the price doesn't fully adjust to news immediately (underreaction), yesterday's winner tends to be tomorrow's winner. This is the basis for **momentum investing**.

The Landmark Momentum Study

Jegadeesh and Titman (1993) showed that buying stocks that performed well over the *past 3–12 months* and selling stocks that performed poorly yields significant abnormal returns.

The Simple Momentum Strategy

1. Each month, rank all stocks by their return over the past 12 months
2. **Buy:** top 10% (past winners)
3. **Sell short:** bottom 10% (past losers)
4. Hold for 3 months
5. Repeat

This strategy earned $\sim 1\%$ per month (12% annually) in US data from 1965–1989 — after transaction costs and risk adjustment!

Momentum Has Crashes

Momentum works on average but has *severe crashes* during market rebounds (when losers bounce sharply). Managing momentum risk is a key challenge for AI systems implementing it.

Definition: Contrarian Strategy

A **contrarian** strategy does the opposite of what everyone else is doing: buy when others are fearful and selling, sell when others are greedy and buying.

Warren Buffett's Famous Quote

"Be fearful when others are greedy, and greedy when others are fearful."

This is pure contrarian logic: when prices are driven down by panic (not fundamentals), buy. When they're driven up by euphoria, sell.

The Logic

- Overreaction creates mispricing
- Corrections to mispricing generate profits
- De Bondt-Thaler (slide 62): past losers outperform over 3–5 years
- The challenge: timing the reversion is hard
- Also: you must tolerate *being wrong for a long time* before you're right

Contrarian vs. Momentum: A Puzzle

Contrarian works at 3–5 years. Momentum works at 3–12 months. Can both be true? Yes — same underlying psychology, different time horizons. Underreaction creates momentum; eventual overreaction creates reversal.

Definition: Herding

Herding occurs when investors ignore their own private information and instead copy the actions of others, because they assume the crowd knows something they don't.

The Restaurant Analogy (Bikhchandani et al., 1992)

You're visiting a new city and need lunch. You see two restaurants side by side. You have no information, so you decide to flip a coin — but as you arrive, you see 10 people walking into Restaurant A and no one going into Restaurant B. You go into A.

But wait: those 10 people might have made the *same inference* from the people before them! If the first person chose randomly and everyone else followed, you've all crowded into a possibly inferior restaurant. This is an **information cascade**.

Herding in markets:

- Mutual fund managers copy each other (career risk)
- Retail investors follow stock tips from social media
- Analysts cluster their earnings estimates

Herding consequences:

- Bubbles (tech bubble 1999, housing 2006)
- Crashes (everyone exits together)
- Momentum: herding amplifies price trends

AI Systems Can Herd Too

If multiple AI systems use similar data, similar models, and similar signals, they will generate correlated trading decisions — a form of AI herding that amplifies the effects described above.

AI Herding Scenario

1. 50 hedge funds deploy FinBERT-based sentiment models
2. All read the same earnings call
3. All generate the same buy signal
4. All submit buy orders simultaneously
5. Price spikes immediately; all get worse fills than expected
6. Backtest showed this strategy “worked,” but live trading doesn’t

The Regulatory Concern

The FSB (Financial Stability Board) and BIS have explicitly flagged AI herding as a systemic risk. When AI systems:

- Use the same training data
- Learn similar patterns
- Deploy similar risk management
- ...the resulting correlated behavior could trigger market instability

Defense: Diversity by Design

Multi-agent systems (Section H) can be designed with *intentional diversity* — different models, different time horizons, different signals — to reduce correlation.

The Attention Hypothesis

Because there are thousands of stocks and limited human attention, retail investors tend to buy stocks that **catch their attention** (in the news, large price moves, high volume) rather than carefully analyzing fundamentals.

The Barber & Odean (2008) Finding

Using data from 66,000 retail brokerage accounts, they showed:

- Retail investors are *net buyers* of attention-grabbing stocks (news events, large price moves)
- The stocks they buy based on attention subsequently **underperform**
- Institutional investors do not show the same attention bias

Attention proxies:

- News mentions (Google News, Bloomberg)
- Search volume (Google Trends)
- Social media mentions (Twitter/X, Reddit)
- Unusual trading volume

AI application:

- LLMs can measure attention in real time across thousands of news sources
- Attention spikes predict short-term buying pressure (even if ultimately reverting)
- Can distinguish: *new information* (price impact is permanent) vs. *noise attention* (price impact reverses)

Definition: Investor Sentiment

Investor sentiment is the overall mood or optimism/pessimism of investors in the market — above and beyond what can be justified by fundamentals. When sentiment is high, investors are willing to pay too much; when low, too little.

Baker & Wurgler's Sentiment Index

They constructed a sentiment index from six market-wide variables:

1. Closed-end fund discount (how much funds trade below asset value)
2. NYSE share turnover (trading volume as fraction of shares outstanding)
3. Number of IPOs (count of initial public offerings in a given month)
4. Average first-day return on IPOs (how much new issues “pop” on debut)
5. Equity share in new issues (how much companies prefer equity over debt)
6. Dividend premium (how much investors pay extra for dividend payers)

Result: High sentiment predicts lower future returns for speculative stocks; low sentiment predicts higher returns.

For LLMs

Baker-Wurgler use market data. LLMs can construct text-based sentiment indices from news, social media, and analyst reports — potentially faster and more granular.

Definition: Mental Accounting (Thaler, 1985)

Mental accounting is the tendency to treat money differently depending on where it came from or what “bucket” it is mentally assigned to, even though money is fungible (a dollar is always a dollar).

The House Money Effect

You win \$200 at a casino. You now feel comfortable making a \$200 bet you'd normally never make — it's “house money,” not your real money. But financially, it's exactly the same \$200 either way.

In markets: investors who have made large profits on a stock treat their “winnings” as free money and take more risk — a pattern LLMs can detect from earnings call tone (excessive optimism after run-up).

Market consequences:

- House money effect: more risk-taking after gains
- Pain of paying: holding losing stocks (avoids the pain of realizing loss)
- Separate accounting for different investments leads to sub-optimal portfolios

AI can detect:

- Stock trading patterns consistent with mental accounting
- Manager language suggesting overconfidence after a run-up
- Fund flows into speculative assets after market gains

Definition: Anchoring (Tversky & Kahneman, 1974)

Anchoring is the tendency to rely too heavily on the first piece of numerical information seen (the “anchor”) when making subsequent estimates or judgments.

The Classic Experiment

Participants are asked: “Is the population of Turkey larger or smaller than 35 million?” Then asked: “What is the population of Turkey?” Another group gets the same question but with “100 million” as the anchor. The second group’s estimates are significantly higher, even though neither group has any idea what the actual answer is.

Anchoring in financial markets:

- **52-week high:** stocks approaching their 52-week high face selling resistance — investors anchor to that price as the ceiling
- **Purchase price:** the price you paid anchors your sense of fair value
- **Analyst targets:** if analyst says \$150, stock rarely rallies far beyond, even if fundamentals support \$200

LLM Applications:

- Extract analyst price targets from research reports
- Identify prominent anchor prices in news coverage
- Detect when prices are near psychological levels (\$100, \$50, round numbers)

A Formal Model of Investor Overconfidence

DHS (1998) provide a theoretical model explaining both momentum and long-term reversal using two behavioral biases: **overconfidence** and **self-attribution bias**.

Overconfidence

Investors **overestimate** the precision of their private signals. They think they know more than they do, so they overweight their own analysis relative to public information.

Self-Attribution Bias

When a trade works out, investors attribute it to skill. When it doesn't, they blame bad luck. Over time, this *increases* overconfidence after a string of successes.

How This Creates Momentum and Reversal

1. Informed investor gets a private signal; overconfident reaction causes initial price overreaction
2. Public information gradually confirms or denies the signal
3. **If confirmed**: overconfidence grows (self-attribution), causing *more* buying → **momentum**
4. **Eventually**: overreaction corrects as true value becomes undeniable → **long-run reversal**

A Model of Under- and Over-Reaction

BSV (1998) propose a model where investors switch between two mental models of earnings: one where earnings are *mean-reverting* (underreaction model) and one where they *trend* (overreaction model). They update their model belief based on recent data.

The Two Regimes

- **After isolated good earnings:** investors assume it was a fluke; they underreact (momentum opportunity)
- **After several consecutive good earnings:** investors switch to the “trending” model; they overreact, buying aggressively (eventually leading to reversal)

Why This Is Relevant for LLMs

BSV's model implies that the *narrative* surrounding a stock (“is this a one-time event or a trend?”) determines how investors will react to future news. LLMs are perfectly suited to extract this narrative context from text — and therefore to predict which regime applies to each stock at each moment.

From Biases to Exploitable Patterns

The Connection: Bias → Price Pattern → Signal

Each behavioral bias creates a systematic, predictable pattern in asset prices. An AI system that can measure the bias can predict the resulting price movement.

Bias / Factor	Price Pattern	Signal Source
Disposition effect	Selling pressure above cost basis	Investor return history
Overreaction	Long-run reversal (3–5 yr)	Past returns
Underreaction	Momentum (3–12 mo)	Past returns + news
Attention	Short-term price impact	News volume, search
Sentiment	Sector-level mispricings	Text-based indices
Anchoring	Price clustering at round #s	Price levels, targets
Herding	Trend amplification	Fund flows, positioning
<i>Risk-based benchmarks (Fama-French, 2015): size, value, profitability, investment</i>		
<i>Behavioral signals should deliver alpha net of these five factors to be credible.</i>		

The LLM Advantage

Traditional quant systems needed to construct these signals from market data. LLMs can extract behavioral signals *directly from text* — earnings calls, analyst reports, news, social media — often with better timeliness and richer context.

★ *Empirical validation:* ChatGPT-based news sentiment predicts next-day stock returns with ~60% directional accuracy and ~4% annualized alpha before crowding (Lopez-Lira & Tang, 2023).

The Central Design Principle of This Tutorial

Rather than asking “What statistical patterns exist in price data?”, ask:

“What human behavior will cause prices to move, and how can I detect that behavior before or as it happens?”

Data-First Approach (Problematic)

1. Run regressions/ML on price history
2. Find patterns
3. Ask: what does this predict?
4. Trade on it

Problem: patterns are likely spurious (multiple testing), not durable (regime changes), and unexplainable.

Behavior-First Approach (Better)

1. Identify a behavioral bias (e.g., disposition effect)
2. Reason about what price pattern it creates
3. Design a measurement (LLM reads investor forums for evidence of anchoring)
4. Test the specific predicted pattern

Advantage: theory-grounded, more durable, interpretable.

LLMs Enable This Approach at Scale

LLMs can read millions of documents and identify behavioral signals that were previously invisible to quantitative systems.

Beyond Traditional Filings and News

Institutional investors increasingly use **alternative data** — non-traditional information sources that were unavailable or unprocessed before AI. LLMs and vision models make many of these sources machine-readable for the first time.

Key Alternative Data Sources

- **Satellite imagery:** count cars in retailer parking lots before earnings; track shipping container density at ports
- **Credit/debit card transactions:** real-time consumer spending by merchant, geography, and demographic
- **Job postings:** company hiring signals predict future revenue; sudden hiring freezes predict downturns
- **Web search volume:** Google Trends for product names predict revenue beats/misses 2–4 weeks early

LLM Role in Alternative Data

- Parse unstructured job descriptions: extract technology stack, team size signals
- Summarize patent filings: detect R&D pivots before public announcements
- Read employee reviews (Glassdoor): management quality and culture signals
- Analyze app store reviews: product satisfaction leading indicator

Key Caveat

Alternative data is expensive (satellite: \$50K–500K/yr), noisy, and subject to regulatory scrutiny (material non-public information rules). Its advantage decays quickly once it is widely adopted.

What We Covered in Section C

Human psychology creates systematic, predictable deviations from rational pricing — and each one is a potential signal.

- **Prospect theory:** losses hurt $\sim 2.25\times$ more than gains feel good
- **Disposition effect:** sell winners too early, hold losers too long
- **Overreaction:** prices overshoot, then revert over years
- **Momentum:** prices undershoot initially, creating short-term trends
- **Contrarian investing:** exploit overreaction at long horizons
- **Herding:** coordinated behavior amplifies trends and crashes
- **Investor attention:** retail buys attention-grabbing stocks (then underperforms)
- **Investor sentiment:** market-wide mood predicts aggregate returns
- **Mental accounting:** house money effect, separate buckets
- **Anchoring:** 52-week highs, purchase prices create resistance/support
- **Behavior-first philosophy:** design signals from human psychology, not price noise

Coming Up: Section D

We now move from *why* investors behave irrationally to *how* their behavior is encoded in text. News articles, earnings calls, and social media posts are rich behavioral signals — but only if we can read them at scale.

Section D

Financial NLP

“Words Are Data Too”

Slides 77–98

The Pioneer Study

Paul Tetlock's 2007 paper "Giving Content to Investor Sentiment: The Role of Media in the Stock Market" was one of the first to rigorously show that **news text predicts stock returns**.

Tetlock's Method

1. Collected the text of 3,000+ "Abreast of the Market" Wall Street Journal columns (1984–1999)
2. Counted negative words using the Harvard General Inquirer dictionary
3. Computed a daily negativity score
4. Measured the correlation with next-day and future stock returns

Key findings:

- High media negativity predicts *lower* stock market returns the next day
- The price impact *partially reverses* after a few days — consistent with the overreaction story

The Dictionary Problem

General English dictionaries are poor for finance. "Risk" is negative in everyday English, but standard in neutral financial language. This led to the Loughran-McDonald word list (next slide).

The Problem with General Dictionaries

Common English sentiment dictionaries (Harvard GI, LIWC) misclassify many financial terms. Loughran & McDonald (2011) built a dictionary specifically for financial text by reading tens of thousands of SEC filings.

Words Misclassified by Harvard GI

Classified as negative in Harvard GI but *neutral or positive* in finance:

- “risk” (neutral in finance — all investments have risk)
- “tax” (neutral)
- “liability” (neutral accounting term)
- “capital” (often positive: capital investment)
- “depreciation” (neutral accounting)

The L&M Solution

Six word lists for financial documents:

- **Negative:** “loss,” “impairment,” “default”
- **Positive:** “profitable,” “innovative,” “awarded”
- **Uncertainty:** “approximate,” “may,” “uncertain”
- **Litigious:** “litigation,” “lawsuit”
- **Strong modal:** “will,” “must,” “required”
- **Weak modal:** “might,” “could,” “possible”

- Uncertainty words in 10-K filings predict higher future volatility (Loughran & McDonald, 2011)
- L&M outperforms Harvard GI in predicting abnormal returns around filings

What Is Tokenization?

Definition: Tokenization

Tokenization is the process of breaking text into smaller units called **tokens**. Tokens are the basic building blocks that language models process — they can be words, parts of words, or even individual characters.

Example: How a Modern Tokenizer Splits Text

Input: "The stock price dropped sharply."

Tokens: ["The", "stock", "price", "dropped", "sharp", "ly", "."]

Note: "sharply" was split into "sharp" + "ly" because the tokenizer handles words it hasn't seen by breaking them into known subwords.

Why subword tokenization?

- Handles unknown words: "ChatGPT-4o" splits into known pieces
- Efficient: vocabulary size $\sim 50,000$ covers all text
- Shares structure: "invest," "investor," "investing" share the "invest" token

Finance implication:

- Domain-specific terms ("EBITDA," "repo," "CDO") may be split oddly
- Finance LLMs benefit from training on financial text (more coherent tokenization of jargon)
- Each token costs money: GPT-4 charges per token

What Is a Language Model?

Definition: Language Model

A **language model** is a system that assigns probabilities to sequences of tokens. In practice, this means: given the previous words, predict what word comes next.

NEXT-TOKEN PREDICTION

A function over text that outputs a distribution

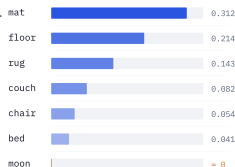
CONTEXT

The cat sat
on the ?

$x = \text{"The cat sat on the"}$

$P(\text{NEXT TOKEN} \mid \text{CONTEXT})$

$p_{\theta}(x_{t+1} \mid x_{1:t})$



non-zero for every word in the vocabulary – even "soon"

sample one → append → repeat

How language models are trained:

1. Take a huge amount of text (books, web, news)
2. For each position in the text, predict the next token
3. Adjust model weights to make correct predictions more likely
4. Repeat billions of times

We sample one word from this distribution, append it, and repeat — which is why the same prompt can give different answers each time.

Definition: Word Embedding

A **word embedding** is a numerical representation of a word as a vector of numbers, where words with similar meanings have vectors that are close together in the vector space.

City Analogy for Embeddings

Imagine plotting cities on a map. New York and Boston are close (similar: both US East Coast cities). London and Paris are close (both Western European capitals). New York and London are far apart (different continents).

Word embeddings do the same for words: “bullish” and “optimistic” are close. “crash” and “collapse” are close. “bullish” and “crash” are far apart. Now arithmetic works: $\text{King} - \text{Man} + \text{Woman} \approx \text{Queen}$.

- Early method: **Word2Vec** (Mikolov et al., 2013) — each word gets one fixed vector
- Problem: “bank” means different things in “river bank” vs. “investment bank” — same vector for both
- Solution: **contextual embeddings** (BERT, GPT) — different vectors depending on context

Definition: BERT

BERT (Bidirectional Encoder Representations from Transformers, Devlin et al. 2019) is a language model that reads text in *both directions* simultaneously, creating rich contextual representations of each word.

The Reading Direction Analogy

Imagine two ways to read the sentence: “The stock did not rise, it _____ .”

GPT-style (left-to-right): It reads each word in order. By the time it gets to the blank, it has processed all words to the left.

BERT-style (bidirectional): It sees the *entire* sentence at once and uses context from both sides. The blank follows “it” and ends the sentence — **BERT can see this structure fully**, producing a much richer understanding.

BERT's training objectives:

- **Masked Language Modeling:** randomly mask 15% of tokens, predict them
- **Next Sentence Prediction:** predict if two sentences follow each other

BERT's output:

- Contextual embeddings for each token
- A [CLS] token embedding representing the whole input
- Fine-tune [CLS] for classification tasks (sentiment, etc.)

What Is Fine-Tuning?

Definition: Fine-Tuning

Fine-tuning is the process of taking a pre-trained model (like BERT, trained on billions of tokens) and continuing to train it on a smaller, task-specific dataset to adapt it for a specific task.

Medical School Analogy

A medical student spends 4 years learning general medicine (pre-training). Then they do a residency in cardiology (fine-tuning). They don't forget general medicine — they just become specialized. The 4 years of general training would be too expensive to redo for every specialty.

Fine-tuning process:

1. Start with BERT (already trained on Wikipedia + books)
2. Gather labeled finance data (e.g., 10,000 sentences labeled positive/negative/neutral)
3. Train BERT on this data: update weights slightly
4. Result: FinBERT — finance-specialized model

Why it works:

- BERT already understands language structure, grammar, syntax
- Fine-tuning just adjusts it for the specific domain vocabulary and tone
- Works well with *small* labeled datasets (as few as 1,000 examples)
- Much cheaper than training from scratch

What Is FinBERT?

FinBERT (Yang et al., 2020) is a BERT model fine-tuned on financial text (financial news, SEC filings, earnings call transcripts) and labeled sentiment data, achieving state-of-the-art financial sentiment classification.

FinBERT in Action — Actual Output Format

Input: “The company reported EPS of \$2.45, beating consensus of \$2.00 by 22.5%.”

Label	Score	Interpretation
Positive	0.923	Strong buy signal
Neutral	0.041	—
Negative	0.036	—

Input: “Supply chain disruptions may impact margins for the next two quarters.”

Label	Score	Interpretation
Negative	0.881	Risk flag
Neutral	0.074	—
Positive	0.045	—

Training Data

- Financial PhraseBank: 4,840 sentences from Reuters, labeled by finance professionals
- Financial news: Wall Street Journal, Reuters
- SEC filings: 10-K, 10-Q, 8-K annual/quarterly/material event filings
- Earnings call transcripts

FinBERT vs. General BERT on Finance

On Financial PhraseBank: FinBERT achieves 88% accuracy vs. general BERT's 73%. Domain-specific fine-tuning matters enormously in finance.

★ From Accuracy to Trading Signal

FinBERT sentiment on earnings calls correlates with **5-day forward returns** at $r \approx 0.10\text{--}0.20$ (realistic; single signals are weak). Combined with fundamental factors, Sharpe ratios improve significantly. This is what Colab 1 measures directly.

What Is an Earnings Call?

Definition: Earnings Call

An **earnings call** is a quarterly conference call where a company's executives (CEO, CFO) present financial results to investors and analysts, followed by a Q&A session where analysts ask probing questions.

What Happens During an Earnings Call

1. **Opening:** CEO sets the tone — strategic overview, highlights
2. **Financial review:** CFO walks through revenue, profit, cash flow numbers
3. **Guidance:** Management gives their expectation for next quarter/year
4. **Q&A:** Analysts from Goldman Sachs, JPMorgan, etc. ask pointed questions. Management must answer live — no time to craft a perfect PR response

Why earnings calls are valuable for AI:

- Rich unscripted language (especially Q&A)
- Management hedging language reveals uncertainty
- Analyst tone reflects institutional sentiment

Available to AI:

- Transcripts: Seeking Alpha, Bloomberg, FactSet
- Audio: tone of voice carries additional signals
- Timing: reactions happen *during* the call

Beyond the Numbers: Linguistic Signals

The financial numbers in an earnings call are the headline. But the *language* carries additional signals that are harder to fake and require AI to extract.

Positive Signals

- **Specific, confident guidance:** “We expect revenue of \$4.2–4.4 billion next quarter”
- **Forward-looking investment language:** “We’re doubling R&D investment”
- **Analysts asking positive follow-ups:** “Could you elaborate on that strong margin expansion?”

Negative / Concern Signals

- **Hedging and uncertainty:** “depends on macro conditions,” “we’ll have to wait and see”
- **Vague guidance:** “we remain optimistic about the year” (no numbers given)
- **Analysts pushing back:** “Can you explain why margins compressed despite revenue growth?”
- **Script deviations in Q&A:** nervous laughter, long pauses, unusual word choice

- Huang et al. (2018): CEO presentation sentiment predicts future stock performance; analyst Q&A is even more informative (less scripted)

What Are SEC Filings?

Public US companies must regularly file detailed reports with the **Securities and Exchange Commission (SEC)**. These filings are public and contain rich textual information beyond the raw financial numbers.

10-K: Annual Report

- Filed once per year
- 50–300+ pages long
- Contains: business description, risk factors, financial statements, management discussion & analysis (MD&A)
- **Risk factor section:** company must disclose all material risks — changes here are highly informative

10-Q: Quarterly Report

- Filed 3 times per year (after Q1, Q2, Q3)
- Shorter than 10-K
- Contains quarterly financial statements and updated MD&A
- Changes from prior quarter's 10-Q reveal evolving management thinking

The EDGAR Database

All SEC filings are freely available at **edgar.sec.gov**. This is a gold mine for NLP: millions of documents, all public, timestamped precisely — ideal training and signal data for financial LLMs.

Definition: Forward-Looking Statements

Forward-looking statements are statements that express expectations about the future: “we expect,” “we believe,” “we anticipate,” “guidance suggests.” They are legally protected from liability (safe harbor rules) if accompanied by disclaimers.

What NLP Extracts from These

- Number of forward-looking statements vs. backward-looking: more forward-looking = management optimistic about future
- **Hedging ratio:** “may,” “might,” “could” vs. “will,” “expect,” “anticipate” — high hedging = uncertainty
- Year-over-year changes: the risk factors section in this year’s 10-K vs. last year’s reveals new concerns management added

Li (2010): Predictive Power

Frank Li (2010) showed that the fraction of “uncertain” words in annual reports predicts:

- Lower earnings persistence (harder to predict future profits)
- Higher future earnings volatility
- Negative future stock returns

Uncertainty in language = real fundamental uncertainty, not just legal hedging.

Step-by-Step: Turning Words into Trading Decisions

1. **Collection:** Gather raw text (earnings call transcript, 10-K filing, news article) at the moment it is released
2. **Preprocessing:** Clean text, remove headers/boilerplate, split into segments relevant to each company/topic
3. **Tokenization:** Convert text into tokens (sub-word units the model understands)
4. **Model inference:** Pass tokens through FinBERT (or similar) to get sentiment scores, uncertainty scores, topic classification
5. **Signal aggregation:** Combine model outputs into a single normalized signal score for each stock
6. **Alpha combination:** Blend the text signal with price-based signals, fundamental signals
7. **Portfolio construction:** Use the combined signal to decide which stocks to buy, hold, or sell
8. **Execution:** Submit orders to the market via execution algorithms (Section J)

Timing Is Critical (Next Slide)

Steps 1–7 must complete *faster than other traders* react to the same information, or the trade will be in the wrong direction by the time it executes.

The Stale Signal Problem

If your NLP system takes 5 minutes to process an earnings call transcript, but the stock has already moved 8% in those 5 minutes (based on the headline numbers), your sentiment signal arrives *after* the price has already incorporated it.

Timeline of an Earnings Release

- **T=0:** Earnings press release hits the wire
- **T=0 to 2min:** HFT systems react to headline numbers; large price move
- **T=2 to 10min:** Fast algorithmic reading of full release; initial NLP signals
- **T=10 to 30min:** Earnings call begins; real-time NLP of spoken text
- **T=30 to 60min:** Full call completes; deep analysis of Q&A
- **T=1 to 24h:** Slower analysis; potential for longer-horizon signals

LLMs at Each Stage

- **T=0–2min:** Too slow for LLMs (humans and HFT dominate)
- **T=2–30min:** Fast FinBERT inference possible; early sentiment signals
- **T=30min–24h:** LLMs can do deep analysis, extract nuanced Q&A signals
- **T=24h+:** LLMs synthesize across multiple sources, identify narrative shifts

The Need for Structured Financial Reasoning

Beyond sentiment, AI systems need to answer specific quantitative questions from financial documents: “What was the year-over-year change in operating margin?” This requires both reading comprehension *and* arithmetic.

FinQA (Chen et al., 2021)	TAT-QA (Zhu et al., 2021)	ConvFinQA (Chen et al., 2022)
• 8,281 Q&A pairs from SEC filings • Requires multi-step numerical reasoning • Tables + text passages • Example: compute a financial ratio from scattered numbers	• Hybrid: table and text in same context • Questions require combining table values with textual context • More complex reasoning than pure table Q&A	• Conversational: multi-turn Q&A • Follow-up questions reference prior answers • Mirrors how analysts actually probe documents • More realistic evaluation setting

GPT-4 Performance

GPT-4 achieves ~68% on FinQA (vs. 61% for specialized fine-tuned models). Yet there's still a gap versus human performance (~91%). Structured financial reasoning remains challenging.

The Multi-Modal Challenge

Financial documents contain not just text and tables but also **charts and graphs**: revenue trends, margin evolution, geographic breakdowns. AI systems that can't interpret these miss important signals.

ChartQA (Masry et al., 2022)

- 9,608 Q&A pairs based on chart images
- Requires reading values from bar charts, line charts, pie charts
- Combination of visual understanding + reasoning
- Harder: numbers must be visually estimated, not read exactly

Example Question

Chart shows quarterly revenue from 2019–2023.
Question: “In which year was the growth rate highest?”
Required: Read all bar heights, compute year-over-year changes, identify maximum. This requires visual understanding + arithmetic reasoning.

Why This Matters for Finance

Company investor presentations are heavy with charts. An AI that can't read charts misses a significant portion of the information content of financial presentations. GPT-4V, Gemini, and specialized models (FinMME) are making progress, but accuracy on financial charts remains below human performance.

Definition: Calibration

A model is **well-calibrated** if, when it says it is 80% confident, it is correct 80% of the time. Poor calibration = the model's stated confidence does not match its actual accuracy.

The Weather Forecaster Analogy

A well-calibrated weather forecaster: when they say "70% chance of rain," it actually rains 70% of the time on such days. An overconfident forecaster says "95% chance of rain" but is right only 70% of the time. In finance: an overconfident model says "I'm 95% sure this is a buy signal" when it's right only 70% of the time — causing you to bet too much and lose more than expected.

Why calibration matters in finance:

- Confidence determines *position size*: bigger bet on higher confidence signals
- Overconfident models = overly large bets = large losses when wrong
- LLMs are notoriously overconfident (they state things confidently even when hallucinating)

Calibration methods:

- Temperature scaling: adjust softmax output
- Platt scaling: train a logistic regression on model outputs
- Conformal prediction: statistical guarantees (covered in Section F)

New Benchmarks: FinBen, XFinBench, FinanceBench

The Benchmark Arms Race

As LLMs have improved, older benchmarks have become saturated (models score near 100%). New, harder benchmarks are needed to distinguish frontier models.

FinBen (2024)	XFinBench (2025)	FinanceBench (2023)
36 tasks across 24 datasets - Tests: information extraction, text analysis, prediction, decision-making - Multi-lingual: English and Chinese financial text - Open-source and regularly updated	4,235 graduate-level finance problems with multi-modal context (text, tables, charts) - Tests five capabilities: terminology, temporal reasoning, forecasting, scenario planning, numerical modelling - Best text-only model (o1): 67.3% — ~12.5pp behind human experts - Weakest areas: temporal reasoning and scenario planning	150 expert-crafted questions from real 10-K/10-Q filings - Tests factual accuracy against ground truth from filings - Specifically tests for hallucination vs. accurate retrieval - GPT-4: ~61–79% (varies by retrieval condition); human CPAs: ~95%

The Benchmark Gap = Risk

A model's benchmark score does not equal deployment readiness. Real-world documents are messier, context is missing, and mistakes are costly. Public benchmarks also suffer **contamination**: test questions leak into training data, so high scores may reflect memorization rather than generalization. Always evaluate on your specific use case.

★ LLMs as Trading Signals: Lopez-Lira & Tang (2023)

ChatGPT sentiment on news headlines predicts **next-day stock returns** with ~60% directional accuracy (vs. 50% random), generating ~4% annualized abnormal return — but *only before arbitrage erodes the edge* (effect decays as more funds adopt similar strategies).

What You'll Build

A complete sentiment analysis pipeline that:

1. Downloads earnings call transcripts from the SEC EDGAR full-text search API
2. Preprocesses text: clean, segment, tokenize using FinBERT's tokenizer
3. Fine-tunes FinBERT on a labeled dataset of earnings call sentences
4. Evaluates: accuracy, precision, recall, and calibration curve
5. Runs inference on a new earnings call to generate a sentiment score
6. Correlates the sentiment score with subsequent 1-day, 5-day, and 20-day returns

Colab Notebook 1

Available at the tutorial GitHub repository. Runs on free Google Colab GPU. Takes ~45 minutes end to end. No local setup required.

Learning Outcomes

After completing Assignment 1, you will understand:

- The full NLP pipeline for financial text
- How fine-tuning improves domain performance
- Why sentiment $\neq$ return (but correlates imperfectly)
- The importance of proper train/test splits respecting time

What We Have Covered

Section A: Markets

How financial markets function: supply/demand, price formation, types of participants, why they are hard for ML.

Section B: Microstructure

The mechanics of trading: order books, bid-ask spread, adverse selection, price impact, Flash Crash, GameStop.

Section C: Behavioral Finance

Human biases: prospect theory, disposition effect, momentum, herding — each creating predictable price patterns.

Section D: Financial NLP

Turning text into signals: word lists, embeddings, BERT, FinBERT, earnings calls, SEC filings, benchmarks.

The Foundation Is Set

You now understand *why* markets are information-driven human behavioral systems, and *how* to extract signals from text. Part 2 shows how to build sophisticated AI systems on top of these foundations.

What Changes in Part 2

Part 1 gave us the building blocks:

- **Behavioral signals:** what patterns exist and why
- **Text signals:** how to extract sentiment from documents
- **Microstructure understanding:** the mechanics of trading

Part 2 takes these building blocks and asks: *how do we build a complete AI system that goes from raw information to trading decisions?*

The Gap

A sentiment score is not a trading system. You need:

- An LLM that understands context (not just sentiment)
- A way to prevent hallucination (RAG)
- A system that plans and acts (agents)
- Multiple specialized agents that coordinate (multi-agent)

Part 2 Sections

- **E:** LLMs from first principles — the engine
- **F:** RAG — the knowledge retrieval system
- **G:** Agentic AI — the planning layer
- **H:** Multi-agent systems — the coordination layer

Part 2

Advanced Techniques

Section E: LLMs from First Principles

Slides 99–188

Presenters: Lahlou (RAG & uncertainty) • Iklassov (agents & multi-agent)

What Is a Token?

Tokens: The Currency of LLMs

We introduced tokenization in Section D (Slide 80). Here we go deeper on two dimensions that matter most for building LLM-based finance systems: **token economics** (cost and context limits) and **finance-specific tokenization quirks**.

Token Examples Using GPT Tokenizer

"The Federal Reserve raised interest rates by 25 basis points."

Tokens: ["The", "Federal", "Reserve", "raised", "interest", "rates", "by", "25", "basis", "points", "."] = **11 tokens**

"EBITDA" → ["EB", "IT", "DA"] = **3 tokens** (uncommon financial term split)

"USD" → ["USD"] = **1 token** (common enough to have its own token)

Token economics:

- GPT-4: ~\$0.03 per 1,000 input tokens
- A 10-K filing: ~50,000–200,000 tokens
- Processing one 10-K with GPT-4: \$1.50–\$6.00
- Processing 10,000 10-Ks per quarter: \$15,000–\$60,000

Context window:

- LLMs can only process a fixed number of tokens at once
- GPT-3.5: 16K tokens (~12,000 words)
- GPT-4: 128K tokens (~96,000 words)
- Claude-3: 200K tokens (~150,000 words)
- Long 10-Ks may exceed even 200K tokens

Definition: Attention Mechanism

Attention is the core mathematical operation of Transformers. It allows the model to focus on the most *relevant* parts of the input when processing each token — deciding which other tokens are most important for understanding the current token.

The Spotlight Analogy

Imagine reading the sentence: “The bank near the river had steep walls.”

When processing the word “steep,” you intuitively focus on “walls” (it describes walls) and “bank” and “river” (context that tells you this is a riverbank, not a financial bank). Your attention *spotlights* the relevant words.

The Transformer’s attention mechanism does exactly this — mathematically, for every token, simultaneously.

Why attention is revolutionary:

- Before attention: RNNs processed text sequentially — long-range dependencies were forgotten
- With attention: every token can directly attend to every other token, regardless of distance

In finance:

- Attention allows the model to connect a risk factor mentioned on page 3 of a 10-K to a financial result on page 40
- Critical for multi-document reasoning across earnings calls

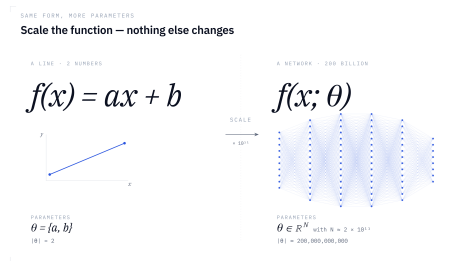
The Transformer Architecture in One Slide

Vaswani et al. (2017): “Attention Is All You Need”

The **Transformer** is the neural network architecture underlying virtually all modern LLMs. It replaces recurrence with attention.

Key components:

1. **Token embeddings**: convert tokens to vectors
2. **Positional encoding**: add information about word order (since attention is position-agnostic)
3. **Multi-head attention**: H parallel attention computations (~ 8 –96 heads)
4. **Feed-forward network**: per-token non-linear transformation
5. **Layer normalization**: stabilize training
6. **Residual connections**: allow gradients to flow
7. **Stack**: repeat steps 3–6 for L layers (~ 12 –96 layers)



Scale of Modern LLMs

Model	Parameters
BERT-base	110M
GPT-2	1.5B
GPT-3	175B
GPT-4	undisclosed (>1T est.)
Llama-3 70B	70B
BloombergGPT	50B

Self-Attention: Every Word Looks at Every Other Word

How Self-Attention Works (with Formula)

For each token, self-attention computes three vectors: a **Query** (what I'm looking for), a **Key** (what I offer), and a **Value** (the information I contribute). The output is a weighted sum of values, where weights measure query-key similarity:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Dividing by $\sqrt{d_k}$ prevents the dot products from becoming too large and killing the softmax gradients.

The Library Catalog Analogy

Imagine you are in a library:

- Your **query** is what you search for: “risk factors for banks”
- Each book has a **key** (title/summary): “Banking Regulation,” “Credit Risk,” “Stock Valuation”
- The **values** are the actual content of each book
- You attend to (read) the books whose keys best match your query

Self-attention does this *simultaneously* for every token, creating a rich contextual representation.

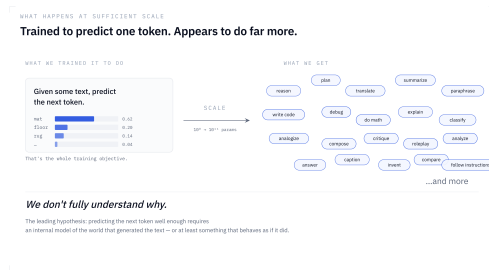
Multi-Head Attention

Using H parallel attention heads allows the model to simultaneously attend to *different types* of relationships (grammatical, semantic, positional) between tokens. Different heads specialize in different linguistic relationships.

Scaling: Why Bigger Models Are Smarter

The Scaling Laws (Kaplan et al., 2020)

Neural language model performance scales *predictably* with three factors: number of **parameters** (model size), amount of **training data**, and amount of **compute**. Bigger = better, in a smooth, predictable way.



At sufficient scale, abilities *not present* in smaller models appear: multi-step reasoning, instruction following, chain-of-thought. The model *appears* to reason — we don't fully understand why; predicting text well enough seems to force an internal representation of the world that generated it.

Finance Implication

Large LLMs can reason about complex financial scenarios that smaller models cannot, but cost far more to run — cost-performance tradeoffs are critical in production. Since 2024 there is also a **third scaling axis**: *inference-time compute* — reasoning models (OpenAI o1, DeepSeek-R1) think longer before answering instead of just growing bigger.

GPT = Generative Pretrained Transformer

GPT models (Radford et al., 2018; 2019; Brown et al., 2020) are **decoder-only** Transformers trained to predict the next token given all previous tokens. They generate text *autoregressively*: one token at a time, left to right.

Text Generation Process

Prompt: "The company's Q3 revenue was"

→ GPT predicts: " \$4.2"

→ New input: "The company's Q3 revenue was \$4.2"

→ GPT predicts: " billion"

→ And so on, until a stop token...

Why GPT Is Useful for Finance

- Generates analyst-style commentary
- Completes financial documents
- Answers questions in natural language
- Summarizes long filings

GPT-4 Capabilities in Finance

- Reads and summarizes SEC filings
- Answers quantitative questions from financial tables (partially)
- Generates investment memos
- Codes financial analysis in Python
- Explains financial concepts in plain language
- Translates filings into multiple languages

GPT's Main Risk

GPT generates plausible-sounding text. It does not inherently verify factual accuracy. In finance, a plausible wrong answer is dangerous.

BERT vs. GPT: Architectural Difference

- **BERT:** Encoder-only. Reads the whole text at once, bidirectionally. Produces a contextual representation of each token. *Not* designed for generation.
- **GPT:** Decoder-only. Reads left to right. Designed for generation. Cannot naturally use right-context.
- **T5, BART:** Encoder-decoder. Can both understand (encode) and generate (decode).

When to Use Each

Use BERT/FinBERT for:

- Classification (sentiment: positive/negative/neutral)
- Named entity recognition (find company names, numbers)
- Embedding generation (turn text into vectors for retrieval)

Use GPT for:

- Report generation
- Question answering (generative)
- Summarization
- Instruction following

Practical Tip

In many finance pipelines, BERT-style models handle retrieval and classification while GPT-style models handle generation. The two architectures are *complementary*, not competing.

Definition: Instruction Tuning

Instruction tuning (also called supervised fine-tuning or SFT) is the process of fine-tuning a base LLM on thousands of (instruction, response) pairs to teach it to *follow instructions* rather than just complete text.

Before Instruction Tuning (Base Model)

Prompt: "Summarize this earnings call."

Response: "Summarize the meeting. Summarize the document. Summarize the..." (just continues the word pattern)

After Instruction Tuning

Prompt: "Summarize this earnings call."

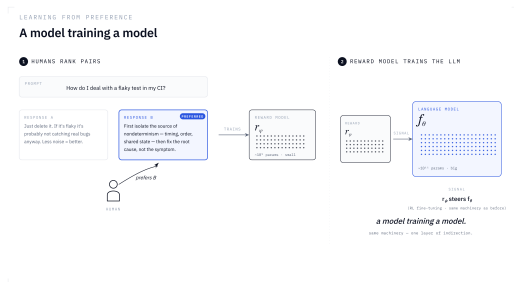
Response: "Apple reported Q3 revenue of \$85.8 billion, a 1% year-over-year increase, driven by strong Services growth of 8%..." (actually follows the instruction)

Why This Matters for Finance

Instruction-tuned models can be given complex, multi-step financial analysis instructions: "Read this 10-K. Identify the three biggest risk factors. For each, assess whether this risk is new compared to last year's filing. Format as a JSON object."

Definition: RLHF

Reinforcement Learning from Human Feedback (RLHF) is a training technique where a language model is improved by having humans evaluate its outputs and using those evaluations as a training signal.



Humans rank pairs of responses; a smaller **reward model** learns to predict their preferences and steers the big model — *a model training a model*.

RLHF Results in:

- More helpful responses
- Fewer harmful outputs
- Better instruction following
- More honest acknowledgment of uncertainty

Finance-Specific RLHF:

- Human raters = finance domain experts
- Reward model learns: accurate + calibrated = good; hallucinated numbers = very bad
- Critical for reducing financial hallucination risk

Definition: Prompt Engineering

Prompt engineering is the practice of designing input text (the “prompt”) to elicit the best response from an LLM. The same model can give vastly different quality answers depending on how the question is framed.

Poor Prompt

“Analyze this stock.”

Result: vague, generic response with no actionable content; likely includes hallucinated statistics.

Better Prompt

“You are a fundamental equity analyst. Given the following Q3 earnings transcript, identify: (1) the 3 most significant positive surprises, (2) the 3 most significant risks, (3) management’s tone on guidance using direct quotes. Be specific and cite numbers. [transcript]”

Key Prompting Principles

- **Role assignment:** “You are a CFA-level analyst...”
- **Specific output format:** “Respond as JSON with fields: sentiment, confidence, key_quotes”
- **Constraint specification:** “Only use information from the provided text. Do not use prior knowledge.”
- **Task decomposition:** break complex tasks into sub-steps
- **Few-shot examples:** show the model what a good answer looks like

Chain-of-Thought Prompting: Show Your Work

Definition: Chain-of-Thought (CoT) Prompting

Chain-of-thought prompting (Wei et al., 2022) encourages LLMs to generate *intermediate reasoning steps* before giving a final answer, similar to showing your work in a math exam.

Without Chain-of-Thought

Q: "If revenue grew 12% in Q3 and the analyst expected 8%, and the stock was at \$100, what is the expected move?"

A: "The stock should rise to \$115." (wrong and unexplained)

With Chain-of-Thought

Q: Same as above + "Let's think step by step."

A: "Revenue beat by $12\% - 8\% = 4$ percentage points. Based on historical earnings response, a 1pp beat typically causes a $\sim 0.5\%$ stock move. So expected: $4 \times 0.5\% = 2\%$ move. Therefore estimate $\approx \$102$. However, other factors..."

Why CoT Improves Finance Performance

- Forces explicit multi-step reasoning
- Reduces arithmetic errors (the model "shows work" so errors are caught)
- Improves calibration: reasoning that doesn't add up is visible
- Makes errors *debuggable*: you can see where the reasoning went wrong
- Scales better than direct answers for complex financial questions

From Prompt Trick to Reasoning Models

Zero-shot CoT (Kojima et al., 2022): merely appending "*Let's think step by step*" — no examples needed — can improve a small model's accuracy on math-style benchmarks by roughly $2.5\times$. Modern **reasoning models** (OpenAI o1, DeepSeek-R1) bake this in: they are trained to generate a long internal chain of thought before answering.

Definition: Few-Shot Prompting

Few-shot prompting includes 2–10 input-output examples in the prompt, teaching the model the desired response format and reasoning style through demonstration rather than explicit instructions.

Few-Shot Sentiment Classification

Example 1:

Text: "Revenue exceeded analyst estimates by 15%." → Sentiment: Positive

Example 2:

Text: "The company faces significant liquidity constraints." → Sentiment: Negative

Example 3:

Text: "Operating margin remained stable at 18%." → Sentiment: Neutral

Query:

Text: "Customer acquisition costs rose 40% while retention rates dropped 5%." → Sentiment: ?

When to Use Few-Shot vs. Fine-Tuning

- **Few-shot:** quick iteration, no training cost, flexible, but limited by context window
- **Fine-tuning:** better performance on specific task, cheaper at inference, but requires labeled data and training time

Definition: Hallucination

An LLM **hallucination** is when the model generates text that is *factually incorrect* but stated confidently, as if it were true. The model “makes up” plausible-sounding information.

A Real Finance Hallucination Example

Prompt: “What was Apple’s Q2 2024 revenue?”

LLM response: “Apple reported Q2 2024 revenue of \$89.5 billion, driven by strong iPhone sales in China and record-breaking Services revenue of \$24.1 billion.”

The actual numbers may be completely fabricated — the model knows it should give specific numbers and does so, even if it doesn’t actually know the correct figures.

Why LLMs hallucinate:

- Trained to produce fluent, coherent text
- No mechanism to distinguish “I know this” from “I’m guessing”
- Fill gaps with plausible patterns from training data
- Outdated training data + knowledge cutoffs

How common is it?

- Studies find 3–20% hallucination rates on factual questions
- Higher for specific numbers, dates, and names
- Very common for questions about events after training cutoff
- Finance is especially high-risk: lots of specific numbers

Why Hallucinations Are Catastrophic in Finance

The Stakes Are Different in Finance

In a chatbot giving cooking advice, a hallucination is annoying. In a financial AI system, a hallucination could cost millions of dollars, trigger regulatory violations, or cause cascading market events.

Hallucination Scenarios

1. AI analyst reports wrong earnings figure → fund bets on wrong direction → large loss
2. AI generates compliance memo with wrong regulatory threshold → regulatory violation
3. AI risk model invents non-existent market correlation → portfolio is underhedged
4. AI trading agent cites hallucinated news event as justification for large trade → wrong trade based on nothing

Defense Mechanisms

- **RAG** (Section F): retrieve actual documents before answering
- **Source citation**: require the model to cite specific text it used
- **Self-consistency**: ask the same question multiple times; if answers differ, it's uncertain
- **Fact-checking**: verify outputs against structured databases
- **Human oversight**: all significant decisions reviewed by humans

The Core Fix

Section F on RAG addresses hallucination directly: by giving the model access to real-time verified information, we can dramatically reduce factual errors.

BloombergGPT (Wu et al., 2023)	FinGPT (Yang et al., 2023)	InvestLM (Yang et al., 2023)
• 50B parameter LLM • Trained on 363B tokens of financial text from Bloomberg's proprietary archive + 345B general text • First large-scale finance-domain LLM • Outperforms same-size general models on finance benchmarks • Proprietary: not publicly available	• Open-source finance LLM framework • Built on top of LLaMA/Falcon base models • Continuously updated with real-time financial data • Designed for fine-tuning on specific finance tasks • Publicly available on GitHub	• Fine-tuned on financial instruction data • Specifically designed for investment analysis tasks • Tests show strong performance on FinQA, ConvFinQA • Smaller than BloombergGPT: more accessible for researchers

The Domain Pre-Training vs. Fine-Tuning Debate

BloombergGPT was pre-trained from scratch on finance text (expensive: ~\$1M+). More recent evidence suggests fine-tuning GPT-4 or Llama-3 on finance data achieves similar results at a fraction of the cost. The optimal approach is still an open research question.

What Can LLMs Do in Finance? A Taxonomy

Information Extraction

- Named entity recognition: companies, dates, amounts
- Financial event extraction: mergers, dividends, guidance changes
- Risk factor identification from 10-K filings
- Earnings surprise detection from transcripts

Generation

- Investment memo drafting
- Earnings summary generation
- Risk report generation
- Client communication drafting
- Code generation for analysis scripts

Analysis & Reasoning

- Multi-document synthesis: combine 10 analyst reports
- Financial Q&A over documents
- Scenario analysis: "What if interest rates rise 100bp?"
- Comparative analysis: "Compare this quarter to the same quarter last year"

Decision Support

- Trade idea generation (preliminary)
- Portfolio commentary generation
- Alert generation from monitoring
- Compliance checking

What LLMs Cannot Do Reliably

Make autonomous trading decisions without human oversight, guarantee factual accuracy, predict future prices.

LLM Limitations: Context Window, Knowledge Cutoff, Cost

Limitation 1: Context Window

LLMs can only “see” a fixed amount of text at once. A 128K token window holds ~200 pages. But 10-K filings can be 400+ pages. Annual earnings call transcripts for a portfolio of 100 stocks exceed any context window.

Solution: Chunking + RAG (Section F)

Limitation 2: Knowledge Cutoff

LLMs are trained on data up to a certain date. They know nothing about events after that date. A model with a January 2024 cutoff knows nothing about a March 2024 earnings surprise.

Solution: RAG with live data feeds

Limitation 3: Cost

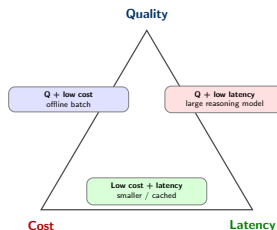
Running GPT-4 on millions of financial documents daily is prohibitively expensive. For real-time signals, cost per inference is critical.

Solution: Smaller specialized models + caching

Limitation 4: Latency

LLMs take 1–30 seconds to generate a response. This is too slow for intraday signal generation in most strategies.

Solution: Use LLMs for longer-horizon signals; use faster models (DistilBERT, etc.) for real-time



Pick two: quality, cost, latency — you cannot optimize all three.

Working the Triangle in Production

- **Routing / cascades:** easy queries → small cheap model; hard ones → the large model
- **Caching:** much real traffic is repetitive; high hit rates cut serving cost by orders of magnitude
- **Distillation:** compress a large model's behavior into a small student
- **Early exit:** stop generating once confidence is high

Section F

Retrieval-Augmented Generation (RAG)

“Grounding LLMs in Facts”

Slides 117–142

The Core Tension

LLMs are powerful reasoners but unreliable fact-keepers. In finance, *facts are everything*. A system that reasons brilliantly from wrong facts produces worse outcomes than a system that doesn't reason at all.

What LLMs Know (Parametric Knowledge)

Information baked into the model's weights during training.

Examples:

- General financial concepts
- Historical events (before cutoff)
- Common reasoning patterns

Reliability: Good for concepts, poor for specific numbers, dates, and recent events.

What LLMs Don't Know

- Yesterday's earnings reports
- Current prices and spreads
- The specific 10-K filing you're working with
- Regulatory changes from last month
- Anything after the training cutoff

But: the model will answer anyway, confidently, making up plausible numbers.

The Solution

Retrieval-Augmented Generation (RAG): before answering, fetch the relevant facts from a verified external knowledge base, and include them in the prompt.

Lewis et al. (2020): Retrieval-Augmented Generation

RAG combines two capabilities: a **retrieval system** that finds relevant documents from a knowledge base, and a **generation model** that produces an answer using the retrieved documents as context.

The Open-Book Exam Analogy

Without RAG, an LLM is like a student taking a **closed-book exam**. They rely entirely on what they memorized. Gaps in memory lead to guesses — sometimes confidently wrong ones.

With RAG, it's an **open-book exam**. Before answering, the student looks up the relevant pages. They can now answer accurately even for obscure details they didn't memorize — as long as the textbook is accurate and they can find the right page.

RAG enables:

- Citing specific passages from real documents
- Answering questions about recent events (after training cutoff)
- Factual accuracy on specific numbers, dates
- Auditability: “here is the source”

RAG requires:

- A knowledge base (indexed documents)
- A retrieval mechanism (semantic search)
- Good chunking of documents
- A generation model that uses retrieved context faithfully

Two Phases: Indexing and Querying

Phase 1: Indexing (Done Once)

1. Collect documents (10-K filings, earnings calls, news)
2. Split into chunks (e.g., 512-token paragraphs)
3. Embed each chunk using an embedding model (BERT, text-embedding-3, etc.)
4. Store embeddings + original text in a vector database (FAISS, Pinecone, Weaviate)

Phase 2: Querying (At Runtime)

1. User submits a question
2. Embed the question using the *same* embedding model
3. Search vector database for the k most similar chunks (e.g., $k = 5$)
4. Construct a prompt: “[System context] + [Retrieved chunks] + [Question]”
5. Send to LLM, get answer grounded in retrieved facts

Finance RAG Example

Question: “What risks did Tesla cite in its most recent 10-K?”

RAG retrieves: the Risk Factors section of Tesla’s latest 10-K

LLM answers: uses only the retrieved text, cites specific sections

Result: accurate, up-to-date, verifiable.

What Goes Into the Knowledge Base?

The knowledge base is only as good as what you put in it. The documents must be:

- **Accurate:** verified data sources (EDGAR for filings, Bloomberg for prices, official press releases)
- **Timely:** continuously updated — a knowledge base that's 6 months old defeats the purpose
- **Relevant:** focused on the use case — don't index cooking blogs for a financial AI
- **Well-structured:** tables, charts, and lists need special handling (they don't embed well as raw text)

Typical Financial RAG Sources

- SEC EDGAR: 10-K, 10-Q, 8-K filings
- Bloomberg/Reuters wire: real-time news
- Earnings call transcripts
- Analyst research reports
- Fed/ECB/BoE policy documents
- ISDA/FINRA regulatory guidance

Indexing Pitfalls

- Including low-quality or unverified sources contaminates responses
- Not updating the index means stale information
- Poor document parsing (e.g., broken PDF parsing) creates garbage chunks that confuse retrieval
- Duplicate documents create redundant, confusing context

What Is a Vector Embedding?

Meaning as Coordinates

A **vector embedding** is a list of numbers (e.g., 768 or 1536 numbers) that represents the *meaning* of a piece of text. Texts with similar meanings have embeddings that are close to each other in this 768-dimensional space.

The City Coordinates Analogy

Think of a city on a map. Its location is described by two numbers (latitude, longitude). Cities near each other have similar coordinates. Similarly, a text's "semantic location" is described by 768 numbers, and similar texts have similar coordinates.

Text	Neighbors (similar embeddings)
"Revenue increased"	"Sales grew," "Top-line expansion"
"Credit default"	"Bond default," "Debt restructuring"
"Q3 earnings beat"	"EPS above consensus," "Profit surprise"

How They Are Created

Embedding models (e.g., text-embedding-3-large, all-mpnet-base-v2) are BERT-like models trained specifically to produce high-quality semantic embeddings. They compress text into dense vectors via the [CLS] token or mean pooling.

Cosine Similarity: Measuring Meaning Distance

Definition: Cosine Similarity

Cosine similarity measures the angle between two vectors. A cosine similarity of 1.0 means the vectors point in the same direction (identical meaning); 0 means perpendicular (unrelated); -1 means opposite (antonyms).

Why “Cosine” Rather Than Euclidean Distance?

Euclidean distance cares about *magnitude*: a long document and a short document about the same topic might have different-sized vectors, giving misleading distances. Cosine similarity only cares about the *direction* — normalized by length. This gives better semantic matching.

Query	Retrieved Chunk	Cosine Similarity
“Apple revenue Q3 2024”	“Apple’s Q3 2024 net sales were...”	0.94
“Apple revenue Q3 2024”	“iPhone market share analysis...”	0.71
“Apple revenue Q3 2024”	“EU AI regulation update...”	0.18

In Practice

We retrieve the top- k chunks with highest cosine similarity to the query. Typical $k = 3$ – 10 . Too few = missing context; too many = LLM distracted by irrelevant text.

Step 2: Retrieval — Finding Relevant Passages

The Retrieval Challenge

Finding the right information quickly is harder than it sounds. The knowledge base may contain millions of chunks. Finding the 5 most relevant ones to a specific question requires efficient search.

Dense Retrieval

- Embed both the query and all chunks
- Find chunks with highest cosine similarity
- **Pros:** captures semantic similarity (synonyms, paraphrases)
- **Cons:** requires embedding all chunks in advance; slow if knowledge base changes frequently

Sparse Retrieval (BM25)

- Keyword-based matching: finds chunks that contain query words
- Classic information retrieval method from the 1970s
- **Pros:** fast, exact term matching, handles proper nouns/tickers well (“AAPL,” “Q3FY2024”)
- **Cons:** misses synonyms; “revenue” won’t match “net sales”

Hybrid Retrieval (Best Practice)

Combine both: use BM25 for exact term matching (essential for financial jargon, tickers, specific dates) plus dense retrieval for semantic similarity. Re-rank the combined results with a cross-encoder for best quality.

Concrete Finance Example

Query: "What were Apple's iPhone unit sales in fiscal Q3 2024?"

BM25 (Sparse) Would Retrieve:

Chunks containing the exact words "Apple," "iPhone," "unit sales," "fiscal," "Q3," "2024."

Good at: retrieving the exact press release paragraph mentioning "Q3 FY2024 iPhone sales."

Misses: documents that say "handset shipments" instead of "unit sales."

Dense (Semantic) Would Retrieve:

Chunks with similar embeddings to the query — may match:

- "Apple Q3 FY2024 handset shipments"
- "iPhone sales volume quarterly"
- "AAPL unit economics fiscal Q3"

Good at: capturing synonyms and paraphrases.

Misses: may retrieve semantically similar but factually different chunks (wrong quarter, wrong product).

Practical Takeaway

In financial RAG systems, exact term matching is critical for specific numbers and dates. Always include a BM25 or lexical component in retrieval — don't rely solely on semantic embeddings.

ColBERT: Late Interaction for Better Matching

The Problem with Bi-Encoder Dense Retrieval

Standard dense retrieval encodes the query into a *single vector* and compares it to single-vector chunk embeddings. This loses fine-grained token-level matching.

ColBERT (Khattab & Zaharia, 2020)

ColBERT (Contextualized Late Interaction over BERT) preserves *all token embeddings* from both query and document. Relevance is computed as the sum of maximum similarities between each query token and the best-matching document token.

Why Late Interaction Helps in Finance

Query: "Apple iPhone revenue Q3 2024"

Chunk: "In the third quarter of fiscal year 2024, Apple's iPhone business generated revenue of..."

ColBERT token-level matching:

- "revenue" ↔ "revenue" (exact token match, high score)
- "2024" ↔ "2024" (exact token match, high score)
- "Q3" ↔ "third quarter" (**contextual**: BERT embeddings for "Q3" and "third quarter" are close in the same training corpus, so their dot product is high — but this is BERT's contextual knowledge, not magic synonym resolution)
- "iPhone" ↔ "iPhone" (exact match)

Single-vector dense retrieval squashes all these into one vector, losing which specific tokens matched.

Important Clarification

ColBERT does **not** automatically know that "Q3" means "third quarter." What it does: it computes BERT-contextualized embeddings, and BERT — having seen both phrases in its training data — produces similar embeddings for them. The semantic bridge comes from BERT's pre-training, not from ColBERT's matching mechanism itself.

The Generation Prompt Structure

A Well-Structured RAG Prompt

[SYSTEM]

You are a financial analyst assistant. Answer questions using ONLY the provided context. If the context does not contain the answer, say "I don't have that information in the provided documents." Never make up numbers.

[RETRIEVED CONTEXT]

[Chunk 1]: Apple's Q3 FY2024 10-Q, page 3: "Net sales for the third quarter of 2024 were \$85.8 billion, compared to \$81.8 billion in the third quarter of 2023."

[Chunk 2]: Apple Q3 2024 Press Release: "iPhone net sales were \$39.3 billion for the quarter."

[QUESTION]

What was Apple's total revenue and iPhone revenue in Q3 FY2024?

[ANSWER]: Apple's total revenue in Q3 FY2024 was \$85.8 billion (up from \$81.8 billion in Q3 FY2023), and iPhone revenue was \$39.3 billion. [Source: Apple 10-Q, Apple Q3 2024 Press Release]

Key Design Rule

Force the model to cite sources and refuse to answer without evidence. Never let it fall back on parametric knowledge for financial facts.

What Is “Chunking”?

Definition: Chunking

Chunking is the process of splitting long documents into smaller, manageable pieces (chunks) that can be individually embedded and retrieved. The goal: each chunk should be small enough to embed meaningfully but large enough to contain a complete thought.

Common Chunking Strategies

- **Fixed-size:** every N tokens (simple but may split sentences mid-thought)
- **Sentence-based:** split on sentence boundaries (natural, but unequal sizes)
- **Paragraph-based:** split on paragraph breaks (best for narrative documents)
- **Semantic:** use a smaller model to detect topic boundaries (expensive but best quality)

Chunking a 10-K Filing

Good approach for a 10-K:

- Parse by section: “Business,” “Risk Factors,” “MD&A,” etc. are natural boundaries
- Each major section becomes one or more chunks
- Tables get special treatment (see slide 130)
- Add metadata to each chunk: company, filing date, section name

Overlap Is Important

Always add a small overlap (e.g., last 50 tokens of previous chunk = first 50 tokens of next chunk) to avoid losing context at boundaries.

The Goldilocks Problem

Chunks that are too small lose context. Chunks that are too large dilute the relevant information and waste the LLM's context window. Getting chunk size right is critical.

Too Small (50 tokens)	Just Right (~256–512 tokens)	Too Large (2,000+ tokens)
• Loses sentence context • Single number without its explanation • “\$4.2 billion” with no context about what it refers to • High retrieval precision but low recall	• Complete paragraph or table row • Contains the fact plus surrounding context • The question “What was revenue?” retrieves a paragraph explaining it • Good balance of precision and recall	• Contains many topics mixed together • Embedding averages over many ideas, becoming vague • The retrieved chunk includes the relevant fact but also 1,000 tokens of noise • Wastes LLM context window

Best Practice: Multi-Granularity Indexing

Store both small chunks (for precise embedding) and larger parent chunks (for full context). Retrieve small chunks, but send the parent chunk to the LLM. This is the “parent-child chunking” or “small-to-big” strategy.

The Table Problem

Financial documents are full of tables: income statements, balance sheets, segment breakdowns, quarterly comparisons. Plain text chunking cannot handle these — a table needs special treatment.

Naive Approach (Broken)

Convert table to text by reading row by row: "Revenue 2023 2022 2021. Product 50B 45B 40B. Services 20B 17B 15B."

Problems:

- Column alignment is lost
- Relationships between cells are unclear
- Embedding of this text is poor

Better Approaches

- **Markdown tables:** convert to markdown format — LLMs trained on markdown read tables better
- **Natural language serialization:** "In 2023, Product revenue was \$50B, up from \$45B in 2022"
- **Hybrid text+table models:** TAT-QA trains models specifically to handle mixed contexts
- **Structured extraction first:** extract tables to a database, use SQL to answer numeric questions

Best Practice for Finance

Extract tables to a structured format (JSON/CSV) and use specialized table-reading models or SQL engines for quantitative questions. Reserve the LLM for qualitative/narrative questions.

The Vision-Language Challenge

Many financial documents include charts (bar graphs, line plots, scatter plots). Text-based RAG systems completely ignore these. Multi-modal models are needed.

Multi-Modal RAG Architecture

1. **Extract charts** from PDFs/presentations as images
2. **Encode with vision model** (CLIP, GPT-4V): get image embedding
3. **Index image embeddings** alongside text embeddings
4. **Retrieve relevant charts** alongside relevant text passages
5. **Generate answer** from multi-modal context: text + chart images

FinMME Benchmark (2024)

FinMME tests multi-modal reasoning on financial charts and tables. Key findings:

- GPT-4V: ~72% on financial chart questions
- Gemini 1.5 Pro: ~69%
- Human performance: ~91%
- The gap is still large for complex reasoning over charts

Practical Note

For most current production finance systems: extract data from charts to structured format (using OCR + vision models) and treat as a table. Full end-to-end multi-modal RAG is still emerging.

Liu et al. (2023): “Lost in the Middle”

LLMs pay more attention to information at the **beginning and end** of their context window. Information placed in the *middle* of a long context is systematically under-attended — and therefore often **ignored** in the model’s answer.

The Library Analogy

Imagine you are handed a stack of 20 pages to read and then asked a question. You carefully read the first few pages and the last few pages. But you skim the middle pages. If the critical fact is on page 11, you miss it.

The Problem:

- RAG retrieves $k = 10$ relevant chunks
- The most relevant one ends up in the middle of the prompt
- Model produces wrong answer because it attended to less-relevant chunks at the edges
- This effect worsens as context length increases

Mitigations:

- Put most relevant chunks **first** (or last) in the context
- Use re-ranking to identify the single most relevant chunk and position it prominently
- Use smaller k (3–5 chunks) for better attention
- Apply recency and relevance weighting in document ordering

Asai et al. (2024): Self-Reflective RAG

Self-RAG trains an LLM to decide *when to retrieve* (not always retrieving wastes time) and to *critique its own outputs* for relevance and factual support.

Standard RAG (Always Retrieves)

- Every query triggers retrieval
- Even for questions the model already knows: “What is a bond?”
- Wastes computational resources
- Retrieved chunks may be irrelevant and confuse the model

Self-RAG (Smart Retrieval)

The model generates special tokens:

- **[Retrieve]**: trigger retrieval when needed
- **[No Retrieve]**: answer from own knowledge when retrieval would not help
- **[Relevant]**: mark retrieved chunk as relevant
- **[Irrelevant]**: mark chunk as not useful
- **[Fully Supported]**: output is fully grounded in retrieved text
- **[Partially Supported]**: some claims are ungrounded

Finance Benefit

Self-RAG can distinguish between “What is a Sharpe ratio?” (no retrieval needed) and “What was the Sharpe ratio of the SPDR S&P 500 ETF in 2023?” (retrieval needed for specific facts).

The Full Self-RAG Cycle

1. **Input:** User asks a question
2. **Retrieve decision:** Model predicts whether retrieval is needed
3. **If retrieve:** Fetch k relevant chunks from knowledge base
4. **Relevance check:** For each chunk, predict whether it is relevant to the query
5. **Generate:** Produce a candidate answer using relevant chunks
6. **Support check:** Evaluate whether the generated answer is fully/partially/unsupported by the retrieved text
7. **Usefulness score:** Rate how useful the generated answer is to the user
8. **Select best:** If multiple generations were made (for different retrieved passages), select the one with highest support + usefulness score

Why This Matters for Finance

In finance, step 6 (support check) is critical: you can audit whether the AI's answer was actually grounded in the retrieved documents, or whether it invented something. This makes the system **auditable** — a key regulatory requirement.

Factuality: Is the Answer Grounded in Evidence?

Definition: Factuality

Factuality (in the context of RAG) measures whether the model's generated answer is consistent with and supported by the retrieved documents, rather than hallucinated or drawn from unverified prior knowledge. In effect, a **claim verifier**: each claim is checked against retrieved evidence — hallucination control as *evidence alignment*, not prompt engineering.

Factual Answer

Retrieved: "Apple Q3 FY2024 revenue: \$85.8B"

Generated: "Apple's Q3 FY2024 revenue was \$85.8 billion."

Factuality: HIGH (directly grounded)

Non-Factual Answer

Retrieved: "Apple Q3 FY2024 revenue: \$85.8B"

Generated: "Apple's Q3 FY2024 revenue was \$89.5 billion, driven by strong China performance."

Factuality: LOW (hallucinated numbers)

Measuring Factuality

- **AlignScore**: trains a model to evaluate if a claim is supported by context
- **SummaC**: NLI-based factuality scoring
- **FactScore**: fine-grained fact checking, one claim at a time
- **RAGAS**: RAG-specific evaluation metrics for faithfulness and relevance

RAGAS Faithfulness

RAGAS measures what fraction of claims in the generated answer can be directly traced to statements in the retrieved context. Target: >0.9 for financial applications.

Calibration in the RAG Context

A well-calibrated RAG system should say “I don’t know” or “the retrieved documents don’t contain this information” when it cannot find a reliable answer — rather than hallucinating a confident-sounding guess.

Ideal Calibrated Behavior

Question: “What was Company X’s bond yield spread on March 15, 2024?”

RAG retrieves: general info about Company X bonds but not the specific date.

Calibrated response: “I found information about Company X’s bonds but do not have data for the specific date of March 15, 2024. The most recent data in my knowledge base is from March 10, 2024, when the spread was 245bp. For the specific date requested, please consult Bloomberg.”

Uncalibrated Dangerous Behavior

Same question.

Uncalibrated response: “On March 15, 2024, Company X’s bond spread was 237 basis points, reflecting improved credit conditions following the Fed’s dovish signals.”

This number is completely fabricated but sounds authoritative.

How to Improve Calibration

Train models on examples of “I don’t know” responses; use retrieval scores as confidence signals; implement citation requirements so unsupported claims are flagged.

Wang et al. (2023): Self-Consistency

Self-consistency samples N independent chain-of-thought answers to the same question (using temperature sampling), extracts the final answer from each, and takes the **majority vote**.

Finance Example: Extracting a Number from a Filing

Question: "What effective tax rate is implied by this 10-K?"

Five sampled answers: 21.3% 21.3% 24.0% 21.3% 21.3%

Majority vote: **21.3%** — and the 4/5 agreement rate doubles as a **confidence score**.

Why it works:

- Independent chains make partially uncorrelated errors
- Majority voting cancels random reasoning mistakes
- Embarrassingly parallel: all N chains can run at once

Costs and gains:

- On math-style problems, adds roughly a further $1.3\times$ accuracy on top of CoT for small models
- Price: $N\times$ the inference cost
- Low agreement $\rightarrow$ flag for human review (next slide: the same sampling idea detects hallucinations)

Manakul et al. (2023): SelfCheckGPT

SelfCheckGPT detects hallucinations by sampling *multiple responses* to the same prompt and checking for consistency. Facts the model actually knows will be stated consistently; hallucinated facts will vary randomly across samples.

The Logic

Ask the model: "What was Berkshire Hathaway's net income in 2023?"

Sample 5 responses:

- Response 1: "\$96.2 billion"
- Response 2: "\$97.8 billion"
- Response 3: "approximately \$96 billion"
- Response 4: "\$89.4 billion"
- Response 5: "\$95 billion"

High variance across samples → model is **guessing**; the answer is likely hallucinated. Low variance → model is confident and likely correct.

Finance Application

Run SelfCheckGPT on any LLM-generated numerical claim. High inconsistency across samples → flag for human verification before using in a trading decision.

Kuhn et al. (2023): Semantic Uncertainty

Semantic uncertainty improves on SelfCheckGPT by recognizing that *different phrasings of the same fact* should be considered consistent. It groups semantically equivalent answers before measuring uncertainty.

The Problem with Naive Sampling Variance

If the model gives these 3 answers:

- “Apple’s Q3 revenue was \$85.8B”
- “Apple reported \$85.8 billion in Q3 sales”
- “Third-quarter revenue for Apple came in at approximately 85.8 billion dollars”

Naive string comparison: high variance (3 different strings). But semantically: *all three say the same thing!* The model is actually confident and consistent.

How semantic equivalence is detected: Kuhn et al. use an **NLI (Natural Language Inference)** model — a classifier trained to decide whether sentence A *entails* sentence B. If “Apple’s Q3 revenue was \$85.8B” *entails* “Apple reported \$85.8 billion in Q3 sales” (and vice versa), they are semantically equivalent and placed in the same cluster. Uncertainty is computed over clusters, not over raw strings.

Implication for Finance

More reliable hallucination detection = fewer false alarms (where the model is actually correct but just paraphrasing). Reduces unnecessary human review while maintaining safety.

Definition: Conformal Prediction

Conformal prediction is a framework for producing prediction sets (rather than single point predictions) with a **mathematically guaranteed coverage probability** — given one critical assumption: that past and future data are exchangeable (roughly: drawn from the same distribution).

The Simple Idea

Traditional model: “This earnings will be \$2.34 per share.” (Point estimate, no guarantee)

Conformal prediction: “With 90% confidence, the earnings will be between \$2.10 and \$2.55.” (Set-valued prediction with formal coverage guarantee)

The guarantee: run this on 100 new examples from the *same distribution* — at least 90 will contain the true answer.

Why it matters in finance:

- Formal uncertainty guarantees replace ad hoc thresholds
- Required for some regulatory model risk frameworks

Critical Caveat for Finance

Financial time series are **not** i.i.d. — regimes shift, volatility clusters, correlations change. The formal guarantee only holds under exchangeability. In practice: use conformal prediction on short, stable windows and re-calibrate frequently. Treat intervals as *approximate*, not exact.

What We Show Live (Demo 2 — in-session, ~10 min)

An end-to-end RAG pipeline running on a real earnings call transcript: retrieve relevant chunks, generate a grounded answer, score its factuality, and flag uncertainty.

1. **Indexing:** Chunked transcript embedded with `all-mpnet-base-v2`, stored in FAISS
2. **Hybrid retrieval:** BM25 + dense retrieval combined with reciprocal rank fusion
3. **Generation:** Llama-3 (or OpenAI API) generates answers grounded in retrieved chunks
4. **Factuality:** RAGAS faithfulness + relevance scores shown live
5. **Uncertainty:** SelfCheckGPT flags low-confidence claims
6. **Financial signal:** Bullish/bearish assessment correlated with 5-day return

Take-Home: Colab Notebook 2

The same pipeline is available as a full, runnable notebook with cached outputs. You will see directly where RAG improves over baseline LLMs (factual questions) and where it still struggles (complex cross-document reasoning).

What We Covered in Section F

- **Hallucination problem:** LLMs make up facts; catastrophic in finance
- **RAG architecture:** index documents, retrieve relevant chunks, generate grounded answers
- **Dense vs. sparse retrieval:** semantic search vs. keyword matching (use both)
- **ColBERT:** token-level late interaction for better precision
- **Chunking:** size and strategy critically affect quality
- **Tables and charts:** special handling required in finance
- **Lost in the middle:** order retrieved chunks carefully
- **Self-RAG:** adaptive retrieval with self-critique
- **Factuality metrics:** RAGAS, AlignScore
- **Uncertainty:** self-consistency, SelfCheckGPT, semantic uncertainty, conformal prediction

Coming Up: Section G

RAG solves the factuality problem for single questions. But financial analysis requires *sequences of decisions*: plan what to research, execute a research workflow, monitor results, and act. This is the domain of **agentic AI**.

Section G

Agentic AI

“AI That Plans and Acts”

Slides 143–164

What Is an Agent?

Definition: AI Agent

An **AI agent** is a system that perceives its environment, makes decisions, takes actions, and repeats this cycle to achieve a goal — autonomously, without step-by-step human instruction for each action.

The Robot in a Maze Analogy

Imagine a small robot placed in a maze. Its goal: find the exit. It doesn't know the maze layout in advance. It perceives its surroundings (walls, open paths), decides where to move, takes a step, perceives again, and repeats until it finds the exit.

An AI agent is like this robot, but its “maze” is the information environment: news, market data, documents. Its “steps” are actions like searching for information, computing analytics, or submitting a trade order.

Agents differ from chatbots:

- Chatbot: single question → single answer
- Agent: complex goal → multi-step plan → sequence of actions → goal achieved

Agents differ from pipelines:

- Pipeline: fixed sequence of steps
- Agent: dynamically decides what to do next based on intermediate results

The Fundamental Agent Cycle

1. **Perceive:** Receive observations from the environment (market data update, new news article, user query, previous action result)
2. **Think:** Use an LLM to reason about what the observations mean and what to do next
3. **Act:** Execute an action from the available tool set (call a data API, run a computation, send a trade order, query a database)
4. **Update Memory:** Store the result of the action
5. **Repeat:** Return to step 1 until the goal is achieved or a stopping condition is met

Finance Agent Example

Goal: Generate a sector rotation recommendation

Step 1: Search news for macro environment signals

Step 2: Retrieve recent sector performance data

Step 3: Run correlation analysis tool

Step 4: Read relevant Fed communications via RAG

Step 5: Synthesize: recommend overweight Healthcare, underweight Tech

Step 6: Draft rationale memo

Finance Tasks Are Multi-Step by Nature

Almost no real financial analysis task is answerable with a single LLM call. The tasks require planning, tool use, and iteration.

Example: Investment Due Diligence

A thorough due diligence requires:

1. Read 10 years of annual reports
2. Extract key financial metrics across years
3. Compare to industry peers
4. Read news for recent developments
5. Analyze management quality from call transcripts
6. Build a valuation model
7. Stress-test assumptions
8. Write a final memo

A single LLM call cannot do this. An agent can.

Example: Real-Time Risk Monitoring

A risk monitoring agent:

1. Continuously reads news and market data
2. Computes portfolio exposures
3. Detects anomalous signals
4. Evaluates if thresholds are breached
5. Drafts alerts for humans
6. Optionally triggers pre-approved hedges

Requires: continuous loop, tool use, memory of past states, conditional action.

Definition: Planning in AI Agents

Planning is the process by which an agent decomposes a high-level goal into a sequence of concrete, achievable sub-tasks, ordered by dependencies.

Planning in LLMs: Task Decomposition

High-level task: "Analyze whether Nvidia is a buy right now."

LLM-generated plan:

1. Retrieve Nvidia's most recent 10-Q (financial health)
2. Search news for Nvidia in the past 30 days (current sentiment)
3. Get analyst consensus and recent rating changes
4. Pull semiconductor sector performance vs. S&P 500
5. Calculate Nvidia's P/E vs. 5-year average and sector average
6. Retrieve earnings surprise history
7. Synthesize all findings into a buy/hold/sell recommendation with confidence level

Planning Failure Mode

LLMs sometimes generate unrealizable plans (steps that cannot be executed with available tools) or plans with incorrect ordering (step 5 depends on step 6's output). Always validate plans before execution.

Definition: Tools (Function Calling)

Tools are external capabilities that an agent can invoke to interact with the world beyond text generation. The LLM decides *when* to use a tool and *what parameters* to pass; the tool executes and returns results.

Finance Agent Tool Set

- `get_stock_price(ticker, date)`
- `get_financials(company, period)`
- `search_news(query, date_range)`
- `retrieve_filing(company, form_type, year)`
- `run_python_code(code)` for calculations
- `get_analyst_ratings(ticker)`
- `compute_portfolio_risk(positions)`
- `submit_trade_order(ticker, size, direction)` [high-risk tool, needs approval]

Tool Use Best Practices

- **Least privilege:** only give agents the tools they need for their task
- **High-risk tools:** irreversible actions (trades, emails) require human confirmation
- **Tool output validation:** check tool responses for errors before passing to LLM
- **Rate limits:** financial APIs have rate limits; agent must respect them
- **Caching:** cache expensive tool results (API calls) to reduce cost

The Memory Problem

An agent's context window is finite. An agent running for hours or days accumulates more information than fits in a single context. Memory management is critical.

Short-Term Memory (Working Memory)

- The current context window: conversation history, recent tool outputs, current sub-task
- Fast access, fully visible to the LLM
- Limited: 128K–200K tokens
- Like a person's working memory: what you're actively thinking about right now

Long-Term Memory

- External storage: vector database, traditional database, files
- Compressed summaries of past actions and findings
- Retrieved as needed (via RAG) into short-term memory
- Like notes in a notebook: not in your head, but accessible

Finance Agent Memory Architecture

- **Episodic memory:** “Yesterday I analyzed Microsoft; here were my key findings” (stored as summary)
- **Semantic memory:** financial facts extracted from past research (in vector DB)
- **Procedural memory:** which tool to call for which task (embedded in system prompt)

Madaan et al. (2023): Self-Refine

Self-Refine is an agent framework where the same LLM *drafts* an output, then *critiques* its own draft, then *revises* based on the critique — iterating until the output meets quality criteria.

Self-Refine for an Investment Memo

Round 1 Draft: “Based on recent earnings, Company X is a buy.”

Round 1 Critique: “This memo is too vague. It doesn't cite specific numbers, doesn't compare to peers, and doesn't discuss risks.”

Round 2 Draft: “Company X reported \$4.2B revenue (+12% YoY), beating consensus by 8%. P/E of 18 is below peer average of 22. Key risks include regulatory exposure...”

Round 2 Critique: “Better. But the regulatory risk section is incomplete.”

Round 3 Draft: More complete, with specific regulatory analysis...

When to Stop

Typical stopping conditions: N rounds reached, critique quality score above threshold, or critic says “no major improvements needed.”

Shinn et al. (2023): Reflexion

Reflexion enables an agent to **learn from past mistakes within a session** by storing verbal reflections (natural language lessons learned) in long-term memory, without needing to update model weights.

Reflexion in Finance

Episode 1: Agent analyzes a company, generates a bullish recommendation. A week later, the company misses earnings badly.

Reflection: "I failed to check the inventory build-up data in the 10-Q, which was a warning sign for demand deceleration. Next time I analyze a manufacturing company, I must check inventory-to-sales ratio trends."

Episode 2: Agent encounters a similar situation. It retrieves the stored reflection and proactively checks inventory data. It catches the problem early.

Limitations of Reflexion

Verbal reflections only persist within a session or if explicitly stored. They don't generalize across different agents or after model restarts. And reflections can be wrong ("I failed because of X" might be a post-hoc rationalization of a random event).

Gou et al. (2024): CRITIC

CRITIC (Critical Thinking with Tools) enables LLMs to verify their own outputs using *external tools* — rather than relying on self-critique alone (which doesn't actually catch factual errors).

CRITIC Workflow for a Financial Claim

LLM generates: "Apple's revenue grew 15% in Q3 2024."

CRITIC step 1: Identify verifiable claim: "15% growth in Q3 2024"

CRITIC step 2: Call tool: `get_financials('Apple', 'Q3_2024')`

Tool returns: "Q3 2024 revenue: \$85.8B. Q3 2023 revenue: \$81.8B. Growth: 4.9%"

CRITIC step 3: Detect discrepancy: $15\% \neq 4.9\%$. Flag and correct.

Corrected output: "Apple's revenue grew 4.9% in Q3 2024."

Why CRITIC Matters More Than Self-Critique

Self-critique: asks the model "is this number right?" — the model has the same false belief it started with, so it says "yes, looks right." CRITIC: calls an external API that returns the actual number. The external world is the ground truth.

Yao et al. (2023): ReAct

ReAct (Reasoning + Acting) is an agent framework where the LLM alternates between **Thought** steps (natural language reasoning) and **Action** steps (tool calls), making the reasoning process transparent and debuggable.

ReAct Trace for a Finance Task

Task: "What is the current debt-to-equity ratio of Tesla?"

Thought 1: I need Tesla's total debt and shareholder equity from the most recent balance sheet.

Action 1: `get_balance_sheet('TSLA', 'most_recent')`

Observation 1: Total debt: \$9.6B, Total equity: \$62.6B

Thought 2: $\text{Debt-to-equity} = \text{Total debt} / \text{Total equity} = 9.6 / 62.6$.

Action 2: `calculate('9.6 / 62.6')`

Observation 2: 0.153

Thought 3: Tesla's D/E ratio is 0.15, which is low and indicates a conservatively leveraged balance sheet.

Final Answer: Tesla's current debt-to-equity ratio is 0.15.

Auditability

ReAct's interleaved thoughts make the reasoning *auditable*: a compliance officer can review exactly what reasoning led to a conclusion and what data was used.

Agent Type 1: Predictive Agent

A **predictive agent** uses LLMs and quantitative models to generate *forward-looking forecasts* — earnings predictions, price targets, volatility forecasts — as the basis for trading decisions.

Architecture

1. Collect all relevant signals: earnings call tone, analyst revisions, momentum, macro indicators
2. LLM synthesizes signals into a directional forecast
3. Forecast is probabilistic: “65% probability of positive surprise”
4. Risk system determines position size from probability
5. Trade is executed according to execution algorithm
6. Post-trade: record forecast and outcome for learning

Finance Example: Earnings Prediction Agent

- Input: 10-Q transcript, sector data, macro environment
- LLM + FinBERT: assess qualitative signals
- Quant model: estimate earnings surprise probability
- Output: “65% chance of positive earnings surprise; 30% chance of in-line; 5% chance of miss”
- Action: small long position sized proportionally to edge

Agent Type 2: Reactive Agent

A **reactive agent** continuously monitors information streams and responds to *events* as they occur, making real-time decisions based on incoming information rather than pre-computed forecasts.

Architecture

1. Monitor: news feeds, earnings calls (live), regulatory filings, SEC 8-K alerts
2. Event detection: new material event identified
3. Rapid assessment: LLM classifies event type and directional implication
4. Risk assessment: is this large enough to trade?
5. Action: if threshold met, execute trade; if uncertain, alert human

Event: Surprise CEO Departure

- **Event:** Company X files 8-K at 11:23 AM: CEO resigned effective immediately
- **LLM assessment:** Unexpected departure, no named successor — historically negative signal
- **Sentiment:** Strongly negative
- **Action:** Flag for human review (trade is not automatic — CEO departure is complex)
- **Human decision:** Short Company X stock with defined stop-loss

Agent Type 3: Hypothesis-Driven Research Agent

A **hypothesis-driven agent** autonomously generates investment hypotheses, designs tests, gathers evidence, and reaches conclusions — automating the research process that human analysts perform.

An AI Analyst at Work

Starting hypothesis: “Rising shipping costs in Q3 will disproportionately hurt small e-commerce companies vs. large platforms (Amazon), because large platforms have own logistics.”

Agent research steps:

1. Retrieve latest shipping cost indices (Baltic Exchange, Freightos)
2. Screen S&P 500 + Russell 2000 for e-commerce companies
3. For each company: retrieve logistics cost disclosures from recent 10-Qs
4. Classify: own logistics vs. third-party dependent
5. Backtest: in previous shipping spike periods, did this pattern hold?
6. Conclusion: confirms hypothesis; design long Amazon / short small e-commerce trade

From Human Analyst to AI Agent

Traditional research workflow: 1 analyst = 2–3 companies analyzed deeply per week.

AI research agent: 10–100 companies analyzed daily (less deeply, but consistently).

Step	Traditional Analyst	AI Research Agent
Screening	Manual, ~1 hour	Automated, <1 minute
Filing reading	2–4 hours per filing	~5–10 minutes per filing
Comp analysis	1–2 hours	~2–5 minutes
Memo drafting	1–2 hours	~1 minute (draft)
Earnings call	Real-time listening	Real-time NLP + summary
Cross-sector	Rarely (time constraint)	Continuous

What AI Agents Cannot Replace

- Deep understanding of business strategy nuance
- Personal relationships with management (IR calls, channel checks)
- Novel insight generation (not pattern matching)
- Judgment in truly unprecedented situations

Always-On Intelligence

A monitoring agent replaces the need for analysts to constantly watch news feeds — it never sleeps, never gets distracted, and processes text much faster than any human.

Portfolio Monitoring Agent: Step-by-Step

1. **Continuous ingestion:** Subscribe to Reuters/Bloomberg news feeds, SEC EDGAR alerts, Twitter/X monitored accounts
2. **Company mapping:** Each news item is mapped to the relevant portfolio company(ies)
3. **Materiality assessment:** Is this event material to the portfolio company? (LLM classifies)
4. **Sentiment and impact:** What is the directional impact? How large is the expected move?
5. **Risk threshold check:** Does this breach any pre-defined risk triggers?
6. **Alert generation:** If material, generate a concise alert: company, event, expected impact, recommended action
7. **Human notification:** Send alert to portfolio manager with full context and supporting quotes

Speed Advantage

A monitoring agent can detect a material 8-K filing within seconds of it hitting EDGAR. A human may not see it for hours.

AI as Co-Pilot, Not Pilot

The safest deployment of agentic AI in finance is as a **decision support system**: the agent gathers information, generates options, and makes recommendations — but a human makes the final decision on significant actions.

Decision Support Workflow

1. PM (Portfolio Manager) says: “I’m considering adding to our biotech position”
2. Agent automatically: retrieves recent clinical trial results, FDA calendar, peer movements, current position size, sector allocation
3. Agent generates: three scenarios (add, hold, reduce) with rationale for each
4. Agent flags: any unresolved uncertainties or risks it couldn’t assess
5. PM reviews and decides
6. Agent executes approved decision

Why Decision Support Is the Right Model

- Avoids autonomous AI trading large positions
- Keeps human judgment in the loop for novel situations
- Creates an audit trail (AI documented its reasoning before the human decided)
- Regulatory compliance: most jurisdictions require human oversight for significant trading decisions
- Builds trust: PM learns to trust the AI gradually, can verify its reasoning

Wu et al. (2023): AutoGen

AutoGen is a framework for building multi-agent systems where multiple LLM-based agents can *converse* with each other, with humans, and with tools to solve complex tasks collaboratively.

AutoGen Finance Workflow

Agent 1 (Data Analyst): "I've retrieved Apple's last 4 quarters of financials. Revenue trend is positive; margin is expanding."

Agent 2 (Risk Manager): "Wait — I see that accounts receivable grew 40% while revenue grew 12%. That's a quality of earnings concern."

Agent 3 (Macro Analyst): "Agree with caution. Rising rates are headwinds for Tech valuations even with strong fundamentals."

Human PM: "Good points. Keep the position but reduce size 20%."

AutoGen Agent Types

- **AssistantAgent:** LLM-powered agent that generates text and calls tools
- **UserProxyAgent:** represents human input; can approve/reject plans
- **GroupChat:** orchestrates multi-agent conversations with speaker rotation
- Custom agents: specialized for specific domains (data agent, risk agent, etc.)

Li et al. (2023): CAMEL

CAMEL (Communicative Agents for “Mind” Exploration of Large Language Models) is a framework where two agents play pre-assigned roles (e.g., investor and analyst) and engage in goal-directed conversations.

CAMEL Finance Scenario: Bull vs. Bear

Role 1 (Bull Analyst): “I recommend buying Company X based on its dominant market position and 20% revenue CAGR.”

Role 2 (Bear Analyst): “I disagree. The 20% CAGR is decelerating — it was 35% two years ago. And the market position is being challenged by three new entrants with VC funding.”

Role 1: “The new entrants lack distribution networks. Company X’s moat is its B2B relationships, which take years to build.”

Role 2: “Fair point on distribution. But B2B contracts renew annually — I’ve seen three large customers recently file for RFPs (requests for proposals). That’s a warning sign.”

Value

Structured role-playing surfaces arguments and counter-arguments that a single agent might miss. The “devil’s advocate” agent is a powerful tool for investment due diligence.

Hong et al. (2024): MetaGPT

MetaGPT assigns different *professional roles* (Product Manager, Architect, Engineer, QA) to different LLM agents, making them collaborate like a software development team. The same paradigm applies to finance.

MetaGPT Applied to Finance Research

Roles in a Finance Research Team:

- **Research Lead Agent:** defines the investment question and research plan
- **Data Engineer Agent:** retrieves and structures raw data from APIs
- **Quantitative Analyst Agent:** runs statistical models and backtests
- **Fundamental Analyst Agent:** reads filings and calls; extracts qualitative signals
- **Risk Manager Agent:** stress-tests conclusions, identifies holes in the thesis
- **Report Writer Agent:** synthesizes into a final investment memo

Each agent specializes; their outputs feed into each other. The system produces better output than any single agent.

Yang et al. (2024): FinRobot

FinRobot is an open-source platform designed specifically for financial AI agents, providing ready-made tools, data connectors, and agent templates for common finance tasks.

FinRobot Components

- **Financial AI Agents Layer:** pre-built agents for market forecasting, document analysis, portfolio management
- **Financial LLM Algorithms Layer:** model selection, fine-tuning pipelines for finance tasks
- **LLMOps + DataOps:** infrastructure for deploying and monitoring finance LLMs in production
- **Multi-Source Data Pipeline:** connects to Alpaca, Yahoo Finance, Refinitiv, EDGAR

FinRobot Agents Available

- Market Forecasting Agent
 - Document Analysis Agent (10-K, earnings call)
 - Trading Signal Agent
 - Risk Assessment Agent
 - Portfolio Management Agent (based on FinRL)
- All open-source; available on GitHub for research use. Strong starting point for building custom finance agent systems.

Research vs. Production

FinRobot is suitable for research prototyping. Production deployment requires additional infrastructure for reliability, security, and regulatory compliance.

What We Covered in Section G

- **Agent definition:** perceive, think, act, repeat — autonomous goal pursuit
- **Why finance:** tasks are inherently multi-step and require diverse information sources
- **Planning:** decompose complex goals into executable sub-tasks
- **Tools:** connect agents to real-world data and actions
- **Memory:** short-term (context) + long-term (external storage)
- **Self-Refine:** iterative improvement through self-critique
- **Reflexion:** verbal lessons learned stored for future episodes
- **CRITIC:** verify claims with external tools, not just self-assessment
- **ReAct:** interleaved reasoning and action for auditability
- **Frameworks:** AutoGen, CAMEL, MetaGPT, FinRobot

Coming Up: Section H

Single agents are powerful but limited. The next step: teams of specialized agents that collaborate, debate, and disagree — because diversity of perspective is a fundamental advantage in markets.

Section H

Multi-Agent Systems

“Many Minds Working Together”

Slides 165–188

The Single-Agent Bottleneck

A single agent faces fundamental limits: it can only focus on one thing at a time, it has one perspective, it can only hold so much in its context, and it cannot specialize deeply in many domains simultaneously.

Single-Agent Problems

- **Breadth vs. depth:** can't be expert in macro + micro + quant + NLP simultaneously
- **Confirmation bias:** a single agent tends to find evidence supporting its initial view
- **Single point of failure:** if the one agent has a bad day (poor prompt, hallucination), everything fails
- **No diversity:** all analyses come from the same model with the same biases

Multi-Agent Solutions

- **Specialization:** each agent is an expert in its domain
- **Debate:** agents challenge each other's views
- **Redundancy:** multiple agents cross-check each other's conclusions
- **Diversity:** different models, different training, different perspectives reduce collective bias

Analogy: The Best Investment Teams Are Multi-Disciplinary

Great hedge funds have macro analysts, sector analysts, quant researchers, risk managers, and traders all working together and challenging each other — because no one person can do it all.

The Sports Team Analogy

A basketball team doesn't put the same player in every position. You have a point guard (fast, handles the ball), a center (tall, dominates the paint), a shooter (precise three-pointers). Each player is optimized for their role. The team wins because of complementary specialization.

A multi-agent financial system works the same way: a macro agent (trained on macro data), a sector specialist agent (trained on sector filings), a quant agent (trained to run calculations), and a risk agent (trained to find problems) combine their expertise.

How to Specialize Agents

- Different system prompts (roles and expertise)
- Different training/fine-tuning datasets
- Different tools available (macro agent gets macro data tools; sector agent gets company filing tools)
- Different memory stores (curated for their domain)

Emergent Capabilities

When specialized agents interact, the *combined* system can solve problems that no individual agent could:

- Macro + sector + quant + risk agent ensemble: complete investment thesis with risk assessment
- Each agent catches blind spots of the others

Definition: Cooperative Multi-Agent System

In a **cooperative** multi-agent system, all agents share a common goal and collaborate — sharing information, dividing labor, and combining outputs — to achieve it together.

Cooperative Finance Research Example

Common goal: Evaluate a potential investment in Company X

Agent 1 (Fundamental): Reads 10-K/10-Q; extracts financial metrics; concludes balance sheet is strong

Agent 2 (Technical): Analyzes price momentum and volume; identifies breakout setup

Agent 3 (Macro): Assesses sector tailwinds; rates the macro environment as favorable

Agent 4 (Risk): Identifies key risks: litigation, regulatory, concentration

Coordinator agent: Aggregates all agents' reports; weights by confidence; produces final recommendation: "Buy with medium conviction; size at 2% of portfolio; stop loss at -8%"

Key Design Requirement

Cooperative agents need a **conflict resolution mechanism** for when their sub-conclusions disagree. We'll cover this in slide 177.

Definition: Competitive Multi-Agent System

In a **competitive** setup, agents are deliberately designed to *argue opposing positions*. The conflict between agents improves the final analysis by surfacing counterarguments that a single cooperative agent might suppress.

Adversarial Architecture

- **Bull Agent:** tasked with finding the strongest case for buying
- **Bear Agent:** tasked with finding the strongest case against
- **Neutral Judge:** evaluates the debate and reaches a balanced conclusion

Neither the Bull nor Bear Agent is seeking the truth — they are advocates. The truth emerges from their competition.

Why Adversarial Works

Psychological research shows that when you *role-play* as an advocate, you find arguments you wouldn't find when seeking balance. The Bull Agent will dig deep for positive signals; the Bear Agent will find every risk. Combined, they cover far more ground than a neutral single agent.

Design Caution

Both agents must be given the same information. If the Bull Agent gets favorable data and the Bear gets unfavorable, the debate is rigged. Symmetry of information is essential for valid adversarial debate.

Du et al. (2023): Society of Mind

Du et al. showed empirically that having multiple LLM instances *debate* each other — proposing answers, critiquing others' proposals, and revising — significantly improves factual accuracy and reasoning quality compared to a single LLM.

Experiment Setup

- Task: mathematical reasoning (GSM8K), factual Q&A, and general reasoning
- Setup: 3 agents each produce an initial answer, read each other's answers, revise, repeat for 3 rounds
- Result: debate improved accuracy by 11% on average vs. single agent
- Mechanism: agents notice each other's errors and flag them explicitly

Why debate works:

- Errors are caught by other agents (errors don't correlate perfectly between agents)
- Competition incentivizes thoroughness
- Multiple perspectives surface different aspects

Finance relevance:

- Debate on earnings interpretation
- Stress-testing valuation assumptions
- Bull/bear analysis generates more complete picture

The Structured Debate Protocol

1. **Round 1 — Initial Positions:** Each agent independently analyzes the question and states its initial position and reasoning
2. **Round 2 — Critique:** Each agent reads the other agents' positions and explicitly identifies flaws, missing evidence, or logical errors
3. **Round 3 — Revision:** Each agent revises its position in light of the critiques, either defending its original view (with counter-arguments) or updating to a new position
4. **Consensus Check:** A judge agent evaluates whether convergence has been reached
5. **If not converged:** additional rounds of debate (up to N maximum rounds)
6. **Final synthesis:** Judge agent produces a final answer that integrates the strongest arguments from all sides

When to Stop Debating

Set a maximum of 3–5 rounds. More rounds show diminishing returns and increasing computational cost. If agents haven't converged after 5 rounds, the disagreement itself is a signal: the question is genuinely uncertain, and the human should be informed of the competing views.

MADDPG: Multi-Agent Deep Deterministic Policy Gradient

Lowe et al. (2017) extended single-agent actor-critic reinforcement learning to the multi-agent setting. Each agent has its own **actor** (policy: what action to take) and **critic** (evaluation: how good was that action).

Key Innovation: Centralized Training, Decentralized Execution

- **Training:** critics see *all agents'* actions and states — global view
- **Execution:** each actor only sees its own observations — local view
- Allows agents to learn coordinated strategies during training that they can execute independently in deployment

Finance Application

- Multiple portfolio management agents for different asset classes
- During training: critics can see overall portfolio risk
- During deployment: each agent makes decisions independently
- Result: agents implicitly coordinate to avoid correlated risk without explicitly communicating

Limitation

MADDPG was designed for cooperative agents in simulated environments. Financial markets are non-stationary, making direct RL application challenging. Used more in simulation and execution optimization than live trading.

Rashid et al. (2018): QMIX

QMIX is a multi-agent RL algorithm designed for **cooperative** tasks where all agents share a single team reward. It learns a mixing network that combines individual agents' value functions into a joint team value.

The Key Insight

QMIX enforces that the joint team value is a *monotonic function* of individual agents' value functions — meaning: if an individual agent's action improves its own value, it also improves the team value. This makes training tractable while allowing coordination.

Finance Analogy

Portfolio management team: individual managers optimize their sub-portfolio (their individual value), and the mixing network ensures these individual decisions combine to optimize total portfolio Sharpe ratio (team value).

Similar to how a fund's individual sector analysts optimize their sleeves while risk management coordinates overall portfolio risk.

Research Applications

QMIX-style approaches have been explored for multi-asset execution optimization: multiple execution agents (one per asset) coordinating to minimize total market impact while maintaining liquidity for the overall portfolio.

Two Different Uses of Disagreement — Don't Confuse Them

In factual debate (slides 170–171): disagreement is resolved through evidence and reasoning — agents converge on a *correct fact*. Goal: eliminate wrong answers.

In investment decisions: disagreement on what to *do* is not necessarily wrong — it reflects genuine uncertainty about the future. Here, disagreement is a *signal* about market conditions, not a bug to fix.

A Concrete Example

Three agents analyze Company X:

- **Fundamental agent**: Strongly positive (excellent balance sheet, earnings growth)
- **Technical agent**: Strongly negative (breaking down from a major support level)
- **Sentiment agent**: Mixed (news is neutral-to-positive, but social media very negative)

Disagreement interpretation: Fundamental and technical are pointing in opposite directions — this is a situation where the stock is at a crossroads. The disagreement predicts *high upcoming volatility*. Appropriate action: **reduce position size, buy volatility protection** (options), and wait for resolution before adding a new directional bet.

The Information Content of Disagreement

- **High agreement across agents:** information is clear, consensus view, lower uncertainty, lower expected volatility
- **High disagreement:** information is ambiguous or contradictory; different analytical lenses give different conclusions; higher uncertainty; *higher expected volatility*

The Analyst Dispersion Effect

Academic evidence: higher dispersion among human analyst earnings estimates predicts higher future stock volatility (Diether, Malloy, Scherbina, 2002). Similarly, higher AI agent opinion dispersion should predict higher volatility.

Using Disagreement in Trading

- **Disagreement = volatility signal:** when agents disagree, the market is likely to move sharply when the uncertainty resolves
- **Options strategy:** buy straddles (profit from large moves in either direction) when agent disagreement is high
- **Position sizing:** scale down position sizes when agent disagreement is high (uncertain situations warrant smaller bets)

Definition: Market Regime

A **market regime** is a persistent state of market behavior characterized by specific statistical properties: e.g., “bull market” (low volatility, positive drift), “bear market” (high volatility, negative drift), “risk-off” (correlations spike, all risky assets fall together).

How Agent Conflict Signals Regime Change

In stable regimes, agents trained on different signals tend to agree (all signals point the same way). At regime transitions:

- Momentum agents signal continuation (past trend still positive)
- Macro agents signal reversal (leading indicators turning)
- Sentiment agents mixed (some investors still bullish, others turning bearish)

This multi-agent disagreement pattern is an early warning of regime change.

Why Early Detection Matters

Regime changes are the most dangerous periods for quantitative strategies. Models trained on one regime often fail in another. An early warning system built on agent conflict can trigger defensive position-taking before the regime fully shifts.

When Agents Disagree, What Happens Next?

Multi-agent systems need explicit rules for resolving conflicting recommendations. Without these rules, agents can deadlock (no action taken) or produce erratic behavior (random choice between conflicting outputs).

Protocol 1: Weighted Voting

Each agent's vote is weighted by:

- Historical accuracy on this type of decision
- Confidence score (calibrated probability)
- Relevance to current market conditions

Final decision: weighted average of recommendations.

Protocol 2: Conservative Default

When agents disagree beyond a threshold: take the **most conservative** action (smallest bet, hedge existing positions, hold cash). Uncertainty calls for caution.

Protocol 3: Escalation Hierarchy

1. Agents try to resolve via structured debate (3 rounds)
2. If unresolved: escalate to a senior supervisor agent
3. If still unresolved: escalate to a human portfolio manager
4. Human makes the final call

Protocol 4: Meta-Learning

Train a meta-agent on historical cases where agents disagreed. The meta-agent learns: "In regime X, the fundamental agent is usually right. In regime Y, the macro agent is usually right." Use regime-conditional weighting.

Escalation Must Be Designed In, Not Added As an Afterthought

The most common mistake in deploying agentic AI is treating human oversight as a failsafe for emergencies. Instead, human escalation should be a **routine, expected part of the workflow**.

Good Escalation Design

Pre-defined rules for when to escalate:

- Decision exceeds pre-defined size threshold (\$X)
- Agent confidence below threshold
- Novel event type with no historical precedent
- Multi-agent disagreement above threshold
- Risk metrics approach pre-set limits
- Model has been running in current regime for longer than expected

Bad Escalation Design

- Only escalate when AI system crashes or throws an error
- Only escalate in market emergencies (too late)
- No clear criteria for what triggers escalation
- Escalation path unclear (who gets the alert? Who decides?)
- No feedback loop: human decisions don't train the system

The Human in the Loop Is a Feature, Not a Bug

Regulatory frameworks (MiFID II, SEC, EU AI Act) will increasingly require documented human oversight. Design it well from the start; don't bolt it on.

Note: LLM-based Agent, Not RL

TradingGPT and FinMem (next slide) are **LLM-based agents** that use prompting and memory structures — they are *not* reinforcement learning systems. They complement the RL approaches (Nevmyvaka, Spooner, FinRL) but use a fundamentally different paradigm: language reasoning instead of reward optimization.

Li et al. (2023): TradingGPT

TradingGPT is a multi-agent trading framework that equips LLM agents with **layered memory** (short-term, medium-term, long-term) and diverse **trading characters** to simulate different investor personalities.

Trading Characters

Different agents are given different “personalities”:

- **Aggressive:** high risk tolerance, large positions, momentum-following
- **Conservative:** low risk tolerance, small positions, contrarian
- **Balanced:** moderate risk, diversified approach
- **Fundamentalist:** only trades on valuation signals
- **Technician:** only trades on chart signals

Layered Memory

- **Short-term:** today's news, current price action
- **Medium-term:** last month's data, trend context
- **Long-term:** historical patterns, fundamental baselines, risk parameters

Each layer updates at different frequencies. Long-term memory is slow to change; short-term memory is updated every trading session.

Why Character Diversity Helps

Different character agents respond differently to the same news. The ensemble captures the *real diversity of investor types* that drives market dynamics.

Yu et al. (2023): FinMem

FinMem is a trading agent framework with an enhanced memory architecture specifically designed to improve financial decision-making by organizing financial information into meaningful temporal layers.

FinMem Memory Architecture

- **Shallow memory:** recent market data, news, order flow
- **Medium memory:** quarterly performance, analyst consensus changes, institutional flow data
- **Deep memory:** fundamental value estimates, long-term industry trends, historical crisis patterns

Each level is summarized and compressed to fit in the agent's context when needed.

FinMem Results (Yu et al., 2023)

On a short paper trading test (real market data, **not a rigorous long-horizon backtest**):

- Annualized return: above buy-and-hold baseline in the test window
- Reported Sharpe ratio: 1.5–2.0 (claimed, over a *very short period*)
- Outperformed single-agent LLM baselines without structured memory

Critical Caveat — Read Before Citing This Number

The test period is **statistically too short** to distinguish skill from luck. A few months of live paper trading cannot establish a Sharpe ratio with any confidence. Apply the PBO and DSR tests (Section I) before treating this figure as evidence of a deployable edge. These results should be read as a proof-of-concept for the memory architecture, not a performance guarantee.

The Execution Problem: A Natural RL Setting

Executing a large order (e.g., sell 1 million shares of Apple over 1 day) is an optimization problem: minimize total market impact + market risk. Nevmyvaka et al. (2006) showed RL outperforms traditional methods.

The RL Formulation

- **State:** remaining shares to sell, time remaining, current market conditions (spread, book depth, volatility)
- **Action:** how many shares to sell right now
- **Reward:** negative total execution cost (we minimize cost = maximize negative cost)
- **Episode:** the full execution period (e.g., 1 trading day)

Compared to Almgren-Chriss

The Almgren-Chriss model (analytical) provides a formula for optimal execution given a linear price impact model.

RL learns the optimal execution from data — it can adapt to:

- Non-linear price impact
- Intraday volume patterns
- Adverse selection detection
- Dynamic market conditions

RL beats Almgren-Chriss in non-linear and dynamic environments.

RL for Market Making

Spooner et al. (2018) formulated market making as an RL problem: the agent decides where to place bid and ask quotes to maximize profit while managing inventory and adverse selection risk.

RL Market-Making State

- Current bid-ask spread
- Current inventory (how many shares held)
- Recent order flow imbalance (proxy for informed trading)
- Time of day (volume patterns)
- Recent volatility

RL Market-Making Actions

- Where to place bid quote (distance from mid)
- Where to place ask quote
- Quantity to quote at each level
- Whether to cancel and replace quotes

Results

RL market makers learned to:

- Widen spreads when order flow toxicity is high (like Glosten-Milgrom, but learned from data)
- Tilt quotes directionally when inventory builds up (lean against imbalance)
- Reduce quoting near the open and close (high adverse selection periods)

Outperformed rule-based market making strategies by 15–25% in simulations.

Liu et al. (2020): FinRL

FinRL is an open-source deep reinforcement learning library for financial portfolio management, providing standardized environments, agent implementations, and performance benchmarks.

FinRL Components

- **Environments:** stock market, cryptocurrency, foreign exchange, commodity trading simulators
- **Agents:** implemented Deep Reinforcement Learning (DRL) algorithms: PPO, A2C, SAC, TD3, DDPG
- **Data pipeline:** integrates Yahoo Finance, Alpaca, Quandl
- **Benchmarks:** standard comparison metrics (Sharpe ratio, maximum drawdown, Calmar ratio)

A Typical FinRL Experiment

1. Download S&P 500 data from Yahoo Finance (2010–2020)
2. Define state: price, volume, technical indicators
3. Train PPO agent to maximize Sharpe ratio
4. Evaluate on 2021–2023 out-of-sample period
5. Compare to buy-and-hold, equal-weight, and Markowitz portfolios

Common finding: RL agents beat simple benchmarks in-sample but often fall short of equal-weight out-of-sample due to overfitting.

The Core Puzzle: Why Would a Text Model Help With Numbers?

LLMs are trained on words, not stock prices. So why use them for time series? **The answer:** LLMs have learned powerful pattern recognition — trends, seasonality, anomalies — from *textual descriptions* of these phenomena (financial news, economic reports). Time-LLM *reprograms* the LLM to apply this reasoning to raw numbers.

Jin et al. (2024): How Reprogramming Works

1. **Patch:** chop the time series into overlapping windows (“patches”)
2. **Project:** map each patch into the *same vector space* as text tokens using a learned linear layer
3. **Align:** find the nearest text concept for each patch (“rising,” “volatile,” “flat”)
4. **Prompt:** prepend a text description of the task (“Forecast next 7 days of AAPL returns”)
5. **LLM:** frozen LLM generates forecast tokens; decode back to numbers

Why This Works for Finance

The LLM has read millions of sentences like “markets entered a bearish phase” and “volatility spiked.” After reprogramming, a patch corresponding to a sharp price drop gets aligned to the text concept “crash” — and the LLM can reason: “after a crash, typically comes either recovery or continued decline, depending on...”

Time-LLM achieves competitive performance on financial forecasting benchmarks compared to purpose-built time series models.

Ansari et al. (2024): Chronos

Chronos is a family of pre-trained time series forecasting models that treat numerical time series as a “language” — tokenizing time series values and training a T5-style transformer model to predict future tokens.

The Tokenization Approach

1. Normalize time series to a fixed range
2. Discretize values into bins (like a vocabulary of “price words”)
3. Treat the sequence of bins as a token sequence
4. Train T5 to predict the next H tokens
5. At inference: decode predicted tokens back to numerical values

Chronos Advantages

- **Zero-shot:** can forecast any time series without task-specific training
- **Probabilistic:** generates full predictive distributions (not just point estimates)
- **Handles diverse series:** trained on 100K+ diverse time series; generalizes broadly
- **Finance performance:** competitive on equity price and macro economic series forecasting

Limitation

Chronos doesn't understand text. It only models the numerical pattern. Combining Chronos with text-based LLMs for joint reasoning is an open research problem.

Rasul et al. (2024): Lag-Llama

Lag-Llama is a foundation model for probabilistic time series forecasting, based on the Llama architecture. It uses **lagged features** (values at different past time lags) as input tokens, enabling flexible forecasting across different frequencies.

Architecture Highlights

- Uses the Llama transformer backbone (decoder-only)
- Input: collection of lagged past values (e.g., lag 1, 2, 3, 7, 14, 21, 30 days)
- Output: parameters of a probability distribution (Student-t distribution)
- Pre-trained on a large corpus of diverse time series
- Fine-tuned with small amounts of target data

Finance Application

For stock return forecasting:

- Input: past 30 days of daily returns, with lags at 1, 5, 10, 20 days
- Output: probability distribution over next 5-day return
- Use: generate trading signals with calibrated uncertainty
- Combine with: text-based signals from FinBERT for multimodal forecast

The Foundation Model Trend

Lag-Llama, Chronos, and Time-LLM represent a trend toward *universal* time series models — analogous to GPT for text. Finance is a natural domain for these models given the vast amount of available time series data.

What You'll Build in Colab Notebook 2 (Multi-Agent)

A multi-agent system that detects regime changes and handles agent disagreement:

1. **Three agent system:** Bull agent (momentum-based), Bear agent (mean-reversion-based), Macro agent (macro-conditions-based)
2. **Market data:** Use real S&P 500 data across multiple historical regimes (2019 bull, 2020 crash, 2020–2021 recovery, 2022 bear)
3. **Agreement tracking:** Measure agent agreement over time; plot the disagreement time series
4. **Conflict detection:** Identify periods of high disagreement
5. **Regime validation:** Check whether high disagreement periods correspond to regime transitions
6. **Conflict resolution:** Implement three protocols (weighted voting, conservative default, escalation) and compare performance
7. **Evaluation:** Sharpe ratio and maximum drawdown across protocols; compare to single-agent baseline

Expected Finding

Conservative default protocol during high-disagreement periods is likely to produce the best risk-adjusted performance — reducing exposure during uncertain transitions.

What We Covered in Section H

- **Why multi-agent:** specialization, debate, redundancy
- **Cooperative vs. competitive:** shared goal vs. adversarial roles
- **Multi-agent debate:** Du et al. shows 11% accuracy improvement
- **MADDPG and QMIX:** RL for cooperative multi-agent systems
- **Disagreement as signal:** high agent conflict → regime transition / high volatility
- **Conflict resolution protocols:** weighted voting, conservative default, escalation
- **Human escalation policy:** designed in from the start, not an emergency measure
- **TradingGPT, FinMem:** layered memory + character diversity
- **RL for execution and market-making:** Nevmyvaka, Spooner results
- **Time series LLMs:** Chronos, Lag-Llama, Time-LLM

Bridge to Part 3

We have now built the full AI system. The critical question remains: **does it actually work in the real world?** Part 3 covers the rigorous testing methodology, risk management, and ethical framework needed to deploy these systems responsibly.

Part 3

Robustness and Ethics

Section I: Rigorous Backtesting

Slides 189–262

Presenters: Madmoun (backtesting & execution) • All (ethics & governance)

Section I

Rigorous Backtesting

“Does Your Strategy Actually Work?”

Slides 190–214

What Is Backtesting?

Definition: Backtesting

Backtesting is the process of evaluating a trading strategy or model by simulating how it would have performed on *historical data*. You apply the strategy to past data and measure the resulting hypothetical profit and loss.

The Sports Team Analogy

Imagine scouting a basketball player. You watch recordings of their past games to evaluate how they play. Backtesting is the financial equivalent: you watch your strategy's "past game tapes" to see how it would have done.

What backtesting measures:

- Returns: how much profit/loss over the period
- Risk: how much volatility and drawdown
- Sharpe ratio: return per unit of risk
- Hit rate: fraction of trades that were profitable
- Maximum drawdown: largest peak-to-trough loss

Why backtesting is necessary but insufficient:

- Necessary: can't deploy blind; need some evidence
- Insufficient: past performance is not future performance; many ways the test can mislead you
- The rest of Section I covers all the ways backtests can lie

The Fundamental Problem

Given enough parameters and enough historical data, you can always find a strategy that looks great in the past. This is **data snooping** or **backtest overfitting** — and it is the most common failure mode in quantitative finance.

The Coin-Flip Analogy

Suppose you flip a coin 1,000 times and record the sequence. Now you try to find a “pattern” that predicts the next flip. With enough creativity, you can find a rule that works 70% of the time on those 1,000 flips. But if you test it on the *next* 1,000 flips, it will perform at roughly 50% (random chance). You found noise, not signal.

Machine learning models applied to financial data face exactly this problem, amplified by the ability to test millions of patterns automatically.

Lopez de Prado's Warning

In his book “Advances in Financial Machine Learning” (2018), Marcos Lopez de Prado estimates that the majority of published quantitative trading strategies are false discoveries that will not perform out-of-sample. Rigorous backtesting methodology is the only defense.

The Most Basic Protection

Never evaluate your strategy on the same data you used to develop it. Split your data into:

- **In-sample (training)**: used to develop and optimize the strategy
- **Out-of-sample (test)**: held out completely, used only for final evaluation

Correct Temporal Split

Timeline: 2010–2023

In-sample: 2010–2019 (use to develop and fit)

Out-of-sample: 2020–2023 (evaluate only once, at the end)

Key rule: once you look at the out-of-sample period to evaluate, it becomes “used” — you cannot then modify your strategy and re-evaluate. That would be “peeking.”

The Hidden Contamination

Even with a formal split, you may still contaminate the out-of-sample period if:

- You use future data in feature construction (look-ahead bias)
- You use the same stocks that survived to today (survivorship bias)
- You choose the evaluation metric after seeing the results (selection bias)

Each of these is covered in subsequent slides.

Definition: Look-Ahead Bias

Look-ahead bias occurs when a backtesting system uses information that would not have been available at the time of the trade — effectively allowing the strategy to “see the future” during the historical simulation.

Classic Example: Earnings Announcements

A company reports earnings after market close on Tuesday. The actual number is known only after Tuesday's close. But if your backtest uses the earnings number in a feature for trades made at Monday's open, you've looked ahead — the Monday trade used Tuesday's information.

The backtest will show great results (“buy before good earnings”) but it's useless in practice because you can't know Tuesday's earnings on Monday.

Common sources of look-ahead bias:

- Using “point-in-time” data incorrectly (restated financials)
- Filling missing data with future values
- Feature normalization using the full dataset (future data in normalization)
- BERT/LLM inference run on historical data but with a model trained on future text

Prevention:

- Use *point-in-time* databases (data as it was known at each historical date, not as later revised)
- Add a time lag between data availability and feature computation
- Simulate data latency explicitly in the backtesting framework

A Concrete Walkthrough

Scenario: Using Annual Report Data

Company X files its 2019 annual report (10-K) on March 15, 2020.

Correct backtesting: Any strategy for January 2020 trades can only use 2018 annual report data (available March 2019). The 2019 annual report is not yet available.

Biased backtesting: Your database stores 2019 annual report data with a label saying “fiscal year 2019.” Your code assigns this to trades during all of calendar year 2019. But the report was filed March 2020!

Effect: Your strategy appears to predict stock performance using data that wasn't available at trade time. It backtests brilliantly. In live trading, it fails — the model tries to use data it can't access yet.

LLM/NLP Specific Risk

When using LLM sentiment models: ensure the model was trained only on data available *before* the date you're generating signals for. An LLM trained on 2023 data cannot be used to generate signals for 2020 trades in a backtest.

Definition: Survivorship Bias

Survivorship bias occurs when a backtest uses only stocks (or funds) that still exist today, excluding those that were delisted, went bankrupt, or were acquired. This creates an upward bias in historical returns.

The Military Aircraft Analogy

In World War II, British statistician Abraham Wald was asked where to add armor to bomber planes — based on data showing where planes that returned from missions had bullet holes. His insight: the planes that were shot *where he saw the holes* survived. They should armor the places *without holes* — because planes hit there didn't come back. You only see the survivors.

In finance:

- In 2010, the S&P 500 included many companies that later went bankrupt (Kodak, Sears, etc.)
- If you backtest on “2024 S&P 500 companies” going back to 2010, you exclude the failures
- Survivorship-biased database shows higher returns because only successful stocks remain

Size of the bias:

- Studies estimate survivorship bias in mutual fund databases: $\sim 1\text{--}3\%$ per year
- For individual stocks: even larger (failed firms had large negative returns before delisting)
- LLM studies: if trained on text from surviving companies only, the model learns an optimistic view of corporate outcomes

How Large Is the S&P 500 Turnover?

The S&P 500 is a living index. Companies are added when they grow large enough and removed when they shrink, merge, or go bankrupt. This means the 2024 S&P 500 is very different from the 2000 S&P 500.

Notable S&P 500 Departures

Companies once in the S&P 500 that no longer exist:

- **Enron**: fraud, bankrupted 2001
- **Lehman Brothers**: bankrupted 2008
- **Sears**: delisted 2018
- **Kodak**: nearly bankrupt, removed 2012
- **General Electric**: demoted to Dow removal, 2018

The Quantitative Impact

If you test a strategy on “today’s S&P 500 going back 20 years,” you implicitly:

- Exclude stocks that had massive losses before delisting
- Include stocks that had massive *gains* before being added to the index
- This is a selection of the best survivors
- Typical bias: 1–4% per year added to backtest returns

Solution

Use a point-in-time index membership database. CRSP (Center for Research in Security Prices) provides historical S&P 500 membership with correct entry/exit dates.

Selection Bias: Choosing the Metric That Looks Best

Definition: Selection Bias

Selection bias occurs when the researcher chooses the evaluation metric, time period, or universe of stocks *after seeing the results* — picking the combination that makes the strategy look best.

The Dartboard Analogy

Throw darts at a blank wall. After they land, draw a target around the cluster. Now you have a perfect score — but it's meaningless because the target was drawn around the result.

In quant finance: "Our strategy has a Sharpe ratio of 2.1... when tested on US small-cap stocks, using our custom return metric, over the period 2010–2017 (excluding 2008–2009 and 2018–2023)."

Forms of selection bias:

- Choosing the best evaluation period (exclude bad years)
- Choosing the universe of stocks (use only stocks where the strategy worked)
- Choosing the metric after seeing results (Sharpe looked bad; use Calmar ratio instead)
- Stopping the backtest early (when it happened to look good)

Prevention:

- Pre-register the evaluation protocol before running the test
- Use the full available data period
- Report all metrics, not just the best ones
- Compare to simple baselines (buy-and-hold, equal-weight)

A Subtle ML-Specific Problem

When you train an ML model on panel data (many stocks $\times$ many time periods), information from stock A at time t can leak into the prediction for stock B at time t , even if their returns are separated in the target variable.

How It Happens

You train a model using features from the cross-section of all stocks on date t to predict returns over the next month. Standard train-test split: train on 2010–2018, test on 2019–2023.

Problem 1: If you normalize features using statistics computed over the full cross-section (2010–2023), you use future information in feature computation.

Problem 2: If any features are computed using returns of other stocks (e.g., “momentum relative to sector average”), and those other stocks overlap between train and test, the model may learn non-generalizable patterns.

Prevention

Normalize features within the training set only; recompute normalization parameters out-of-sample using only past data. Never use future cross-sectional statistics in feature construction.

Definition: Walk-Forward Validation

Walk-forward testing simulates how the strategy would actually have been developed and deployed over time: periodically retrain on the most recent data and test on the immediately following period — rolling forward through history.

Walk-Forward Procedure

1. Train on 2010–2013; test on 2014 (record results)
2. Train on 2010–2014; test on 2015 (record results)
3. Train on 2010–2015; test on 2016 (record results)
4. ...continue through 2023
5. Aggregate all out-of-sample test periods: this is your realistic performance estimate

Why Walk-Forward Is Better

- Simulates real deployment: model is retrained as new data accumulates
- Each test period is truly out-of-sample (model never saw it during training)
- Tests model adaptability to regime changes
- Reveals if strategy degrades over time (common if based on a decaying edge)

The Cost

Walk-forward testing requires training the model many times (N retraining steps). For expensive LLM fine-tuning, this may be computationally prohibitive. Consider cheaper proxy evaluations or smaller models for walk-forward, reserving full models for final validation.

The Problem with Standard K-Fold Cross-Validation

Standard k -fold cross-validation randomly shuffles data and creates folds. For financial data, this creates **temporal leakage**: features in the training fold may be correlated with (or causally related to) labels in the test fold, just because they are temporally close.

Purged Cross-Validation

Purging: remove from the training set any observations that overlap in time with the test set labels. This prevents the model from learning spurious correlations driven by temporal proximity.

Embargo: additionally remove observations near the boundary of the test set (a “buffer zone”) to prevent information leakage through slowly-evolving features.

Concrete Example

Test set: predict returns for month T .

Without purging: training set includes observations from month $T - 1$ using features that incorporate price data from week before T — which partially overlaps with the test label (month T return).

With purging: all training observations within an “embargo period” of the test fold are removed. No temporal overlap possible.

What Is an Embargo Period?

Definition: Embargo Period

An **embargo period** is a buffer of time between the end of the training set and the start of the test set, deliberately excluded from both, to prevent information from one set from leaking into the other through slowly-evolving features.

Why Slowly-Evolving Features Cause Problems

Many financial features are computed over rolling windows (e.g., 20-day moving average of volume). If the training set ends on June 30 and the test set starts July 1, the July 1 features still contain data from June 12–30 — which overlap with the last training observations. The embargo period removes both June 16–30 from training and June 1–15 from testing, eliminating overlap.

How long should the embargo be?

- At minimum: length of the longest rolling window in your features
- For 20-day momentum: 20-day embargo
- For 60-day MA: 60-day embargo
- For LLM-generated sentiment: match to signal decay period (often 5–10 days)

Embargo in practice:

- Lopez de Prado's `mlfinlab` library implements purged cross-validation with embargo
- When using walk-forward: add the embargo buffer between each train and test window
- Increases the total amount of data “wasted” but dramatically reduces false discoveries

The Scope of the Problem

Without an embargo, your cross-validation performance estimate is *optimistically biased* because the model implicitly uses features that contain future information. The bias is not random; it consistently inflates apparent performance.

Quantitative Study (Lopez de Prado, 2018)

On a financial dataset with auto-correlated features (typical), the difference in measured model performance:

- **Standard k-fold cross-validation:** Sharpe ratio = 1.8 (optimistic)
- **Purged k-fold with appropriate embargo:** Sharpe ratio = 0.9 (realistic)

The standard approach overstated performance by 2×! Deploying based on the standard CV estimate would lead to massive disappointment (and financial loss) in production.

LLM-Specific Note

For LLM sentiment signals with typical decay of 3–5 days: use a 5-day embargo at minimum. For earnings-driven signals (one month between reports): use a 30-day embargo. Match embargo to your signal's auto-correlation structure.

The Sharpe Ratio: Risk-Adjusted Return

Definition: Sharpe Ratio (Sharpe, 1966)

The **Sharpe ratio** is the most commonly used measure of risk-adjusted return in finance. It measures how much excess return you earn per unit of risk (volatility).

$$\text{Sharpe Ratio} = \frac{R_p - R_f}{\sigma_p}$$

where R_p = portfolio return, R_f = risk-free rate (e.g., T-bill rate), σ_p = standard deviation of returns.

Interpreting Sharpe Ratios

Sharpe Ratio	Assessment	Context
< 0	Worse than cash	Losing money
0–0.5	Poor	Not worth the risk
0.5–1.0	Acceptable	S&P 500 long-run ≈ 0.5
1.0–2.0	Good	Competitive hedge fund
> 2.0	Excellent	Top quantitative funds
> 3.0	Suspicious	Likely backtest overfitting

High Sharpe $\neq$ Real Alpha

A strategy with Sharpe 3.0 in backtest is more likely the result of overfitting than genuine skill. Healthy skepticism increases with Sharpe ratio when no out-of-sample validation is provided.

Bailey & Lopez de Prado (2014): The Deflated Sharpe Ratio

The **Deflated Sharpe Ratio (DSR)** adjusts the observed Sharpe ratio downward to account for the fact that researchers tested many strategies and reported only the best one. It asks: “What is the probability that a strategy with this observed Sharpe ratio is a true discovery, given how many strategies were tested?”

Intuition: The Multiple Testing Penalty

If you flip a fair coin 20 times, there's a 64% chance you'll get at least 6 consecutive heads somewhere in the sequence — purely by chance. If you run 100 different backtests and the best one has a Sharpe of 2.0, how much is due to luck?

The DSR penalizes the observed Sharpe based on:

- Number of strategies tested
- Distribution of Sharpe ratios of all tested strategies
- Skewness and kurtosis of returns (non-normal returns require larger samples)

Practical Target

$DSR > 0$ (i.e., adjusted Sharpe ratio is significantly above zero after the multiple testing penalty). If $DSR < 0$, the strategy is likely a false discovery even if the raw Sharpe looks good.

The Statistical Challenge

When you run N statistical tests, the probability that at least one gives a false positive (spurious significance) grows rapidly with N — even if all strategies are pure noise.

The Math of False Positives

With each test at 5% significance level (p-value threshold):

- 1 test: 5% chance of a false positive
- 10 tests: $1 - (0.95)^{10} = 40\%$ chance of at least one false positive
- 50 tests: $1 - (0.95)^{50} = 92\%$ chance of at least one false positive
- 100 tests: $1 - (0.95)^{100} = 99.4\%$ chance of at least one false positive

A researcher who runs 100 backtests and publishes the best one is almost guaranteed to be reporting a false discovery!

Why this happens in quant finance:

- Thousands of potential signals (price, volume, sentiment, macro...)
- ML models implicitly test millions of feature combinations
- Academic incentive: only significant results get published

How bad is it? Harvey et al. (2016) show that to account for the multiple testing in 316 published factor studies, the minimum t -statistic for a new factor to be credible has risen from 2.0 to 3.0 since 1990.

Definition: Bonferroni Correction

The **Bonferroni correction** controls the *family-wise error rate* (probability of any false positive) by dividing the significance threshold by the number of tests: use α/N instead of α .

Application

Testing 100 different trading strategies at 5% significance level:

Without correction: threshold $p < 0.05$

With Bonferroni: threshold $p < 0.05/100 = 0.0005$

Only strategies with $p < 0.0005$ (very strong evidence) are considered significant.

Advantages

- Simple to apply
- Provides strong protection against false discoveries
- Standard in clinical trials and genetic research

Disadvantages

- Very conservative: many true strategies will be rejected
- Doesn't account for correlation between tests
- With 1,000 tests, the threshold becomes unrealistically strict

For Finance

Bonferroni is so conservative that almost no strategies would pass. Better to use Benjamini-Hochberg (next slide) which controls a less strict but more useful criterion.

Benjamini & Hochberg (1995): FDR Control

The **Benjamini-Hochberg (BH) procedure** controls the *False Discovery Rate (FDR)*: the expected fraction of rejected hypotheses that are false. This is less strict than Bonferroni but more practical.

BH Procedure Step-by-Step

Say you test 100 strategies, get 100 p -values. Want $FDR \leq 5\%$:

1. Sort p -values from smallest to largest: $p_1 \leq p_2 \leq \dots \leq p_{100}$
2. Find the largest k such that $p_k \leq k \times 0.05/100$
3. Reject all hypotheses with $p \leq p_k$

Example: if 10 strategies have $p < 0.05/10 = 0.005$, reject all 10. At most 5% of those 10 are false positives (0.5 strategies on average).

Finance Interpretation

BH allows you to test many strategies and publish a portfolio of “significant” ones, with a guaranteed bound on what fraction are false discoveries. More practical than Bonferroni for systematic research programs.

White (2000): Consistent Reality Check for Data Snooping

White's Reality Check tests whether the best-performing strategy from a set of N strategies is significantly better than a benchmark, after accounting for data snooping.

How It Works

1. Run N strategies and record their out-of-sample performance relative to benchmark
2. Under the null hypothesis (no strategy has true edge), simulate the distribution of the **maximum** performance across N strategies using bootstrap resampling
3. Compare observed maximum performance to this null distribution
4. p -value: probability of seeing this extreme a maximum performance purely by chance, given N strategies were tested

Practical Takeaway

White's Reality Check answers: "Is my best strategy actually good, or would I have expected a strategy this good by chance from testing N strategies?" A strategy with $p > 0.05$ under this test has no significant edge.

Limitation

White's Reality Check assumes strategies are tested independently. Correlated strategies (all based on the same signal) require the improved Hansen's SPA test.

Hansen (2005): Superior Predictive Ability Test

Hansen's SPA test extends White's Reality Check by eliminating poorly-performing models from the bootstrap comparison, making it more powerful (less conservative) while maintaining proper size control.

The Improvement Over White's

White's Reality Check is conservative because it includes clearly inferior strategies in the null distribution, inflating the critical value. Hansen's SPA identifies and removes strategies that are clearly losers, focusing the test on strategies that are competitive. This gives more power to detect true edges.

Finance Application

Test a universe of 500 momentum strategies with different lookback periods and rebalancing frequencies:

1. Compute out-of-sample performance for all 500
2. Apply SPA test to determine if the best performer is a true discovery
3. If SPA $p > 0.10$: the best strategy is not significantly better than chance
4. If SPA $p < 0.05$: strong evidence for a real edge

Both Tests Are Necessary

Report both White's Reality Check and Hansen's SPA. If your strategy passes both (both $p < 0.05$), you have strong statistical evidence for a real edge.

Bailey, Borwein, Lopez de Prado, Zhu (2016): PBO

The **Probability of Backtest Overfitting (PBO)** uses combinatorial cross-validation to estimate the probability that your best strategy was selected by chance from a family of related strategies.

The Method: Combinatorial Purged Cross-Validation (CPCV)

1. Divide data into T time blocks
2. Generate all $\binom{T}{k}$ possible combinations of train/test splits
3. For each combination: which of your N strategies performs best in training?
4. Evaluate that strategy's performance in the corresponding test period
5. PBO = fraction of combinations where the train-best strategy underperforms in test

Interpretation

PBO = 0%: strategy consistently wins in-sample and out-of-sample. Strong evidence against overfitting.

PBO = 50%: strategy wins in-sample and loses out-of-sample half the time. Significant overfitting.

PBO = 100%: strategy always fails out-of-sample. Pure in-sample fitting.

Rule of thumb: Deploy only if PBO < 25%.

Harvey, Liu, Zhu (2016): ... and the Cross-Section of Expected Returns

Harvey et al. (2016) analyzed 316 factors published in top finance journals, all claiming to predict stock returns. Their key argument: the field has been running an implicit multiple testing experiment for decades, and most reported findings are likely false.

Key Findings (precisely stated)

- 316 factors documented in academic literature (as of 2016)
- **Main finding:** the minimum t -statistic required for a new factor to be credible has risen from 2.0 to **3.0** since 1990 — because the cumulative number of tested factors means the bar for significance must be higher
- **Implication:** many factors published with $t \approx 2$ are likely false discoveries, though Harvey et al. do not compute an exact false-discovery rate
- The field faces a replication crisis analogous to psychology

Implications for LLM-Based Signals

The same problem applies to AI-generated factors:

- LLMs can generate thousands of text-based features
- Testing all combinations without correction → guaranteed false discoveries
- The solution: pre-register hypotheses based on behavioral theory *before* testing
- Or: apply rigorous multiple testing corrections (BH, DSR, PBO) to all tested signals

Due Diligence Protocol

1. **Look-ahead bias:** Are all features using only data available at trade time? Is a point-in-time database used?
2. **Survivorship bias:** Does the universe include companies that later delisted or went bankrupt?
3. **Transaction costs:** Are realistic bid-ask spreads and market impact included?
4. **Slippage:** Is the assumed execution price realistic (not always at the daily close)?
5. **Walk-forward testing:** Was the model retrained periodically on expanding history?
6. **Embargo period:** Is an appropriate buffer between train and test periods applied?
7. **Multiple testing:** How many strategies were tried? What is the DSR or PBO?
8. **Selection bias:** Were the time period, universe, and metric pre-defined or chosen post-hoc?
9. **Regime coverage:** Does the test period include a bear market, a crisis, and a bull market?
10. **Capacity:** Would this strategy still work at the scale you plan to deploy, given market impact?

A Strategy That Passes All 10

Is worth taking seriously. A strategy that fails even one is suspicious. A strategy that fails multiple is almost certainly a false discovery.

What You'll Fix in Colab Notebook 3

You receive a backtest with *intentionally introduced flaws*. Your task: identify and fix each one.

1. **Flaw 1: Survivorship bias** — the universe uses 2024 S&P 500 membership going back to 2010. Fix: load correct historical membership from a provided CSV.
2. **Flaw 2: Look-ahead bias** — earnings feature is assigned to the date the earnings were *for*, not the date they were *released*. Fix: apply a 1-day lag for reporting delay.
3. **Flaw 3: No transaction costs** — the original backtest assumes zero cost. Fix: add 5bp round-trip cost.
4. **Flaw 4: No embargo** — the train/test split has no buffer. Fix: add a 20-day embargo.
5. **Flaw 5: Selection bias** — the metric was chosen after seeing results. Fix: use pre-specified Sharpe ratio comparison to buy-and-hold.

Expected Learning

You'll see the Sharpe ratio of the “backtest” drop from ~ 2.5 (impressive) to ~ 0.6 (mediocre) after all flaws are corrected. This is the most common experience when rigorous backtesting is applied to seemingly promising strategies.

The Six Stages Between a Notebook and Real Money

Most strategies die somewhere in this pipeline. Understanding each stage prevents deploying too early.

1. **Research prototype** (Colab/Jupyter): Proves the signal exists on historical data. Statistically clean (purged CV, embargo). *No latency requirements.*
2. **Paper trading** (live market, simulated orders): Signal runs on real-time data. Orders are logged but not executed. Reveals feed latency and data quality issues. *Run for ≥ 6 months across at least one volatile regime.*
3. **Shadow mode** (live orders, tiny size): Real orders at 1–2% of target size. Tests broker connectivity, fill rates, slippage. Reveals queue position effects.
4. **Controlled live** (live at 25% target size): First real P&L. Kill-switch and drawdown limits active. Human reviews every end-of-day.
5. **Full deployment**: All risk controls, audit logging, automated monitoring, and escalation paths in place. Circuit breaker wired.
6. **Continuous monitoring**: Daily drift tests, regime-change alerts, model performance attribution. Schedule quarterly recalibration.

The Most Common Mistake

Skipping stage 2 or 3. A strategy that looks good in backtests but is never paper-traded is speculative, not deployed. Each stage typically reveals issues the previous one missed.

Section J

Risk Management & Execution

“Surviving the Real World”

Slides 215–228

Why Backtesting Is Not Enough

The Gap Between Research and Live Trading

Even a perfectly rigorous backtest (all flaws corrected, no look-ahead, proper cross-validation) cannot fully predict real-world performance. Live trading adds costs and risks that are impossible to fully simulate.

What Backtests Cannot Capture

- Real transaction costs (they vary with market conditions)
- Market impact of your own trades (changes the prices you can achieve)
- Technology failures and latency spikes
- Counterparty risk (broker, exchange failures)
- Regulatory changes that affect strategy mechanics
- Novel market events (COVID-19, meme stocks) with no historical precedent

The Research-to-Production Discount

In practice: expect live performance to be **30–70% lower** than backtest performance, due to all these factors combined.

This means a backtest Sharpe of 1.5 may produce a live Sharpe of 0.5–1.0. Still potentially worthwhile — but you need realistic expectations.

The Lesson

Never deploy a strategy based solely on backtest results. Always paper-trade (simulate on live market data without real money) for a statistically significant period before committing capital.

All the Costs of Trading

A trade that looks profitable on paper may not be profitable after all transaction costs are accounted for.

Cost Type	Size (typical)	Description
Bid-ask spread	0.01–0.5%	Pay the spread every time you trade
Broker commission	0–0.01%	Broker fee per share (declining)
Market impact	0.05–1%	Your orders move the price against you
Exchange fees	Very small	Maker/taker fees ($\sim 0.002\%$)
Financing costs	Depends	Interest on borrowed capital or short rebate
Slippage	Variable	Difference between expected and actual fill
Opportunity cost	Hard to measure	The alternative use of capital

The Intraday Trading Killer

For a high-frequency strategy: if you trade 20 times per day with 5bp round-trip cost, you spend 100bp (1%) per day in costs. To be profitable, your daily return must exceed 1%. That is a very high bar.

Kelly Criterion — Continuous Normal-Return Formulation (Kelly, 1956)

For a strategy whose *log-returns are approximately normal* with expected excess return μ and volatility σ , the optimal capital fraction is:

$$f^* = \frac{\mu - r_f}{\sigma^2}$$

This is the **full-Kelly** fraction. In practice, most funds use **half-Kelly** ($f^*/2$) or **quarter-Kelly** to reduce variance and drawdown. (See also: *discrete win/loss formulation in Section J.*)

Worked Example

Strategy has annualized $\mu = 12\%$, $r_f = 4\%$, $\sigma = 20\%$:

$$f^* = \frac{0.12 - 0.04}{0.20^2} = \frac{0.08}{0.04} = 2.0 \text{ (200\%)}$$

Full-Kelly says use 2× leverage! In practice: use half-Kelly = 100% of capital, no leverage.

Kelly in Practice

- Full-Kelly produces very large drawdowns (up to 50%) even for good strategies
- LLM-based signals have uncertain μ and σ estimates — over-betting is common
- Standard practice: scale Kelly by 0.25–0.50 and impose hard drawdown limits as an independent circuit breaker

Revisiting Market Impact in the Execution Context

We introduced price impact in Section B. Now we quantify it for execution planning.

The Square-Root Law of Market Impact

Empirically, market impact follows an approximate square-root law:

$$\Delta p \approx \sigma \sqrt{\frac{Q}{V_{\text{daily}}}}$$

where: σ = daily volatility, Q = order size, V_{daily} = average daily volume

Example: Want to sell 100,000 shares of a stock with \$1M daily volume (10% participation rate) and 2% daily volatility:

$$\Delta p \approx 0.02 \times \sqrt{0.10} = 0.02 \times 0.316 = 0.63\%$$

On a \$50 stock, impact $\approx$ \$0.32 per share. On 100,000 shares: total impact cost $\approx$ \$32,000.

Capacity Constraint

This calculation shows why strategies have **capacity limits**: the more you trade, the worse your execution price. Very good strategies only work for small amounts of capital.

Almgren & Chriss (2000): Optimal Liquidation of Portfolio Holdings

The **Almgren-Chriss model** provides a mathematical framework for the optimal execution of a large order: minimizing the combined cost of market impact and market risk.

The Trade-Off

Trade fast: low market risk (price doesn't have time to move adversely) but high market impact (flood the order book).

Trade slow: low market impact (spread the order over time) but high market risk (price may move against you over the extended period).

Almgren-Chriss finds the **efficient frontier** of this trade-off.

Key Outputs

- **Optimal trading schedule:** how many shares to sell in each time interval
- Given a risk tolerance λ : more risk-averse $\rightarrow$ trade faster; less risk-averse $\rightarrow$ trade slower (VWAP-like schedule)
- Explicit formula depending on: order size, daily volume, volatility, temporary/permanent impact coefficients

Modern Extensions

TWAP (Time-Weighted Average Price) and VWAP (Volume-Weighted Average Price) are simple special cases. RL-based approaches (slide 181) generalize to non-linear impact and dynamic market conditions.

Definition: Funding Liquidity Risk

Funding liquidity risk is the risk that a trader cannot obtain the financing needed to maintain their positions, forcing them to liquidate at unfavorable prices.

The 2008 Crisis Analogy

During the 2008 financial crisis, many hedge funds had profitable positions (the right bets on housing collapse) but were forced to sell them at huge losses because:

1. Their prime brokers (who provided leverage) reduced credit limits ("margin calls")
2. Investors redeemed their capital (withdrawals)
3. The funds had to sell good positions to raise cash, further depressing prices
4. Lower prices triggered more margin calls and redemptions — a spiral

Brunnermeier-Pedersen Mechanism:

- Market liquidity (bid-ask spreads) and funding liquidity (margin availability) are interconnected
- When one deteriorates, the other does too — creating spirals
- "Liquidity spirals" can amplify small shocks into systemic crises

AI System Design Implication:

- AI systems must maintain liquidity buffers
- Cannot assume leverage will always be available
- Must stress-test under scenarios where funding dries up

Definition: Queue Position

When you submit a limit order, it joins a **queue** at its price level in the order book. Orders that were submitted earlier are at the front of the queue (time priority). Queue position determines whether and at what price your limit order fills.

Why Queue Position Matters

Suppose the best bid is \$100.00 with 10,000 shares queued. You submit a limit buy for 5,000 shares at \$100.00. You are at the back of the queue.

If a seller arrives willing to sell 7,000 shares: the first 7,000 shares of buyers get filled at \$100.00. The front 7,000 of the queue get their orders filled. Your 5,000 shares, being at the back, might only get 2,000 shares filled (what remains after the first 7,000).

Result: You got only partial fill. Your strategy assumed full execution; reality is partial.

Backtesting Implication

Most backtests assume that limit orders at a given price are always filled. In reality, fill rate depends on queue position. This systematic overestimation of fill rates inflates backtest performance, especially for market-making and high-frequency strategies.

Definition: Volatility Targeting

Volatility targeting is a risk management technique that scales position sizes to maintain a constant target level of portfolio volatility, regardless of how volatile markets are.

Simple Example

Target portfolio volatility: 10% per year.

Current estimated volatility of the strategy: 20% per year (markets are turbulent).

Required position scaling: $10\% / 20\% = 0.5$ (halve all positions)

Next month: markets calm down. Strategy volatility: 8% per year.

Required scaling: $10\% / 8\% = 1.25$ (increase positions 25%)

Benefits:

- Prevents catastrophic losses in high-volatility regimes
- Consistent risk profile across different market conditions
- Improves Sharpe ratio vs. fixed position sizing
- Natural AI risk control: scale down during uncertain periods

Challenges:

- Requires real-time volatility estimation (GARCH, realized volatility)
- Volatility can spike faster than positions can be reduced (gap risk)
- Selling during high volatility amplifies market stress (systemic risk)

Definition: Drawdown

A **drawdown** is the percentage decline from a portfolio's peak value to its current value. Maximum drawdown is the largest such decline over the entire history of the strategy.

Drawdown Control Rules

Automated rules that reduce or halt trading when losses exceed thresholds:

- **Daily VaR limit:** if today's loss exceeds X%, stop trading for the day
- **Monthly drawdown limit:** if monthly loss exceeds Y%, reduce all positions by 50%
- **Maximum drawdown limit:** if cumulative loss from peak exceeds Z%, cease trading entirely and await human review

Why These Rules Are Necessary

- AI systems can lose money faster than humans can respond
- Without hard limits, a malfunctioning AI could blow up an entire portfolio before a human notices
- Drawdown limits create “circuit breakers” — analogous to exchange circuit breakers from the Flash Crash
- Also important for investor/client relations: drawdowns beyond certain thresholds trigger redemptions

Setting the Limits

Limits should be set based on the strategy's historical volatility during stressed periods, not its average behavior. Use worst-case scenarios, not expected scenarios.

The Non-Negotiable Safety Mechanism

Any AI system that can submit financial orders *must* have a kill switch: a mechanism that immediately halts all new order submissions and can cancel pending orders, triggered either automatically or by a human.

Automatic Kill Switch Triggers

- Real-time P&L loss exceeds threshold (e.g., -3% intraday)
- Order submission rate exceeds anomaly threshold (fat-finger detection)
- Network connectivity lost to exchange
- Risk limits breached (position size, drawdown, VaR)
- Market halt or circuit breaker detected
- Anomalous price movements detected (e.g., price $> 5\sigma$ move in 1 second)

Manual Kill Switch Requirements

- Accessible to any authorized operator in < 5 seconds
- Works even if primary system is malfunctioning
- Has an independent pathway (not routed through the same system being stopped)
- Cancels all open orders at all exchanges simultaneously
- Logs activation time and reason for audit trail

Regulatory Requirement

MiFID II (EU) and FINRA (US) both require documented kill switch capabilities for algorithmic trading systems. This is not optional.

Kelly Criterion — Discrete Probability Formulation (Kelly, 1956)

When a strategy can be modelled as a sequence of *binary bets* (win with probability p , lose with probability $q = 1 - p$, payoff ratio b), the optimal capital fraction is:

$$f^* = \frac{p \cdot b - q}{b}$$

where p = win probability, $q = 1 - p$, b = gain per dollar risked. (*The continuous normal-return version $f^* = (\mu - r_f)/\sigma^2$ appears in Section J.*)

Simple Example

A strategy: 60% chance of +10% return, 40% chance of -10% return.

$f^* = (0.6 \times 1 - 0.4)/1 = 0.20 = \text{bet 20\% of capital on each trade.}$

Why Kelly matters for AI:

- Provides a principled framework for translating model confidence into position size
- Larger positions when model is more confident = efficient capital allocation

Half-Kelly in Practice

Full Kelly is very aggressive and sensitive to errors in p and b estimates. Most practitioners use **half-Kelly** ($f^*/2$) or quarter-Kelly to reduce variance and protect against model estimation errors.

Risk Is Not an Afterthought

The biggest mistake in AI system deployment is treating risk management as a layer added on top of the core system after the fact. Risk controls must be designed into the system architecture from day one.

Risk Control Hierarchy

1. **Signal level:** model calibration, uncertainty quantification
2. **Position level:** Kelly criterion, position limits per security
3. **Portfolio level:** sector exposure limits, correlation limits, VaR target
4. **Strategy level:** daily P&L drawdown limits
5. **System level:** kill switches, order rate limits
6. **Firm level:** aggregate exposure across all strategies
7. **Regulatory level:** FINRA/MiFID II compliance monitoring

For AI-Specific Risks

Additional controls needed for LLM/agent systems:

- **Hallucination guard:** fact-check all factual claims before use
- **Confidence threshold:** only act if model confidence exceeds minimum
- **Anomaly detection:** flag when inputs are out-of-distribution (regime change)
- **Human review queue:** maintain a backlog of decisions for human audit
- **Model performance monitoring:** detect when live performance diverges from expected

The Gap Between Research and Production

Why Great Research Systems Often Fail in Production

A research system is optimized for discovery. A production system is optimized for reliability. These have different requirements that are often incompatible without deliberate engineering.

Research Environment

- Clean, pre-processed data
- No latency constraints
- No reliability requirements
- Can retry failed API calls
- Results batched and analyzed later
- No real money at stake

Production Environment

- Messy, real-time, incomplete data
- Latency is critical (seconds matter)
- 99.9% uptime required
- Failures must be handled gracefully
- Real-time monitoring and alerting
- Real money at stake: every mistake has cost

The Translation Checklist

Before going live: data pipeline validation, latency profiling, failure mode testing (what happens if the LLM API is down?), paper trading period, gradual capital ramp-up (start at 1% of target size), monitoring dashboard with real-time alerts.

Summary: Risk Management Is a First-Class Design Requirement

- **Backtesting is necessary but not sufficient:** live trading always differs from simulation
- **Transaction costs:** spread, impact, slippage, financing must be modeled realistically
- **Market impact:** the square-root law quantifies execution costs for large orders
- **Almgren-Chriss:** optimal execution trades off impact vs. market risk
- **Funding liquidity:** leverage can disappear in crises (Brunnermeier-Pedersen)
- **Volatility targeting:** scale positions to maintain constant risk
- **Drawdown controls:** hard limits on intraday and cumulative losses
- **Kill switches:** mandatory, independent, fast-acting
- **Kelly criterion:** principled position sizing from model confidence
- **Research-to-production gap:** reliability, latency, and failure handling are new challenges

Coming Up: Section K

We have covered the technical and statistical risks. Now we turn to the deeper question: even if our AI system works perfectly, *should it*? Section K addresses the ethics and governance of deploying AI in financial markets.

Section K

Ethics and Governance

“Power and Responsibility”

Slides 229–250

A Perfect Storm of Ethical Complexity

Financial AI combines several features that make ethical analysis particularly difficult:

- **High stakes:** decisions affect real people's savings, retirements, and financial security
- **Speed:** AI acts faster than human ethical review can follow
- **Opacity:** complex ML models are difficult to audit and explain
- **Competition:** disclosing your ethical practices can be competitively disadvantageous
- **Systemic effects:** correlated AI behavior can destabilize entire markets, harming people who never interacted with the AI
- **Dual use:** the same tools that help researchers understand markets can be used to exploit retail investors
- **Regulatory lag:** technology moves faster than regulations can adapt

The Standard ML Ethics Framework Is Insufficient

Most ML ethics work focuses on fairness, bias, and privacy in classification systems. Finance AI raises different and harder questions: market manipulation, systemic risk, and information asymmetry. These require a specialized ethical framework.

The Three Core Tensions of This Tutorial

We Promised to Confront the Ethics Head-On

This section presents three deep tensions that do not have easy resolutions. We present arguments on both sides and encourage you to form your own view.

Tension 1	Tension 2	Tension 3
Discovering vs. Exploiting Biases Is it ethical to use knowledge of cognitive biases to trade against people experiencing those biases?	AI and Systemic Risk Do correlated AI strategies create systemic risks that justify regulatory intervention, even at the cost of innovation?	Transparency vs. Competitive Advantage How much should AI trading firms be required to disclose about their systems, given that disclosure undermines their edge?

No Easy Answers

These are genuine ethical dilemmas debated by academics, regulators, and practitioners. We will lay out the strongest arguments on each side and identify the key principles at stake.

The Setup

Behavioral finance (Section C) tells us that investors make systematic, predictable mistakes. Our AI systems are explicitly designed to identify and profit from these mistakes. Is this ethical?

The Positive Case: Discovery

- Academic research identifies biases to *help* investors (by educating them)
- Quantitative strategies that exploit biases provide liquidity and price discovery
- Arbitrageurs who trade against overreaction pull prices *toward* efficient levels
- The resulting price efficiency benefits all investors (more accurate prices)
- Competition among strategies eventually eliminates the bias (the market learns)

The Negative Case: Exploitation

- Retail investors are systematically disadvantaged by their own psychology
- AI tools deepen this asymmetry — the sophisticated exploit the naive
- The profits of bias-exploiting strategies come *directly* from less sophisticated investors
- “Price efficiency” is small comfort to the retail investor who lost money
- The benefits accrue to wealthy firms; the costs are borne by ordinary people

Quantifying the Asymmetry

The information and technology gap between institutional AI traders and retail investors has grown dramatically over the past decade.

Institutional AI Advantages

- Real-time alternative data (satellite imagery, credit card transactions, web traffic)
- Millisecond execution infrastructure
- Teams of PhDs developing models
- Access to Bloomberg Terminal (\$24,000/year)
- LLMs processing millions of documents daily
- Sophisticated risk management

Retail Investor Reality

- Access to delayed (15-minute) or real-time price data (free apps)
- Execution in seconds (not milliseconds)
- Decisions often driven by social media, news, and emotion
- Limited ability to diversify (high stock-specific risk)
- Behavioral biases (disposition effect, herding, overreaction) unmitigated
- Known to be systematically exploited by HFT payment-for-order-flow arrangements

Barber et al. (2009): Retail Investors in Taiwan

Analysis of the full Taiwan stock market (1995–1999): retail investors collectively *lost* \$32 billion over 5 years in aggregate trading. Institutions captured this value. The transfer from retail to institutional is well-documented empirically.

Is Profiting from Biases Discovery or Predation?

The Core Question Revisited

Does an AI that exploits the disposition effect provide social value, or does it simply transfer wealth from the less sophisticated to the more sophisticated?

The Grocery Store Analogy (Positive)

A supermarket uses behavioral science to design its layout — placing high-margin items at eye level, discount items at the bottom shelf. Consumers “irrationally” buy more expensive items. But they’re not forced to; they could read shelf labels more carefully. The market is competitive; better-designed stores prosper. This is legal and standard.

The Predatory Loan Analogy (Negative)

Predatory lenders specifically target financially illiterate people with complex products designed to extract maximum value. Even if technically legal and voluntary, this is widely considered unethical because it systematically exploits cognitive limitations of a vulnerable population.

Where Does Finance AI Fall?

The answer likely depends on: (1) the sophistication of the counterparty (retail vs. institutional), (2) whether the bias is informed (investor chose a risky strategy knowing the odds) or uninformed (investor was genuinely misled), (3) whether the AI actively *creates* the bias or merely profits from one that already exists.

Strongest Case FOR Bias-Exploiting AI

1. Markets allocate capital. Efficient prices (produced by arbitrage) mean capital flows to the most productive uses. This benefits the whole economy.
2. Retail investors voluntarily participate in markets. They are not obligated to. If they understand the risks and choose to participate, their losses are not exploitation.
3. AI reduces transaction costs (tighter spreads from HFT market making), benefiting all participants including retail investors.
4. Regulation that restricts sophisticated strategies tends to entrench existing advantages rather than level the playing field.

Strongest Case AGAINST

1. Cognitive biases are not choices — they are hardwired. Saying investors “voluntarily” experience the disposition effect is like saying people “voluntarily” get hungry. Exploiting involuntary psychological processes is categorically different from competitive business advantage.
2. The social value of the last few microseconds of price efficiency (from HFT) is negligible compared to the costs of the wealth transfer it facilitates.
3. A system where sophisticated AI extracts value from retail investors erodes public trust in markets, reducing long-run market participation and capital formation.
4. The KDD research community should consider whether publishing bias-exploiting methods creates publication externalities (more on this in slide 250).

The Question

Do AI trading strategies, collectively, increase the probability or severity of financial crises? And if so, does this justify restricting their deployment even if each individual strategy is profitable and legal?

The Mechanism for Systemic Risk

- Many AI systems trained on similar data learn similar strategies
- Similar strategies generate correlated positions
- Correlated positions lead to correlated liquidations during stress events
- Correlated liquidations amplify market crashes
- Amplified crashes cause broader economic harm (pension funds, 401ks, credit markets)

Historical Precedents

- **2007 Quant Quake:** A sudden unwinding of similar quant strategies caused a multi-standard-deviation loss event in August 2007 — before the financial crisis officially began
- **Flash Crash 2010:** Correlated HFT behavior amplified a single institutional order into a market-wide crisis
- **March 2020:** Systematic strategies and risk parity funds all de-risked simultaneously as volatility spiked, amplifying the COVID crash

The Homogenization Hypothesis

If all major asset managers deploy similar LLM-based sentiment strategies, similar RAG pipelines reading the same news feeds, and similar multi-agent systems with similar risk rules, the financial system becomes more *fragile* — not more robust — because its participants' behavior is correlated.

A Stylized Scenario

Year: 2027. 50 major hedge funds deploy FinBERT-based earnings call sentiment models. All read the same earnings call. All extract “negative guidance” signal. All submit sell orders within 200ms. The stock falls 12% before any human can react. One of the funds' stop-losses triggers automatically, adding more selling. Others follow. A normal 3% earnings miss becomes a 15% crash. The stock recovers partially over 3 days as manual analysis reveals the sell-off was overdone.

The Externality Problem

Each individual fund's strategy is rational and beneficial for that fund. The harm comes from the *collective* behavior. This is a classic negative externality — the same type that justifies government intervention in pollution, traffic, and public health. Markets cannot solve negative externalities without coordination.

Financial Stability Board: Monitoring AI in Finance

The **Financial Stability Board (FSB)** is the international body that monitors global financial stability, reporting to the G20. It has issued two landmark reports on AI in finance.

FSB (2017): Artificial Intelligence and Machine Learning in Financial Services

- First major regulatory body report on AI in finance
- Identified key risks: opacity of AI decisions, potential for correlated behavior, speed of AI-driven markets
- Recommended: monitoring for herding behavior, requirements for explainability, stress testing of AI systems under adverse conditions

FSB (2024): Report on AI in Finance

- Updated assessment with focus on LLMs and generative AI
- New concerns: AI hallucination in financial analysis, concentration risk (few AI providers serving many firms), AI-enabled cyber threats
- Recommendations: third-party model risk management standards, AI governance frameworks at firm level, international coordination on AI standards

What This Means for Practitioners

Regulatory pressure on AI in finance is increasing. Systems deployed today must be designed with governance and explainability in mind, not as afterthoughts. The regulatory requirements are likely to become more stringent, not less.

The Question

Financial firms argue that disclosing AI trading strategies destroys their value. Regulators and the public argue that opaque AI systems operating in public markets create unacceptable risks. Who is right?

The Disclosure Problem

- A trading strategy is worth money only if competitors don't know it
- If a firm must disclose its LLM-based sentiment strategy, competitors copy it and arbitrage away the edge
- This reduces the incentive to invest in research and development
- Result: fewer innovations, less efficient markets

The secrecy may actually serve the public interest by incentivizing innovation that eventually produces more efficient markets.

The Public Interest Case

- AI systems operating in public markets affect public goods (price discovery, market stability)
- The public has a legitimate interest in knowing what algorithms are moving prices
- Opaque systems cannot be audited for market manipulation, insider trading, or fairness
- The 2010 Flash Crash happened in part because no regulator understood what the algorithms were doing
- Transparency is a prerequisite for accountability

Definition: Explainability (Interpretability)

Explainability in AI refers to the ability to provide human-understandable reasons for a model's decisions. Why did the model predict this stock would go up? Which features drove the decision?

LIME (Ribeiro et al., 2016)

LIME (Local Interpretable Model-agnostic Explanations) explains individual predictions by fitting a simple (linear) model around each specific input.

For a stock prediction: LIME would tell you that for this specific earnings call, the model's "buy" signal was driven 40% by the word "expansion," 30% by the CFO's tone, and 30% by the revenue growth figure.

SHAP (Lundberg & Lee, 2017)

SHAP (SHapley Additive exPlanations) uses game theory to assign each feature a "contribution" to the prediction, with theoretical guarantees on consistency.

Finance application: SHAP values show which sentences from an earnings call drove the sentiment score, allowing compliance officers to verify the model's reasoning is based on material public information, not insider signals.

Limitation

LIME and SHAP explain ML models. For LLMs, attention visualizations and chain-of-thought prompting provide complementary but incomplete explanations. Full mechanistic interpretability of LLMs remains an open research problem.

Definition: Counterfactual Explanation

A **counterfactual explanation** answers: “What would have needed to change for the model to give a different prediction?” It describes the *minimum change* to the input that flips the output.

Counterfactual for a Credit Decision

Model decision: “Deny loan to Company X.”

Counterfactual: “If Company X’s debt-to-equity ratio were 1.2 (instead of 2.4) and their interest coverage ratio were 3.0 (instead of 1.5), the model would have approved the loan.”

This is actionable: it tells Company X exactly what needs to change.

Finance applications:

- Explain why a stock was rated “sell” vs. “hold” (counterfactual: what earnings beat would flip it?)
- Regulatory compliance: show that the AI’s decision was based on permitted factors
- Client reporting: explain portfolio changes in actionable terms

Limitations:

- Counterfactuals can be unrealistic (“if interest rates were negative 5%...”)
- Multiple counterfactuals may exist; choosing which to report is a modeling decision
- For LLMs: generating meaningful counterfactuals over text inputs is challenging

The Regulatory Landscape for Explanations

Several existing and emerging regulations create legal requirements for AI explainability in finance.

GDPR (EU, 2018)

Article 22: individuals have the right to explanation for automated decisions that significantly affect them. Applies to AI credit scoring, investment suitability assessments, and AML/KYC systems.

In practice: if an AI denies someone a loan, they have the right to know why.

EU AI Act (2024)

High-risk AI applications (including AI in credit assessment, insurance, and potentially trading systems affecting retail investors) must provide documentation, testing, and human oversight. “Black box” systems in high-risk categories may be prohibited.

MiFID II (EU, 2018)

Requires investment firms using algorithmic trading to document their systems and demonstrate that they can be adequately controlled and monitored. Explainability is an implicit requirement of meaningful control.

SEC Regulation

The SEC has brought enforcement actions against “AI washing” — firms claiming to use AI without actually doing so. As AI becomes central to trading, the SEC will require accurate disclosure of AI system characteristics, implying some level of explainability.

The World's First Comprehensive AI Law

The **EU AI Act** (adopted 2024, phased implementation 2025–2027) creates a risk-based regulatory framework for AI systems in the EU.

Risk Tier	Finance Examples	Requirements
Unacceptable High Risk	Manipulative subliminal AI Credit scoring, insurance pricing	Banned entirely Conformity assessment, documentation, human oversight
Limited Risk Minimal Risk	Chatbots for customers Back-office automation	Transparency disclosure No specific requirements

Where Trading AI Falls

The classification of LLM-based trading systems is still being determined. Systems that directly affect retail investment decisions or credit are likely “high risk.” Pure institutional trading systems may fall into lower tiers — but this is an active regulatory debate.

What High-Risk Means for Finance AI

- Register the system in the EU AI database
- Conduct conformity assessment (internal or external)
- Implement human oversight mechanisms
- Maintain detailed technical documentation
- Report serious incidents to regulators

Markets in Financial Instruments Directive II (EU, 2018)

MiFID II is the EU framework governing investment services and markets. Its rules on algorithmic trading directly affect AI trading systems.

Key Requirements for AI Trading

- **Annual self-assessment:** test all algorithms before deployment and annually; document testing results
- **Kill switch:** must have the ability to cancel orders and halt algorithms immediately
- **Market abuse prevention:** systems must not generate orders that could constitute market manipulation (even inadvertently)
- **Synchronized clocks:** timestamps on all orders must be accurate to the microsecond
- **Stress testing:** demonstrate system can handle stressed market conditions

What MiFID II Means in Practice

1. Before deploying an LLM-based trading agent: run it in a test environment and document its behavior under various market scenarios
2. Have a compliance team sign off on whether the agent's strategies could constitute market manipulation
3. Maintain a kill switch that can halt the agent in < 5 seconds
4. Keep audit logs of every order, linked to the reasoning that produced it
5. Annually repeat testing as the LLM updates or market conditions change

European Securities and Markets Authority

ESMA (ESMA, 2024) issued guidance on the use of AI (including LLMs) in investment services, clarifying how existing frameworks apply and proposing additional safeguards.

Key ESMA Principles

1. **Human oversight:** material decisions must retain meaningful human control; AI cannot replace the “responsible person” required by existing regulation
2. **Model risk management:** LLMs used for investment decisions must be subject to a model validation process similar to that for quantitative models
3. **Data governance:** training data for finance AI must be of documented quality; bias in training data must be assessed

Key ESMA Principles (cont.)

4. **Explainability:** firms should be able to explain AI-driven investment decisions to clients and regulators in plain language
5. **Fairness:** AI systems must not systematically disadvantage certain client groups
6. **Vendor risk:** using third-party AI (OpenAI, Anthropic, etc.) does not transfer regulatory responsibility; the firm remains accountable

ESMA's Position on LLM Hallucinations

ESMA specifically flags LLM hallucinations as a risk: any AI-generated financial analysis that contains material factual errors could constitute misleading communication to clients — a regulatory violation with serious consequences.

US Securities and Exchange Commission

The SEC has taken an increasingly active stance on AI in investment management, with several recent enforcement actions and guidance documents.

AI Washing Enforcement

In 2024, the SEC charged investment advisors with **AI washing**: claiming to use AI in investment decisions when they did not. This establishes that:

- Claims about AI must be accurate and substantiated
- “AI-powered” is a material claim that investors rely on
- Misleading AI claims = securities fraud

Expected AI Disclosures

The SEC expects registered investment advisors to disclose in Form ADV:

- Whether AI or ML is used in investment decision-making
- The nature of AI system oversight (human vs. automated)
- Material conflicts of interest arising from AI use
- How AI-related risks are disclosed to clients
- Material limitations or known failure modes of AI systems

Practical Implication

Every slide in this presentation that describes an AI trading system corresponds to a disclosure obligation if that system is deployed by a registered investment advisor. Design your systems with disclosure in mind from day one.

What Is Responsible Deployment?

Beyond Compliance: Genuine Responsibility

Compliance with regulations is the floor, not the ceiling. Responsible deployment means actively designing AI systems that are safe, fair, and beneficial — going beyond what is legally required.

Principles of Responsible Deployment

- **Test exhaustively before deploying:** paper trade for at least 6 months across different market regimes
- **Disclose proactively:** tell clients what AI is doing on their behalf in plain language
- **Design for failure:** plan for all failure modes, not just happy paths
- **Build in humility:** assume your model is wrong more often than backtests suggest
- **Monitor continuously:** performance in the live environment may diverge from expectations; detect this early

The Responsible Deployment Hierarchy

1. **Safety:** the system cannot cause catastrophic harm
2. **Reliability:** the system does what it's supposed to consistently
3. **Fairness:** the system doesn't systematically disadvantage certain groups
4. **Transparency:** stakeholders can understand what the system does
5. **Privacy:** client data is protected
6. **Accountability:** when things go wrong, responsibility is clear
7. **Sustainability:** the system's aggregate effects on markets are considered

Mitchell et al. (2019): Model Cards for Model Reporting

Model cards are standardized documentation accompanying ML models that describe the model's intended use, performance characteristics, limitations, and ethical considerations.

What a Finance AI Model Card Should Include

1. **Model description:** architecture (e.g., FinBERT fine-tuned on earnings calls), training data, training period
2. **Intended use:** which decisions this model is designed to support; which decisions it is *not* suitable for
3. **Performance metrics:** accuracy, Sharpe ratio, Deflated Sharpe ratio, out-of-sample test results
4. **Limitations:** known failure modes (volatile markets, regime changes), data cutoff date
5. **Biases:** what biases may exist in training data or model outputs
6. **Ethical considerations:** who is affected by the model's decisions; how are adverse impacts mitigated
7. **Maintenance:** how often is the model updated; who is responsible for monitoring

Regulatory Alignment

Model cards align with EU AI Act documentation requirements and ESMA's model validation expectations. They are becoming standard industry practice for responsible AI deployment.

The Non-Negotiable Requirements

All significant AI-assisted financial decisions must have two properties: **human oversight** (a qualified person could review and override the decision) and **audit trail** (a complete record of what the AI did and why).

Effective Human Oversight

- Oversight must be *meaningful*: the human must have enough information and time to actually review the decision, not just rubber-stamp it
- “Automated approval” with a 2-second human review window is not meaningful oversight
- Oversight frequency should scale with decision size and novelty
- Humans reviewing AI decisions must understand what the AI is doing well enough to catch errors

What Audit Trails Must Capture

For every significant AI-driven decision:

- Timestamp (microsecond precision)
- Input data used (with versions)
- Model outputs (including confidence scores)
- Reasoning trace (ReAct-style: what did the agent think and what tools did it call?)
- Who approved (if human oversight required)
- Actual market outcome (for retrospective review)

Audit Trail Storage

Regulators can request audit trails for decisions going back years. Storage and retrieval of audit data must be designed from the start — not added reactively after a regulatory inquiry.

A Unique Ethical Challenge for Academic Researchers

When researchers at MBZUAI, MIT, Stanford, or CMU publish a new AI technique for trading markets with open-source code, they do not just advance science — they change the competitive dynamics of financial markets. Is this an ethical responsibility?

The Positive Case for Publishing

- Open research enables competition: small funds can access the same tools as large ones
- Academic publication is slower than industry; by the time a paper is published, the edge may already be arbitrated
- Democratization of AI tools *reduces* the institutional advantage over retail
- Scientific progress requires open dissemination

The Negative Case

- Publishing a working bias-exploiting strategy enables thousands of funds to exploit retail investors more efficiently
- When many funds adopt the same published strategy, the resulting coordinated behavior can be destabilizing (herding risk)
- The academic community benefits (citations) while markets bear the systemic risk cost
- Some published strategies (e.g., momentum crash strategies) may accelerate market crashes

Our Position

Publication with responsible disclosure: describe the method; share the code; but include a discussion of the systemic implications, market capacity, and ethical considerations — as this tutorial does.

The Full Pipeline: One Diagram

From Raw Information to Trading Decision



Every Component Has Been Covered

Each box corresponds to one or more sections of this tutorial. The full system is complex but each piece is understandable in isolation.

What This Tutorial Adds to Your ML Toolkit

1. **Finance is a uniquely hard ML domain:** adversarial, non-stationary, low signal-to-noise. Do not be fooled by clean backtests.
2. **The behavior-first chain is this tutorial's core thesis:** Tetlock (2007) showed text predicts returns → FinBERT reads behavioral signals from earnings calls → RAG grounds those signals in verified documents → agents act on grounded analysis → multi-agent debate reduces overconfidence. Every module serves this one chain.
3. **RAG before generation:** for any factual finance task, retrieval-augmented generation dramatically outperforms pure LLM inference. Build this in from the start.
4. **Multi-agent diversity:** multi-agent disagreement is a signal, not a problem. Use it for uncertainty quantification and regime detection.
5. **Multiple testing matters:** most published factors are false discoveries. Apply DSR, PBO, and Walk-Forward testing before claiming a result.
6. **The ethics are your problem too:** if you publish methods that are deployed in markets, you share responsibility for their effects. Think carefully about publication externalities.

What This Tutorial Adds to Your Quant Toolkit

1. **LLMs read behavior, not just prices:** the durable insight is behavioral — FinBERT on earnings calls captures disposition effects and overconfidence *as they happen in text*. This is fundamentally different from adding another price feature.
2. **But hallucinations are catastrophic:** never use raw LLM outputs for factual claims in trading. Always use RAG, CRITIC, or external verification.
3. **Agentic AI changes the research process:** multi-step financial analysis can be automated with agents — fundamentally changing the productivity of quant research teams.
4. **Multi-agent conflict = regime signal:** implement multi-agent disagreement metrics as a volatility predictor and regime change early warning system.
5. **Your backtest is probably lying:** apply all 10 checklist items, plus the Kelly criterion for sizing, before deploying. The research-to-production gap (Section J) is real and large.
6. **Regulatory pressure is increasing:** design for compliance from the start. Explainability, audit trails, and kill switches are not optional; they are required.

What This Tutorial Tells You About AI in Markets

1. **Correlated AI is a systemic risk:** multiple firms using similar AI on similar data creates dangerous herding behavior. Diversity-of-model requirements should be explored analogously to diversity-of-bank requirements in Basel III.
2. **LLM hallucinations are a regulatory concern:** AI-generated financial analysis containing material errors can constitute misleading communication to investors. Require disclosure of AI use and its known limitations.
3. **Speed mandates are necessary:** circuit breakers designed for human-speed markets need to be redesigned for microsecond AI-speed markets. The 2010 Flash Crash is not the last such event.
4. **The audit trail requirement is achievable:** modern AI systems can log their reasoning (ReAct-style traces). Require this; it enables genuine after-the-fact oversight without preventing innovation.
5. **International coordination is critical:** AI risks in finance are global; national regulation creates regulatory arbitrage. FSB coordination on minimum standards is essential.
6. **The retail investor gap is growing:** consider whether mandated investment in financial literacy (using AI for investor education) should accompany the growth of institutional AI.

Foundations

- Kahneman & Tversky (1979): Prospect Theory
- Glosten & Milgrom (1985): LOB with information
- Jegadeesh & Titman (1993): Momentum
- Tetlock (2007): Media and stock markets
- Loughran & McDonald (2011): Finance sentiment words

LLMs and RAG

- Devlin et al. (2019): BERT
- Brown et al. (2020): GPT-3
- Lewis et al. (2020): RAG
- Wu et al. (2023): BloombergGPT
- Asai et al. (2024): Self-RAG

Multi-Agent and RL

- Yao et al. (2023): ReAct
- Du et al. (2023): Multi-agent debate
- Shinn et al. (2023): Reflexion
- Nevmyvaka et al. (2006): RL for execution
- Liu et al. (2020): FinRL

Backtesting and Ethics

- Lopez de Prado (2018): Advances in Financial ML
- Bailey et al. (2016): Probability of Backtest Overfitting
- Harvey et al. (2016): Cross-section of expected returns
- FSB (2024): AI in Financial Services
- Mitchell et al. (2019): Model Cards

Assignment 1: Complete FinBERT Sentiment Pipeline

What You Build

1. Download earnings call transcripts from SEC EDGAR full-text search
2. Preprocess and label sentences using Financial PhraseBank
3. Fine-tune FinBERT for 3 epochs
4. Evaluate: accuracy, precision, recall, F1 per class
5. Run inference on a new earnings call
6. Compute correlation between sentiment scores and 5-day and 20-day returns
7. Plot calibration curve of model confidence vs. actual accuracy

Technical Stack

- Python, PyTorch, HuggingFace Transformers
- Google Colab (free T4 GPU)
- FinBERT: ProsusAI/finbert on HuggingFace Hub
- SEC EDGAR: sec-api or EDGAR full-text search
- Evaluation: scikit-learn metrics
- Finance data: yfinance for return computation

Key Learning Outcome

You will observe that FinBERT has significantly stronger sentiment signal than a general BERT model, but the correlation between sentiment and returns is still modest ($r \approx 0.10\text{--}0.20$). This is realistic and expected — single signals are weak, and combination is required.

Assignment 2: Multi-Agent Disagreement as a Market Signal

What You Build

1. Three LLM-based agents (Bull, Bear, Macro) using different signal sets
2. Disagreement metric: cosine distance between agent recommendation vectors
3. Historical disagreement time series on S&P 500 2018–2023
4. Regime labeling: align with NBER recession dates and VIX regimes
5. Show: disagreement spikes at regime transitions (2020 March, 2022 bear)
6. Conflict resolution: compare weighted voting vs. conservative default
7. Evaluate: risk-adjusted returns, max drawdown

Expected Results

- Disagreement correlates 0.4–0.6 with VIX (uncertainty measure)
- Conservative default during high-disagreement periods reduces maximum drawdown by 20–40%
- At cost of approximately 5–10% reduction in total return
- Sharpe ratio: conservative default $\approx$ weighted voting Sharpe (better risk/return tradeoff)

Key Research Question

Is multi-agent disagreement a better volatility signal than standard VIX? This is an open research question that you can contribute to!

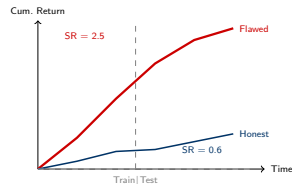
Colab Notebook 3: Flawed Backtest Repair

Assignment 3: Find and Fix the Five Flaws

What You Repair

Start with a "promising" strategy (Sharpe 2.5) and systematically fix:

1. Survivorship bias (load historical S&P 500 membership)
2. Look-ahead bias (add reporting delay to earnings data)
3. Zero transaction costs (add 5bp round-trip)
4. No embargo period (add 20-day buffer between train/test)
5. Post-hoc metric selection (pre-commit to Sharpe vs. buy-and-hold)



Sharpe After Each Fix

Version	SR
All flaws	2.5
Fix survivorship	1.8
Fix look-ahead	1.3
Add tx costs	0.9
Add embargo	0.7
Fix metric sel.	0.6

The Take-Home Message

A Sharpe of 0.6 may still be worth pursuing — it is a realistic, honest estimate. The dangerous path is deploying based on the Sharpe of 2.5. This exercise builds visceral intuition for why rigorous testing methodology matters.

Where the Frontier Is

Foundational Questions

- How can LLMs be reliably calibrated for financial factual claims?
- What is the optimal architecture for a financial RAG system that handles mixed text, tables, and charts?
- Can conformal prediction provide meaningful coverage guarantees for LLM-generated financial forecasts?
- How much does domain pre-training (BloombergGPT) vs. fine-tuning (FinGPT) matter as base models scale?

Applied Questions

- Is multi-agent disagreement a better volatility predictor than VIX?
- Can LLM-based agents detect earnings manipulation before regulators?
- What is the systemic risk contribution of AI herding, and can it be measured in real time?
- How should kill switch criteria be calibrated for LLM-based agents? (Different from deterministic algorithms)
- Can behavioral biases in LLMs (LLMs trained on human text may inherit human biases) create systematic trading errors?

An Invitation

If any of these questions excite you, we welcome collaboration. Contact details are on the final slide. The tutorial materials, data, and code are available for academic research use.

The Bigger Picture: AI in Markets Is Just Beginning

Where We Are in the Trajectory

We are at the **beginning** of AI in financial markets, not the middle or end. The tools covered in this tutorial — FinBERT, RAG, agentic AI, multi-agent systems — represent the state of the art in 2025. By 2030, the landscape will look completely different.

What Will Likely Happen Next

- **Larger, more capable models:** LLMs that genuinely understand financial theory and reason through complex scenarios
- **Real-time multi-modal:** vision, audio (live earnings call audio), and text combined in real time
- **Fully autonomous agents:** end-to-end investment process from data to execution with minimal human input for routine decisions
- **AI-designed markets:** exchanges optimizing their microstructure using AI

What Remains Constant

- The behavioral biases of human investors (evolved over millennia, not going away)
- The fundamental market microstructure (supply, demand, information asymmetry)
- The statistical challenges (multiple testing, non-stationarity, low signal-to-noise)
- The ethical obligations (fairness, systemic stability, transparency)

The Enduring Skill

Understanding the *principles* — behavioral psychology, information economics, rigorous statistics, ethical reasoning — will matter more than knowing any specific tool, because tools will change every 2 years.

Sections A–B: Markets & Microstructure

Glosten & Milgrom (1985). J. Financial Economics.
Kyle (1985). Econometrica.
Easley, Kiefer, O'Hara & Paperman (1996). J. Finance.
Huang & Polak (2011). LOBSTER data. Working paper.
Nevmyvaka et al. (2006). ICML.
Almgren & Chriss (2001). J. Risk.
Brunnermeier & Pedersen (2009). Rev. Financial Studies.

Section C: Behavioral Finance

Kahneman & Tversky (1979). Econometrica.
De Bondt & Thaler (1985). J. Finance.
Jegadeesh & Titman (1993). J. Finance.
Barber & Odean (2008). Rev. Financial Studies.
Baker & Wurgler (2007). J. Economic Perspectives.
Daniel, Hirshleifer & Subrahmanyam (1998). J. Finance.
Barberis, Shleifer & Vishny (1998). J. Financial Economics.

Section D: Financial NLP

Tetlock (2007). J. Finance.
Loughran & McDonald (2011). J. Finance.
Yang et al. (2020). FinBERT. arXiv.
Devlin et al. (2019). BERT. NAACL.
Chen et al. (2021). FinQA. EMNLP.
Masry et al. (2022). ChartQA. ACL.

Sections E–F: LLMs and RAG

Vaswani et al. (2017). Attention Is All You Need. NeurIPS.
Brown et al. (2020). GPT-3. NeurIPS.
Wei et al. (2022). Chain-of-Thought. NeurIPS.
Lewis et al. (2020). RAG. NeurIPS.
Wu et al. (2023). BloombergGPT. arXiv.
Asai et al. (2024). Self-RAG. ICLR.
Manakul et al. (2023). SelfCheckGPT. arXiv.

Sections G–H: Agents & Multi-Agent

Yao et al. (2023). ReAct. ICLR.
Shinn et al. (2023). Reflexion. NeurIPS.
Madaan et al. (2023). Self-Refine. NeurIPS.
Du et al. (2023). Improving Factuality via Debate. arXiv.
Lowe et al. (2017). MADDPG. NeurIPS.
Liu et al. (2020). FinRL. arXiv.

Sections I–K: Backtesting, Risk, Ethics

Lopez de Prado (2018). Advances in Financial ML.
Bailey et al. (2016). PBO. J. Portfolio Management.
Harvey, Liu, Zhu (2016). J. Finance.
Lundberg & Lee (2017). SHAP. NeurIPS.
FSB (2024). AI in Financial Services.
Mitchell et al. (2019). Model Cards. FAT*.

Thank You!

Questions and Discussion Welcome

Contact

Zangir Iklassov

Zangir.Iklassov@mbzuai.ac.ae
MBZUAI, Abu Dhabi, UAE

Hachem Madmoun

MBZUAI, Abu Dhabi, UAE

Jean-Jacques Duhot (remote contributor)

Brevan Howard Asset Management
London, UK

Salem Lahlou

MBZUAI, Abu Dhabi, UAE

Tutorial Resources

- **Slides:** Available at KDD 2026 proceedings
- **Colab Notebooks:** GitHub (link in proceedings)
- **Data:** SEC EDGAR (public), Yahoo Finance (free)
- **Code:** Apache 2.0 license

Acknowledgments

We thank the KDD 2026 program committee for accepting this tutorial. We gratefully acknowledge support from MBZUAI and the quantitative research team at Brevan Howard Asset Management.

**“The future belongs to those who understand
both humans and machines.”**